ShopSteward

A Personal Operations Agent for Small Merchants

Group Project Report

ISY5001 Intelligent Reasoning Systems · Practice Module
NUS Institute of Systems Science

Group 40

Member	Student ID
Ying Hao	A0326313N
Liu Kairui	A0353368W
Feng Tangyu	A0356670X
Zhang Junming	A0355479L

Project links

Source code: https://github.com/yhnapoleon/ShopSteward/tree/submission
Promotion video: https://youtu.be/F_eJIUC-ETU
System design video: https://youtu.be/3pUuZFNzfN4

October 2026

Contents

Executive Summary 5

1 Business Case, Market Research and Literature Review 6

1.1 Introduction 6

1.2 The restocking decision of a small shop 6

1.3 Current practice: tools for the next order 8

1.4 Related research 9

1.5 Objectives and scope 11

2 System Design 13

2.1 Design overview: a neuro-symbolic, event-driven purchasing agent 13

2.2 Event-driven architecture and a digital twin 15

2.3 Demand forecasting: a gradient-boosted and neural ensemble 16

2.4 Routine restocking: constraint-filtered candidate search 17

2.5 Supplier failure: recovery cases and simulation-based optimisation 18

2.6 Multi-agent sourcing: orchestrator–worker with verified quote cards 20

2.7 Explanation layer: a configurable expert team 23

2.8 Agent runtime and human-in-the-loop 24

2.9 Context engineering and memory 26

2.10 Hybrid, agentic RAG over business documents 28

2.11 Learning loops 30

3 System Development and Implementation 35

3.1 Data and knowledge preparation 35

3.2 Technology stack and process layout 41

3.3 Model access 43

3.4 The system in use: four scenarios 45

3.5 Implementation challenges and solutions 47

3.6 Testing and verification 50

4 Experiments 52

4.1 Method 52

4.2 Forecast model selection 53

4.3 Planning search: daily rule, beam search and enumeration 54
4.4 Rules or a language model for ordering decisions 57
4.5 Post-training: SFT and GRPO 59
4.6 One reviewer per supplier or one reader for all 60
4.7 The sourcing loop end to end 61
4.8 Four orchestration modes for the explanation 62
4.9 Prompt evolution with GEPA 63
4.10 Retrieval 64
4.11 Quotation accuracy and rule reuse 65
4.12 Delegated work across time: completion and recovery 66
4.13 Summary: design decisions and their evidence 67
5 Findings and Discussion 69
5.1 Key findings 69
5.2 Deviations from the proposal 70
5.3 Limitations 71
5.4 Future work 72
5.5 Conclusion 73
References 74
Appendix A Project Proposal 76
Appendix B Mapped System Functionalities against MR, RS and CGS 86
B.1 Machine Reasoning (MR) 86
B.2 Reasoning Systems (RS) 87
B.3 Cognitive Systems (CGS) 88
B.4 Techniques learned beyond the courses 90
Appendix C Installation and User Guide 91
C.1 Requirements 91
C.2 Installation 91
C.3 Configuration and databases 91
C.4 Models and the forecast service 92
C.5 Starting and stopping 93

C.6 Using the system 94
C.7 Troubleshooting 95

Executive Summary

A small shop pays for stock before it knows whether the quantity is right. The supplier's answer, the delivery, the sales, the settlement of proceeds and a revised demand estimate arrive over the following days, and each can make the order too small or too large. Current tools help at the moment of ordering; the days after it are left to an owner who is also serving customers.

ShopSteward is a personal operations agent for small merchants, built here as a purchasing assistant for one shop. The owner states a goal and its constraints once. The system forecasts demand, proposes a purchase for the owner to approve, follows the order until the goods arrive and re-plans whenever sales, deliveries or demand change. When a supplier fails, it reads the other suppliers' documents, compares every feasible recovery order and prepares the order message the owner sends. Algorithms compute every quantity and every amount of money. Language models read documents, interpret the owner's requests and explain results. The owner approves every purchase.

The system uses all four of the course's technique groups: business rules with event-driven re-planning, informed search and simulation-based optimisation, a demand-forecasting ensemble trained on public retail data, and language-model agents with bounded memory and document retrieval. It also improves itself from code-scored feedback, by post-training an open model and by evolving its own instructions.

Every experiment is scored by code; Table 4-16 and Figure 5-1 collect the results. Re-planning on each update cut unmet demand by 45%. A language model decided 99.0% of ordering requests correctly against 61.5% for keyword rules, and post-training brought an 8B open model to 94.0%. The sourcing loop recovered from a supplier failure in 12 of 12 test worlds, and a delegated task survived restarts and resumed from timers and business events. The system has been validated in a simulated shop; use with merchants is the next step (Chapter 5).

1 Business Case, Market Research and Literature Review

1.1 Introduction

Restocking a small shop means committing cash before the information that would justify the commitment exists. The owner pays for stock against sales that have not yet happened, with money the rest of the business also needs, and the facts that settle whether the quantity was right arrive one at a time over the following days: whether the supplier accepts, when the goods land, how many units sell, when the proceeds become spendable, whether the demand estimate still holds. Each arrival can require the decision to be revised. The purchase is decided once; keeping it right until the goods are on the shelf and the cash is accounted for is continuing work, and in a shop run by one person that work is assigned to no one.

ShopSteward is a personal operations agent for small merchants. In this project it takes on that continuing work for one purchasing decision at a time: it holds the owner's goal and constraints, recomputes the plan as each fact arrives, finds and prices another supplier when the first fails, and leaves every spending decision with the owner. The project's thesis is that the value of an intelligent system in a small shop lies more in this follow-through than in the first calculation, and that the follow-through can be built so that every step is open to the owner's inspection. Chapter 4 tests this directly: Section 4.3 compares a better first calculation with faster follow-through.

This chapter makes the case for building it. Section 1.2 describes the restocking decision and shows why its structure calls for a reasoning system rather than a calculator. Section 1.3 reviews how owners make the decision today and where current tools stop. Section 1.4 reviews the research the design draws on. Section 1.5 states the objectives and the scope that Chapters 2 to 5 design, build, test and discuss.

1.2 The restocking decision of a small shop

The decision maker. The target user is an owner who controls purchasing for a shop with identifiable stock, unit costs and quantities, whether the shop sells online, from a neighbourhood storefront or both. The owner has no planning staff and limited cash, and makes purchasing decisions in the gaps of a day spent selling. Business is run from records the owner already keeps: a stock count, a cash balance, the orders placed and the suppliers' terms.

The decision. For each product the owner decides, over a horizon of about a week, how many units to buy and from whom. The inputs are the units on hand, the units already ordered and still in transit, the cash that can be spent, the demand expected over the coming days and the supplier's price, minimum order, pack size and lead time. The two errors are not symmetric. Under-ordering forfeits the sales that fall in the stockout, and those sales do not return. Over-ordering converts cash into stock that may take weeks to sell, and for a small business cash is the constraint that binds first. Across retail as a whole, IHL Group estimates the cost of out-of-stocks and overstocks at US$1.73 trillion a year, about 6.5% of sales [1]; the figure describes the size of the problem, not the market for this system. In a 2026 survey of 400 inventory and operations professionals in the United States, from founder-run businesses to enterprise teams, 44% reported a stockout at least once a month [2]. The example that runs through this report makes the decision concrete: with 20 units on hand, 60 expected over the coming week and a ¥300 cash floor, the owner orders 40; when the estimate later rises to 70, 20 more are needed (Sections 2.4 and 3.4).

Why the decision is hard. Four properties make it more than arithmetic. Table 1-1 states each property, what it demands of any system that takes the decision on, and the course technique group that supplies it. The third property is the one that separates this problem from a one-off calculation, and Figure 1-1 draws it as a timeline.

Table 1-1. Four properties of the restocking decision and the reasoning each one demands.

Property of the decision	What it demands	Technique group	Sections
Demand is revealed after the decision, and many of a small shop's products sell on only some days	A forecast learned from sparse, irregular history, with its error measured	Knowledge discovery and data mining	2.3, 4.2
The feasible orders are few and constrained: the cash floor, the supplier's minimum order and pack size, the window in which the goods are needed	Rules that remove infeasible candidates, and a search that ranks the rest and can justify its choice	Decision automation; business resource optimisation	2.4, 2.5, 4.3
The decision carries state through time: an order in transit, proceeds not yet spendable, a revised estimate, a supplier's refusal, each changing the right quantity	Re-planning triggered by each event, with the owner's approval bound to the plan that was shown	Decision automation (event-driven rules)	2.2, 4.3.2, 4.12
The inputs that matter most are written in language: the owner's goal, the suppliers' documents, a supplier's reply	Reading documents into verified fields, interpreting requests, and explaining decisions in terms the owner can check	Cognitive systems	2.6–2.10, 4.4, 4.6

The unattended interval. Between the order and the shelf lie several days in which the state of the decision changes repeatedly and no one is responsible for re-examining it. Each delivery notice, sale or revised estimate should prompt a check of stock, cash and the open order; in a shop run by its owner the check happens when there is time, and the cost of a missed check appears later as an empty shelf or a cash shortfall. This interval is what the project addresses: a decision made well once and then left to look after itself.

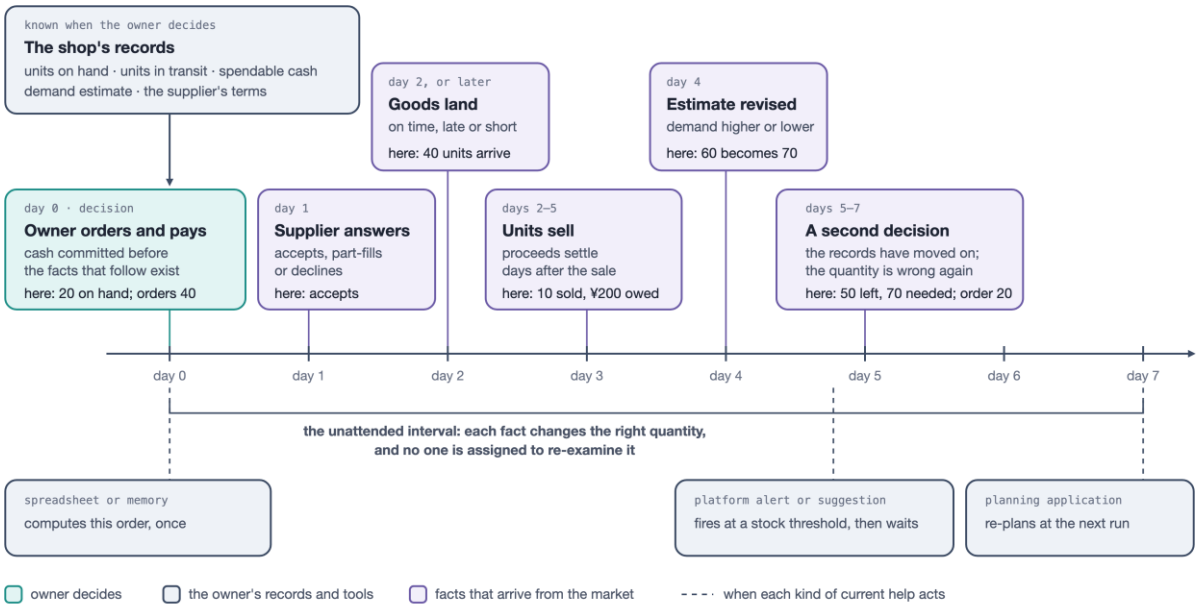


Figure 1-1. One purchase and the facts that arrive after it. The owner (teal) commits cash with the records held at the time (grey); the facts that settle whether the quantity was right (purple) arrive over the following days, and each changes the right quantity. The dashed marks show when each of the three kinds of current help, reviewed in Section 1.3, acts. The numbers are those of the promotion example used in Chapters 2 and 3; the days are illustrative.

1.3 Current practice: tools for the next order

Owners meet this decision today with three kinds of help; the capabilities below are taken from the vendors' own material. What they share is the unit of work: each produces a recommendation, an alert or a plan, and leaves the follow-through to the owner.

Memory, spreadsheets and messages. The most common method is the owner's own record. In Forthcast's 2026 interviews with 13 ecommerce operators, 11 planned replenishment in spreadsheets and six named cash tied up in stock, or poor visibility of it, as a main concern [3]; the panel is small and vendor-run, so the counts indicate practice rather than measure it. A spreadsheet computes the first order correctly. It does not notice a late delivery, a demand revision or a supplier's refusal, and it does not recompute itself.

Alerts and suggestions inside the selling platform. Point-of-sale and commerce platforms have added replenishment features. Square introduced Smart Stock Alerts in 2021 to warn sellers before an item runs out [4]. Shopify's Sidekick answers a question about what to reorder from the store's sales history and drafts the purchase order, which Shopify then tracks through receiving [5]; since December 2025 its Pulse feature also proposes up to five actions without being asked [6]. These features answer the question of the next order and keep the owner in control at each step. Each is set off by the owner, by a stock threshold or by Pulse's own review, and each starts a new recommendation; none stays attached to an order once it is placed.

Planning applications. Applications such as Prediko connect a demand plan to incoming stock, a budget and purchase orders, and support re-planning when the plan drifts [7]. They are built around a planning routine run by a person whose job includes planning. For an owner-operator the limiting resource is that person's attention, so a stronger planning tool helps only when someone runs it.

Table 1-2. Existing approaches, their documented capabilities and what ShopSteward adds.

Existing approach	Documented capability	What ShopSteward adds	Where it is tested
Spreadsheets and the owner's records [3]	Correct arithmetic for one order at a time	A goal that stays active for days and is re-planned on every sale, delivery and demand revision, without the owner's prompting	Sections 4.3.2, 4.12
Platform alerts and suggestions [4][5][6]	Reorder prompts and suggested quantities; purchase orders drafted for approval	Candidate orders filtered by the cash floor and supplier terms, with the rule that removed each rejected candidate; approval bound to the exact plan the owner saw	Sections 4.3.1, 4.12
Planning applications [7]	Demand-to-purchase plans, budgets, re-planning by a planner	Recovery when a supplier fails: alternative suppliers read from their documents, every feasible order compared over the week, the order message prepared for the owner	Sections 4.6 to 4.8
All of the above	Figures and recommendations	An explanation of each decision in which every number traces to the solver's result; corrections the owner makes once and the system applies again	Sections 4.8, 4.9, 4.11

The comparison does not claim that these products cannot follow up; it records what their documentation describes and what this project builds and measures in a defined setting. The difference lies in what sets the help in motion and whether it continues. Each kind above starts from the owner, a schedule or a stock threshold, and each time produces a fresh recommendation. ShopSteward starts from every fact that bears on an open order, and carries the same task from one fact to the next, until the goods arrive, the cash is accounted for and the owner has been told what happened. A further difference follows from where small shops buy: the suppliers an owner turns to when the usual one fails are seldom on the selling platform at all, and their terms exist only in documents and messages, so reading those documents is part of the task (Section 2.6).

1.4 Related research

The design rests on research from seven directions. Each paragraph names the finding the project relies on and the section where the project applies or tests it.

Humans, operations research and language models. Baek, Fu, Ma and Peng study multi-period inventory control with operations-research heuristics, language-model agents and human decision makers. On a benchmark of more than 1,000 instances, combining operations-research methods with language models outperformed either alone, and in a controlled experiment human–AI teams earned higher average profits than humans or agents working alone [8]. InvAgent applies language-model agents to multi-echelon inventory decisions directly, with the model choosing quantities [9]. Duan, Hu and Jiang have the model instead elicit the parameters a store manager knows and hand the optimisation to a solver; against a frontier model used end to end, the hybrid cut total inventory cost by 32.1%, and giving the end-to-end model perfect information

did not close the gap [10]. ShopSteward takes the complementarity reading: the solver computes quantities, the model handles the owner's language and the supplier's documents, and the owner decides. The owner's part has evidence of its own: Dietvorst, Simmons and Massey find that people go on using an imperfect algorithm when they can adjust its output, even slightly [11]; ShopSteward lets the owner revise a plan (Section 4.4) and have a correction applied to later work (Section 4.11). Section 4.4 also measures what the model contributes to the ordering decision, and Finding 1 in Section 5.1 reads the result.

Why the arithmetic stays with algorithms. Language models state the principle of a calculation more reliably than they carry it out; on symbolic and arithmetic tasks the failures trace to execution rather than to missing knowledge [12], and in the inventory study above the bottleneck was computation, not information [10]. This is the basis for the division of labour in Section 2.1: every quantity and every amount of money is computed by code, and the model's role is to decide which computation to call and to explain the result.

Forecasting intermittent retail demand. The M5 competition on Walmart's daily sales established gradient-boosted trees as the strongest point-forecast family for retail series, with Tweedie objectives suited to the many zero-sales days of slow-moving items [13][14]. Later work shows the limits of a single objective on overdispersed counts and argues for distributional treatment of the counts [15]. Section 2.3 follows this evidence with an ensemble of three LightGBM models with L1 and Tweedie losses and a neural network that learns the daily shape; Section 4.2 reports its accuracy against a moving-average baseline.

Search under constraints. The cash floor has a classical counterpart: Chao, Chen and Wang characterise the optimal ordering policy of a retailer whose orders are limited by the cash on hand [16]. Replenishment over several days is a sequence of order decisions, and the number of sequences grows exponentially with the menu of quantities and the horizon. Informed search, in which partial sequences are scored by an evaluation function and only the best are extended, is the course's answer to this growth [17]; simulation-based optimisation, which scores each candidate by simulating its consequences, extends the same idea to recovery orders, whose outcome depends on the day each offer's goods arrive. Sections 2.4 and 2.5 apply both; Section 4.3 compares the search with a daily rule and with exhaustive enumeration.

Agents that read documents. Orchestrator–worker systems, in which a lead agent delegates independent reading tasks to parallel workers, outperformed a single agent by 90.2% on an internal research evaluation whose tasks spread evidence across many sources; a separate agent then checks that each claim is attributed to its source [18]. Supplier documents have this structure: each supplier's terms are independent of the others. Section 2.6 gives each supplier a reviewer agent whose quoted excerpts are checked against the source document; Section 4.6 measures the gain, and Section 4.8 shows the opposite case, where evidence that must be read together is best read in a single call.

Learning from executable feedback. Two recent methods improve a model from feedback that a program can compute. Group Relative Policy Optimisation trains model weights from rewards on executed outcomes [19]; GEPA evolves instruction text by reflecting on scored failures and keeping a Pareto front of candidates, and reaches or exceeds reinforcement learning with far fewer rollouts [20]. ShopSteward scores every experiment by code rather than by a judge model (Section 4.1), so the same predicates that evaluate the system can train it: Section 4.5 applies GRPO to the ordering decision and Section 4.9 applies GEPA to the explanation instructions.

Durable agents with bounded memory. A delegated task must outlive the conversation that started it. The agent runtime therefore follows the pattern of a state machine with durable checkpoints and interrupts for human input [21], adds leased workers so that a task survives the process that was running it, and keeps the owner's standing instructions as small, versioned

memory entries in the style of recent open agent frameworks [22]. Sections 2.8 and 2.9 describe the runtime and the context assembled for each model call; Section 4.12 checks that a task survives restarts and resumes from timers and business events.

Where this project departs from the work reviewed. The studies above compare language models, solvers and people on decisions taken at fixed periods, each with that period's information. A small shop adds the interval between periods: facts that arrive on an open order, alternative suppliers whose terms exist only in documents, and an owner who approves each purchase. ShopSteward carries one decision through that interval with the division of labour these studies support, and Chapter 4 measures each part of it.

Taken together, the research supports a system in which learned forecasts supply demand, rules and search choose orders, parallel agents read documents, a single durable agent carries the task and the owner approves every purchase. Section 2.1 maps these components to the course's four technique groups, and Appendix B maps them to the topics of the three courses.

1.5 Objectives and scope

The project's central question is whether the owner of a small shop can hand over a continuing purchasing task and return to work that has been carried out correctly. The report answers it through seven objectives; the first six are tested in Chapter 4 and the seventh is shown in Table 2-1 and Appendix B.

1. Keep every purchase within the owner's cash floor and the supplier's terms, and find the best feasible order with a search that is compared against a simple daily rule and against exhaustive enumeration (Section 4.3.1).
2. Re-plan when sales, deliveries or demand estimates change, and measure the effect on unmet demand against planning on a fixed daily schedule (Section 4.3.2).
3. Establish what a language model contributes to the ordering decision over keyword rules, and whether that capability can be trained into an open model the shop could run itself (Sections 4.4 and 4.5).
4. Recover from a supplier's failure by reading the alternative suppliers' documents into verified offers, comparing every feasible recovery order and explaining the choice in terms the owner can check (Sections 4.6 to 4.9).
5. Let the owner correct the system once, in ordinary words, and have the correction applied to later work (Section 4.11).
6. Keep a delegated task alive across conversations, process restarts, timers and business events, with every purchase bound to the plan the owner approved (Section 4.12).
7. Integrate the course's four technique groups (business rules, informed search, data mining and cognitive systems) on one shared business state (Table 2-1, Appendix B).

Scope. The delivered system serves one shop. Each delegated task plans one product against the store's cash, with the product's usual supplier and, when that supplier fails, alternative suppliers found in the documents on file. The forecasting model is trained on public retail data and forecasts from a product's own sales history; sales, deliveries and supplier responses come from a market simulator that stands in for the shop's channels, so that every experiment can be repeated; purchases are executed in that simulated market, and orders to suppliers found through documents are written by the system and sent by the owner. Goods are modelled by stock, unit cost and quantity over a seven-day horizon. Pricing, promotion design, planning across several products under one cash reserve and connection to a live store system are outside this scope; Section 5.4 shows how each builds on a mechanism already in place, and Section 5.3 discusses the limitations that follow from the scope. The four technique groups are all in use, against the

three planned in the proposal; Section 5.2 records this and the other extensions beyond the proposal.

Chapter 2 describes the system that meets these objectives, beginning with the division of labour between algorithms, language models and the owner. Chapter 3 covers the data, the implementation and the system in use. Chapter 4 tests each design decision, and Chapter 5 reads the results together and sets out what comes next.

2 System Design

ShopSteward is a purchasing assistant for a small shop. The owner hands it a goal such as “keep the weekend promotion stocked and keep at least ¥300 in cash”, and the system carries that goal across days: it forecasts demand, proposes purchases the owner can approve, follows each order until the goods arrive, and recomputes the plan whenever sales, demand or deliveries change. When a supplier cannot deliver, it reads the alternative suppliers’ documents, compares every feasible order over the coming week, and prepares the order message the owner sends.

This chapter describes how the system is built. Section 2.1 gives the overall design and maps it to the course’s technique groups. Section 2.2 covers the event-driven core. Sections 2.3 to 2.7 follow the two decision pipelines: routine restocking (forecasting and planning) and supplier failure (recovery optimisation, multi-agent sourcing and explanation). Sections 2.8 to 2.10 describe the language-model layer that runs across both pipelines: the agent runtime, context engineering and memory, and document retrieval. Section 2.11 covers the three ways the system improves itself: post-training the ordering policy, evolving its prompts, and distilling skills from experience.

2.1 Design overview: a neuro-symbolic, event-driven purchasing agent

The design rests on one division of labour. **Algorithms compute every quantity and every amount of money**: a forecasting ensemble predicts demand, a rule-filtered search chooses routine order quantities, and a simulation-based optimiser compares recovery orders. **Language models handle language**: they read supplier documents, retrieve passages, interpret the owner’s requests and explain the algorithms’ results. The owner approves every purchase; the approval request goes from the interface straight to the business service, and the agent has no tool that approves or places an order.

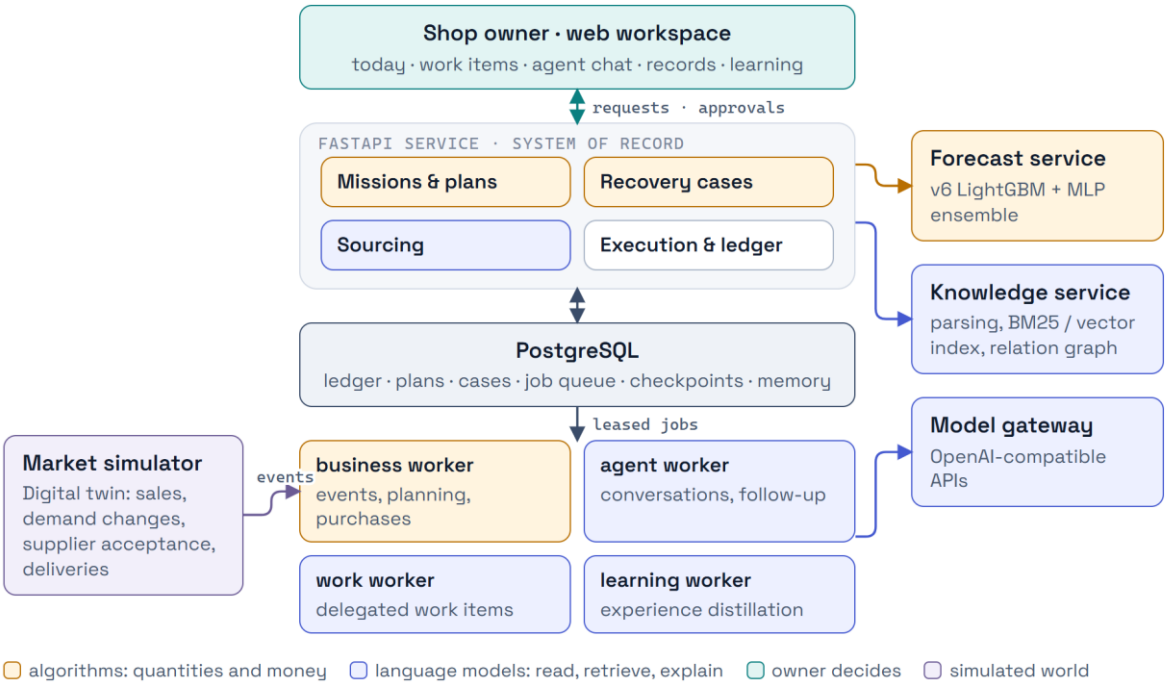


Figure 2-1. System architecture. Amber components compute quantities and money; blue components use language models; the owner (teal) approves purchases; the simulator (purple) stands in for the market.

Figure 2-1 shows the components. A FastAPI service is the single system of record: it owns the ledger, missions and plans, recovery cases, sourcing runs and purchase execution, all stored in PostgreSQL. Long-running work goes through a PostgreSQL job queue served by four worker pools. Three services sit beside it: the v6 forecast service, a knowledge service for document parsing and retrieval, and a model gateway that speaks the OpenAI-compatible Chat Completions and Responses APIs. A market simulator acts as a digital twin, producing sales, demand revisions, supplier acceptances and deliveries as an event stream.

The two decision pipelines map to the two business situations the owner faces:

- **Routine restocking** (Sections 2.3–2.4): forecast → candidate search → owner approval → execution → events → re-planning.
- **Supplier failure** (Sections 2.5–2.7): recovery case → simulation-based optimisation → multi-agent sourcing of alternative offers → re-optimisation → expert explanation → owner approval → assisted order.

Table 2-1 maps these components to the four technique groups of the course. All four are covered; Appendix B gives the detailed mapping to the MR, RS and CGS courses.

Table 2-1. Course technique groups and the components that implement them.

Technique group	ShopSteward components	Sections
Decision automation (business rules, knowledge-based reasoning)	Rule filters on every candidate order (cash floor, minimum order, pack size, arrival window, task cap); plan binding and re-validation before execution; evidence policy; five-action ordering policy	2.2, 2.4, 2.8
Business resource optimisation (informed search, evolutionary computing)	Candidate search in the routine planner; beam search over multi-day order sequences; simulation-based optimisation over recovery orders; GEPA's reflective evolutionary search over prompt components	2.4, 2.5, 2.11
Knowledge discovery and data mining	Gradient-boosted and neural demand-forecasting ensemble; experience distillation from operation episodes	2.3, 2.11
Cognitive systems (knowledge base and graph, natural-language interface)	Multi-agent document reading; conversational agent with memory; hybrid retrieval with a business-relation graph; natural-language constraints parsed into a typed task frame	2.6–2.10

2.2 Event-driven architecture and a digital twin

Purchasing unfolds over days, so ShopSteward is **event-driven**: every change in the world, and every timer, re-enters the same planning loop, whether or not a conversation is open.

Job queue. Work is stored as rows in a `job_runs` table. Each row carries a deduplication key and a request hash, so the same request enqueued twice becomes one job. Workers claim jobs with `SELECT … FOR UPDATE SKIP LOCKED`, take a 30-second lease and renew it every 10 seconds, so several workers share one queue without a separate broker. Retryable failures are retried with exponential backoff, up to three attempts in total. Partial unique indexes allow only one active check per mission, so a burst of events triggers one re-plan. Table 2-2 lists the four worker pools.

Table 2-2. Worker pools and the jobs they run.

Pool	Jobs
business	event sync every 5 s, freshness check, mission checks, purchase execution and reconciliation, recovery-case follow-up
agent	conversational agent runs and background follow-up (Section 2.8)
work	delegated work items (“hand me a task”), including quotation processing
learning	experience distillation batches, one at a time (Section 2.11.3)

Digital twin. The simulator produces four event types: `SALE_RECORDED`, `DEMAND_REVISED`, `PURCHASE_ACCEPTED` and `GOODS_RECEIVED`. The business worker pulls them every 5 seconds from a per-store cursor, checks that each page of up to 100 events continues the sequence without gaps, and applies them to the ledger idempotently in sequence order. Each batch that changes the store wakes the store’s missions.

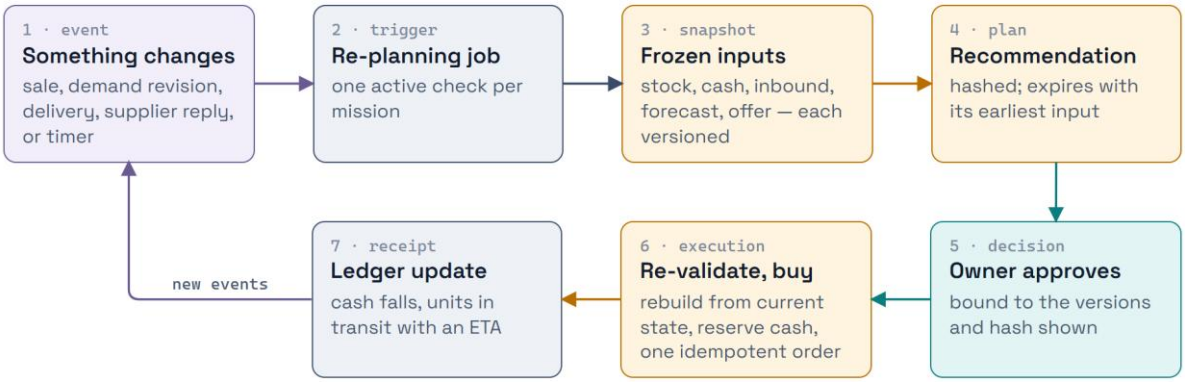


Figure 2-2. The plan lifecycle. Every event re-enters the loop; the owner’s approval is bound to the exact plan that was shown.

Plan lifecycle. Figure 2-2 shows the loop. A mission is re-planned on its timer (every 5 to 3,600 seconds), after each batch of business events, when a purchase succeeds or fails, when a supplier replies or goods are recorded as received, when a recovery case closes, or on request. Each plan freezes its inputs into a snapshot (stock, cash, inbound orders, forecast and offer, each

with a version), evaluates the candidates (Section 2.4) and stores the result with a SHA-256 hash over the snapshot, candidates, recommendation, proposed purchase and expiry time. A plan expires at the earliest of 15 minutes, the forecast's validity and the offer's validity. If a new trigger arrives with an unchanged input hash, the existing plan is reused.

The owner's approval carries the plan version, the store's state version and the proposal hash that were on screen. The execution job then rebuilds the plan from current state; if the rebuild produces the same candidates and purchase, it reserves the cash and sends a single purchase to the simulator with the action ID as its idempotency key, then reconciles the result after 5, 10, 30 and 60 seconds. One purchase per store can be in flight at a time. An accepted purchase lowers cash, adds the units to inventory in transit with an expected arrival time and writes a ledger entry; a later `GOODS_RECEIVED` event moves them on hand, which in turn triggers the next plan.

This loop is what lets a delegated task outlive the conversation that started it. In the promotion scenario, the owner confirms 40 units and leaves; when demand is later revised from 60 to 70, the revision event alone produces a new plan for 20 more units and puts it in front of the owner. In simulation, checking the plan as soon as a demand revision or a delivery notice arrives, instead of at the next daily run, cut unmet demand by 45% (Section 4.3.2).

2.3 Demand forecasting: a gradient-boosted and neural ensemble

Demand comes from a **weighted ensemble of three LightGBM models and one multilayer perceptron**, trained on the Walmart M5 dataset (300 product–store series, training data up to 2016-03-27). The service receives 84 to 374 days of complete daily sales for one product in one store and returns a forecast for each of the next seven days.

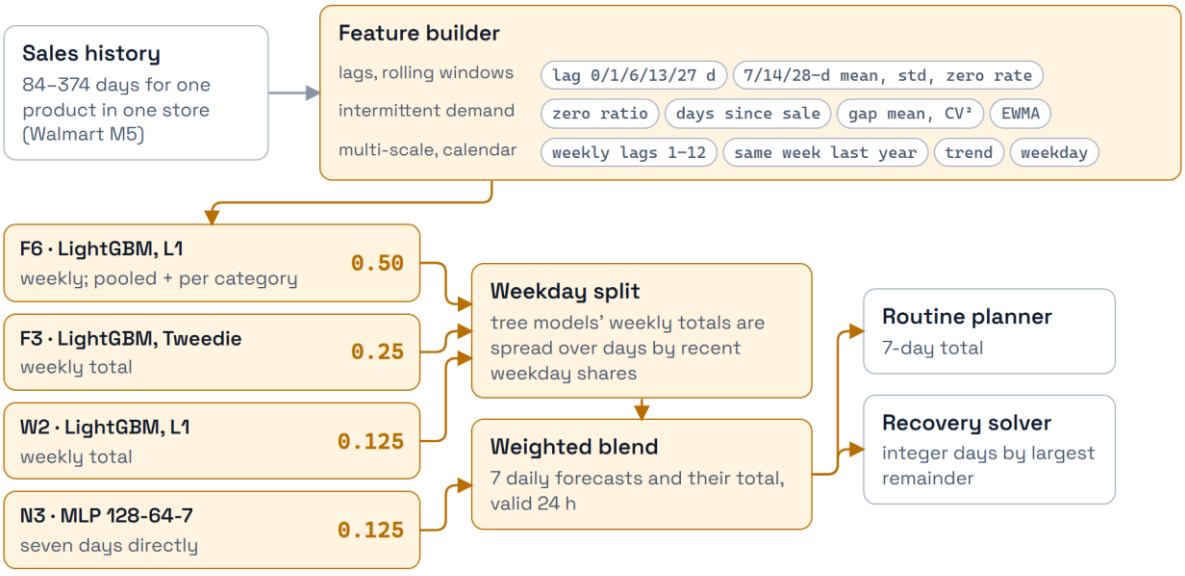


Figure 2-3. The v6 forecasting ensemble. Tree models predict weekly totals that are spread over the week by recent weekday shares; the MLP predicts the seven days directly.

Features. The feature builder describes each series at several time scales: lags of 0, 1, 6, 13 and 27 days; rolling 7-, 14- and 28-day means, standard deviations and zero rates; intermittent-demand statistics (share of zero days, days since the last sale, mean and squared coefficient of variation of the gaps between sales, exponentially weighted averages of sale occurrence and size); weekly lags 1–12; 112-, 182- and 364-day levels; trend; the same week last year; and calendar position.

Models. Table 2-3 lists the four components. They differ in loss and in target so that their errors differ: L1 loss estimates the median and is robust to promotion spikes, the Tweedie loss models the many zero days of slow-moving items, and the neural network learns the daily shape directly where the trees learn weekly totals.

Table 2-3. Components of the v6 ensemble.

Component	Model and loss	Target	Weight
F6	LightGBM, L1; a pooled model plus one per product category	weekly total	0.50
F3	LightGBM, Tweedie (variance power 1.3)	weekly total	0.25
W2	LightGBM, L1	weekly total	0.125
N3	MLP, 128 → 64 → 7	seven daily values	0.125

The tree models' weekly totals are spread over the days by the series' recent weekday profile,

$$\text{share}(d) = \frac{\sum \text{sales on weekday } d \text{ over the last 4 weeks} + 1}{\sum \text{sales over the last 28 days} + 7},$$

and the four daily vectors are blended with the weights above. The output is seven daily forecasts, their total rounded up, and a 24-hour validity. The routine planner uses the seven-day total as remaining demand. The recovery solver needs whole units per day, so it allocates the total with the **largest-remainder method**: each day receives the integer part of its forecast, and the leftover units go to the days with the largest fractional parts.

Over an eight-week review of 2,400 product-weeks, the ensemble's weekly WAPE was 34.98%, against 37.26% for a 28-day moving average (Section 4.2).

2.4 Routine restocking: constraint-filtered candidate search

The routine planner is a **finite-candidate search with business-rule filtering**. A mission's policy lists the order quantities to consider (up to 20 values; the demo mission uses 0, 20, 40 and 80). For each candidate quantity *q* the planner computes:

- the **inventory position**: stock on hand plus inbound orders expected to arrive before the end of the seven-day horizon;
- the **shortage**: max(0, demand − position − *q*);
- the spend and the cash left after the purchase.

Five rules then decide feasibility: cash after the purchase must stay at or above the mission's cash floor; a non-zero order must meet the supplier's minimum, be a multiple of the pack size and arrive before the horizon ends; and it must not exceed any quantity cap the owner set for the task (Section 2.9). Among feasible candidates the planner recommends the one with the smallest shortage and, among those, the smallest quantity: the cheapest order that closes the gap.

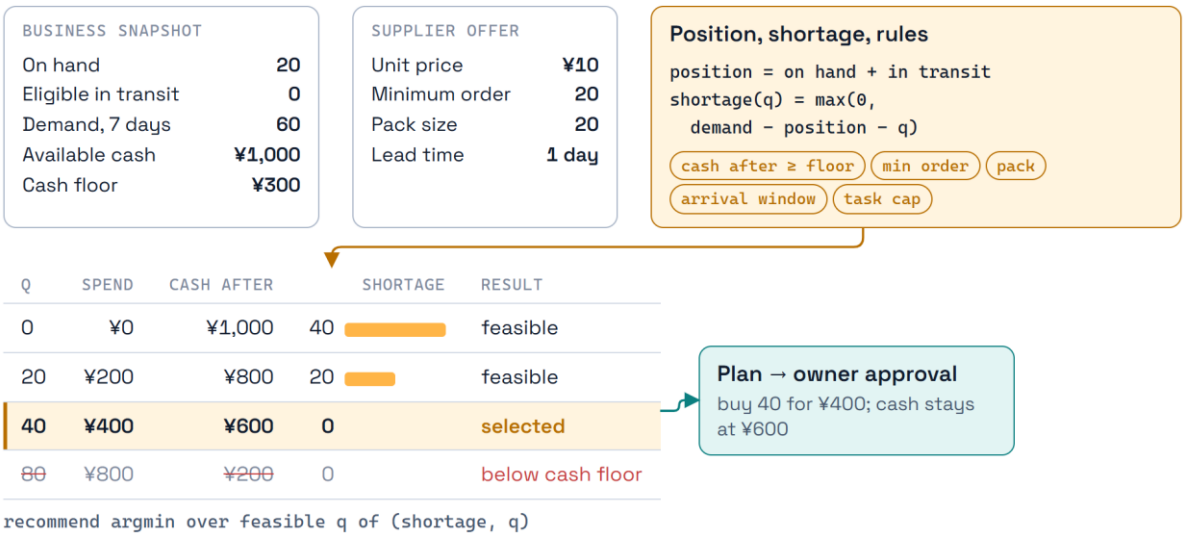


Figure 2-4. A routine plan for the promotion scenario. Buying 40 closes the gap; buying 80 would also close it but would leave ¥200, below the ¥300 cash floor.

Figure 2-4 shows the promotion scenario from the project proposal. With 20 units on hand, demand of 60 and ¥1,000 in cash, 40 units is the smallest order that leaves no shortage; 80 units is rejected because it would leave ¥200. The candidate table is stored with the plan and shown to the owner, so every recommendation comes with the alternatives it beat and the rule that removed each rejected one. The same rules run again at execution time, when the plan is rebuilt from current state before any money moves.

Planning across several days: beam search over order sequences. Buying the right total still leaves a timing decision: an early order protects availability but carries stock and commits cash sooner. The multi-period planner therefore searches over **sequences of daily orders** for the rest of the horizon and ranks each complete sequence lexicographically, by projected unmet demand, then stock carried in unit-days, then total quantity, under the same cash-floor and delivery rules. It uses **beam search**, an informed search: each partial plan is extended by every permitted quantity, infeasible extensions are dropped, and each survivor is completed with the daily replenishment rule to estimate its consequences; the best B partial plans go on to the next day, and the best completed plan seen is returned. Only the first order is proposed; the plan is rebuilt after the next event. With four permitted quantities, seven days give 16,384 sequences, few enough for exhaustive enumeration to serve as the exact reference. On 30 test scenarios beam search with $B = 1$ reached the enumeration optimum in all 210 decisions with about 7 plans evaluated per decision instead of 3,121, and it stayed near 15 evaluations as the menu grew to six quantities, where enumeration needs 279,936 (Section 4.3).

2.5 Supplier failure: recovery cases and simulation-based optimisation

When the usual supplier fails, the owner opens a **recovery case**. Its input is a budget, seven days of demand (either the owner's scenario or the v6 forecast), an optional quantity cap and, optionally, suppliers to use or exclude. Each mission has at most one open case.

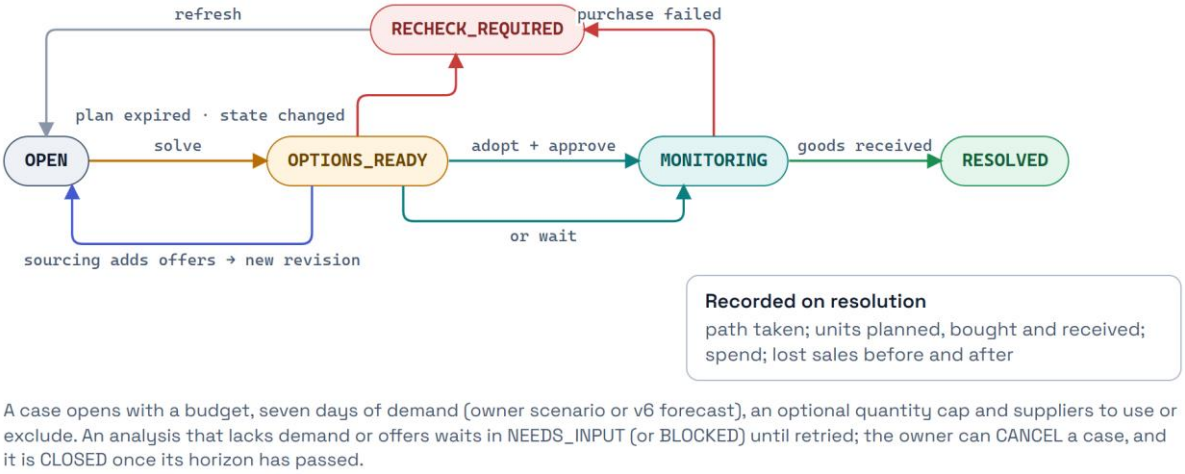


Figure 2-5. Recovery case lifecycle. Sourcing adds offers and returns the case to OPEN as a new revision; a change of state or a failed purchase sends it to RECHECK_REQUIRED.

Case lifecycle. Figure 2-5 shows the states. Solving an OPEN case produces OPTIONS_READY. The owner either adopts a candidate, which becomes a recovery plan on the same approval and execution path as a routine plan (Section 2.2), or chooses to wait; both lead to MONITORING. When the bought goods (or the original orders) have arrived, the case is RESOLVED and records its outcome: the path taken, units planned, bought and received, spend, and lost sales before and after the decision. An expired plan, a change in business state or a failed purchase moves the case to RECHECK_REQUIRED, from which a refresh returns it to OPEN. Suppliers that declined, failed or only partly filled an order are excluded when the case is analysed again.

Simulation-based optimisation. The recovery solver evaluates every candidate order by simulating the next seven days, then picks the best by a lexicographic objective. The candidates are "wait" plus, for each offer (at most three), every pack multiple from the minimum order up to what the budget allows:

$$q \in \{q_{\min}, q_{\min} + p, \dots\}, \qquad q \le \left\lfloor \frac{\min(\text{budget},\ \text{cash} - \text{floor})}{\text{unit price}} \right\rfloor,$$

where $q_{\min}$ is the minimum order rounded up to the pack size p, giving at most 100 quantities per offer and 301 candidates in total. For each candidate the solver steps through the seven settlement days:

1. existing inbound orders arrive on their expected dates, and the proposed order arrives after its lead time;
2. available = stock + arrivals;
3. served = min(available, demand); lost = demand − served;
4. closing stock = available − served.

Lost sales are not back-ordered. Each candidate is also checked against the offer's validity, minimum order, pack size, arrival within the horizon, budget, cash floor and task cap, and every failed check is recorded as a rejection reason. The feasible candidates are ranked by (lost units, spend, closing stock, supplier, quantity): fewest lost sales first, then the cheapest way to achieve them.

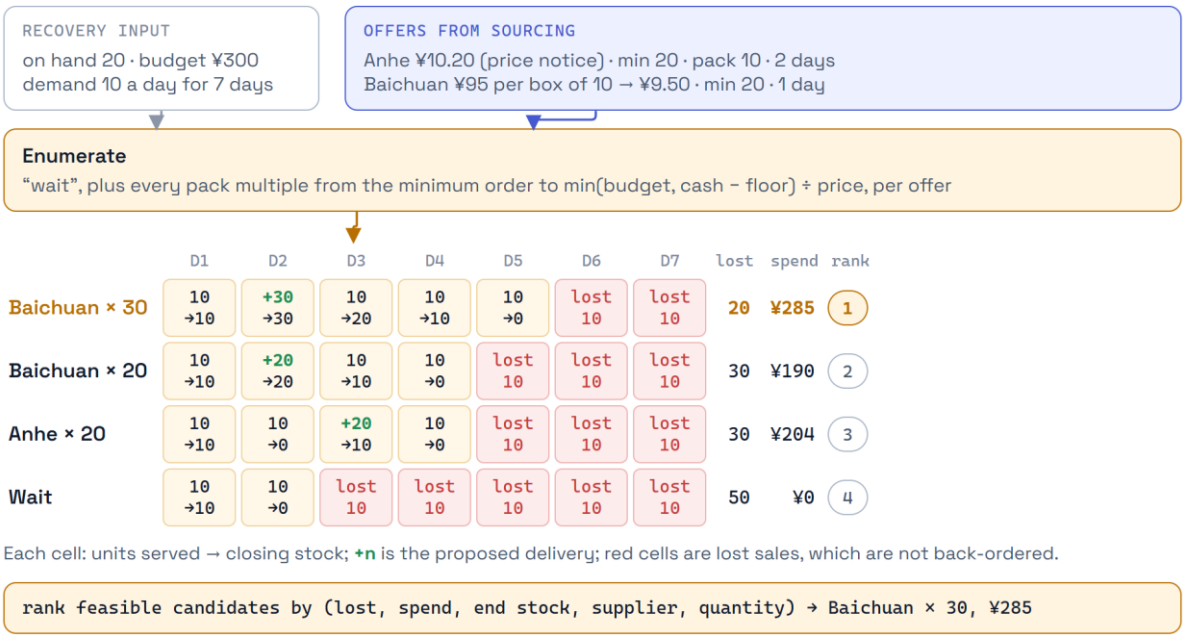


Figure 2-6. Recovery solver output for the supplier-failure scenario (solver run on the demo's offers). Ordering 30 units from Baichuan loses the fewest sales within the ¥300 budget.

Timing decides the outcome, which is why the solver simulates each day. In Figure 2-6 the shop has 20 units and sells 10 a day. Anhe's 20 units arrive on day 3 and Baichuan's on day 2, so both 20-unit orders cover days 3 and 4 and lose 30 units; Baichuan's is ranked higher only because it costs less. Thirty units from Baichuan, the largest order the budget allows, cover day 5 as well and lose 20. Stepping through the days makes each order's arrival date, the orders already inbound and the sales lost before a delivery part of the comparison, which a single total-quantity check cannot see. Because the candidate set is bounded and small, the solver enumerates it completely and returns the exact optimum under these rules.

2.6 Multi-agent sourcing: orchestrator–worker with verified quote cards

Recovery needs alternative offers, and those live in supplier documents: quote sheets, price notices, cooperation terms and delivery records, often in different formats and sometimes contradicting each other. ShopSteward reads them with an **orchestrator–worker multi-agent pipeline**. Each worker is a **ReAct agent** that returns a **structured quote card with verbatim citations**, and a **verifier with model escalation** checks every card before the solver uses it.

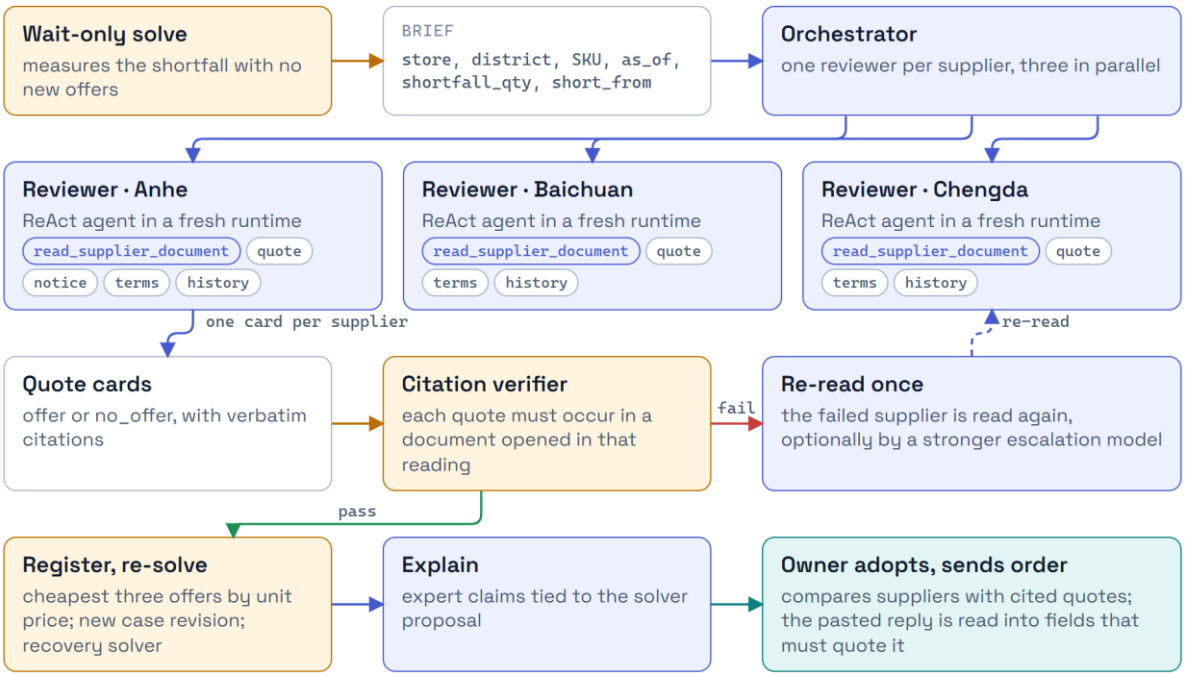


Figure 2-7. The sourcing pipeline. One reviewer agent per supplier reads that supplier's documents; verified cards become offers, and the case is solved again.

Brief and fan-out. Sourcing starts by solving the case with no new offers. The "wait" result gives the shortfall, which becomes a brief: store, delivery district, SKU, date, shortfall quantity and the first day of shortage. The orchestrator starts one reviewer per supplier, at most three in parallel, and skips suppliers the case excludes.

Reviewer agents. Each reviewer runs in a fresh LangGraph runtime and sees only its own supplier. Its context holds the brief, the supplier's name, the list of that supplier's documents and the output contract; the documents themselves are read through its single business tool, `read_supplier_document`. Its instructions come from a versioned text bundle (Section 2.11.2): open every document, report only what the documents support, and leave the choice of supplier and quantity to the solver.

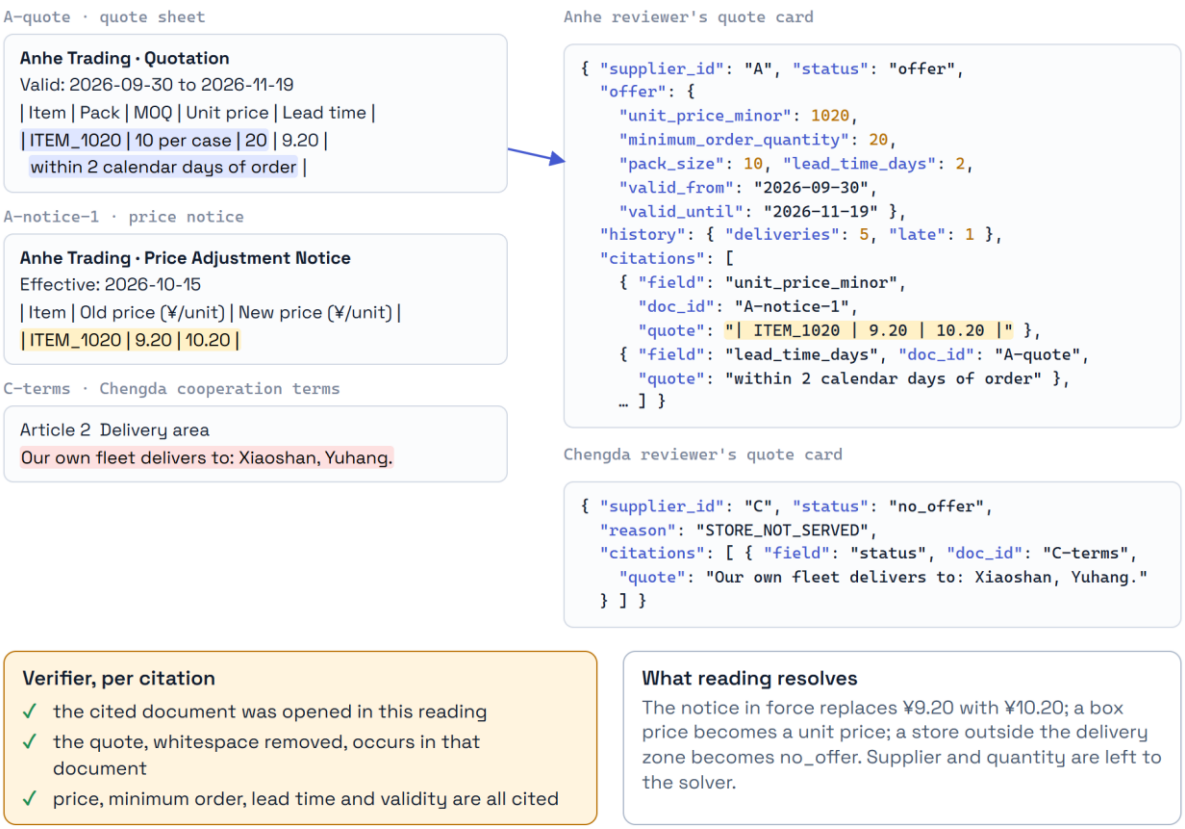


Figure 2-8. Reading the demo's supplier documents. The Anhe reviewer applies the price notice in force (¥9.20 → ¥10.20) and cites it; the Chengda reviewer returns no_offer because the store lies outside its delivery zone.

Quote cards. Each reviewer returns a card (Figure 2-8). A card's status is either `offer` or `no_offer`. An offer card gives the unit price in fen, minimum order, pack size, lead time in days, validity dates and version, plus a delivery history (deliveries and late deliveries). A no_offer card gives the reason: the SKU is not covered, the store is outside the delivery zone, or supply is withdrawn. Every decision-relevant field carries a citation `{field, doc_id, quote}`: the unit price, minimum order, lead time and end of validity on an offer, and the status on a no_offer.

Verification and escalation. The verifier checks each citation in code: the cited document must have been opened during this reading, and the quoted text, with whitespace removed, must occur in it. A supplier whose card fails is read once more, optionally by a stronger escalation model. This cascade lets a fast model do most of the reading while a stronger model handles the cases it gets wrong.

Re-optimisation and explanation. Verified offers are registered with the case, cheapest three by unit price, as a new case revision. The recovery solver runs again with these offers (Section 2.5), and the expert team explains the result (Section 2.7). The owner sees one row per supplier with its best feasible order, the cited quotes, the explanation and a seven-day chart of demand, shortfall and arrivals.

Assisted ordering. Suppliers found through documents are not connected to the system, so the owner sends the order. ShopSteward generates the order message from a template (Chinese or English). When the owner pastes the supplier's reply back, a model reads it into fields (decision, quantity, unit price, total, delivery date and order reference), and each field is kept only if its

quoted text occurs in the reply. The reply is then classified as confirmed, declined, partially filled or needing attention; a confirmed delivery date replaces the planned arrival time.

Why one reviewer per supplier. Isolation keeps one supplier's terms from leaking into another's card, for example applying Anhe's price notice to Baichuan, and it lets the readers run in parallel. In a direct comparison with a small model, reading each supplier in its own call produced 33 of 42 cards with every decision-relevant field correct, against 21 of 42 when one call read all suppliers together; with a stronger model the per-supplier arrangement got all 42 right in a median of 6 seconds per world, against 13 seconds. With the full pipeline, the sourcing loop resolved all 12 evaluation worlds end to end (Sections 4.6–4.7).

2.7 Explanation layer: a configurable expert team

The solver's output is a ranked table; the owner needs to know why the top candidate wins, which days still run short and what the alternatives cost. The explanation comes from a **team of role-specialised expert agents that emit typed claims**, which **code merges** into one answer under one of **four orchestration modes**.

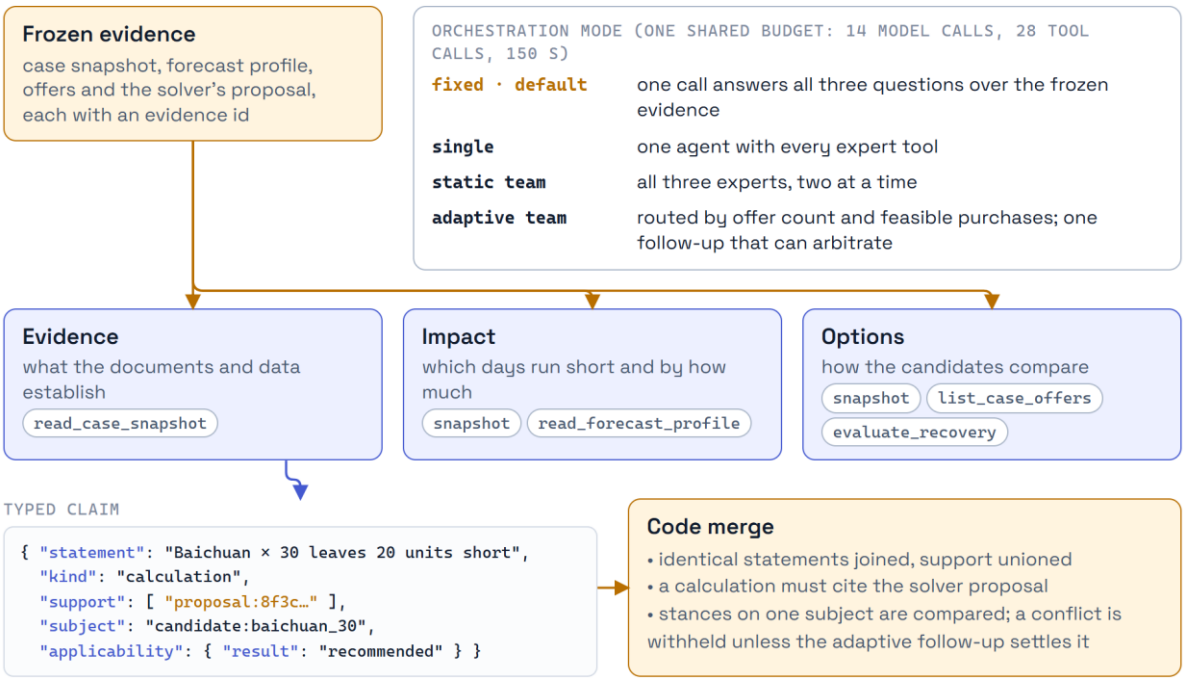


Figure 2-9. The expert team. Each claim states its kind and cites evidence IDs; calculations must cite the solver's proposal, and code merges the claims.

Evidence and roles. Each explanation works on frozen evidence: the case snapshot, the forecast profile, the offers and the solver's proposal, each with an evidence ID. Three roles each answer one question: **evidence** (what the documents and data establish), **impact** (which days run short and by how much) and **options** (how the candidates compare). Each role has read-only tools over the frozen evidence: `read_case_snapshot`, `read_forecast_profile`, `list_case_offers` and `evaluate_recovery`. The role questions are components of a versioned text bundle that GEPA can evolve (Section 2.11.2).

Typed claims. Every expert answers with claims of the form `{statement, kind, support, subject, applicability}`. The kind is one of `source_fact`, `interpretation`, `assumption` or

`calculation`, and `support` lists evidence IDs that were supplied or returned by a successful tool call.

Code merge. Claims are joined by code. Identical statements are merged and their support combined. A calculation must cite the solver's proposal, so every number in the explanation traces to the solver. Claims about the same subject are grouped by stance ("recommended" counts as a kind of "feasible"); if two stances disagree, the contested claims are withheld unless the adaptive mode's follow-up settles them.

Table 2-4. Orchestration modes. All modes share one budget of 14 model calls, 28 tool calls and 150 seconds per explanation.

Mode	How it runs
fixed (default)	One call answers all three role questions over the frozen evidence
single	One agent holds every expert tool and may use the whole budget
static team	All three experts run, at most two at a time
adaptive team	Evidence and impact always run; options is added when there are several offers, or one offer with no feasible purchase. One follow-up round can call a further expert and arbitrate a conflict.

Routing in the adaptive mode reads structured signals from the case (the number of offers and of feasible purchases). On 24 held-out cases the fixed pipeline answered 23 correctly with a single model call each, the best of the four modes, so it is the default; the team modes remain available as configuration (Section 4.8).

2.8 Agent runtime and human-in-the-loop

The owner talks to ShopSteward through a conversational agent and can also hand it a task to finish in the background. Both run on a **LangGraph state machine with durable checkpoints**, designed for **human-in-the-loop** work: the agent can stop to ask a question and resume, its plan changes go to the owner for approval, and it can act on its own when the business changes.

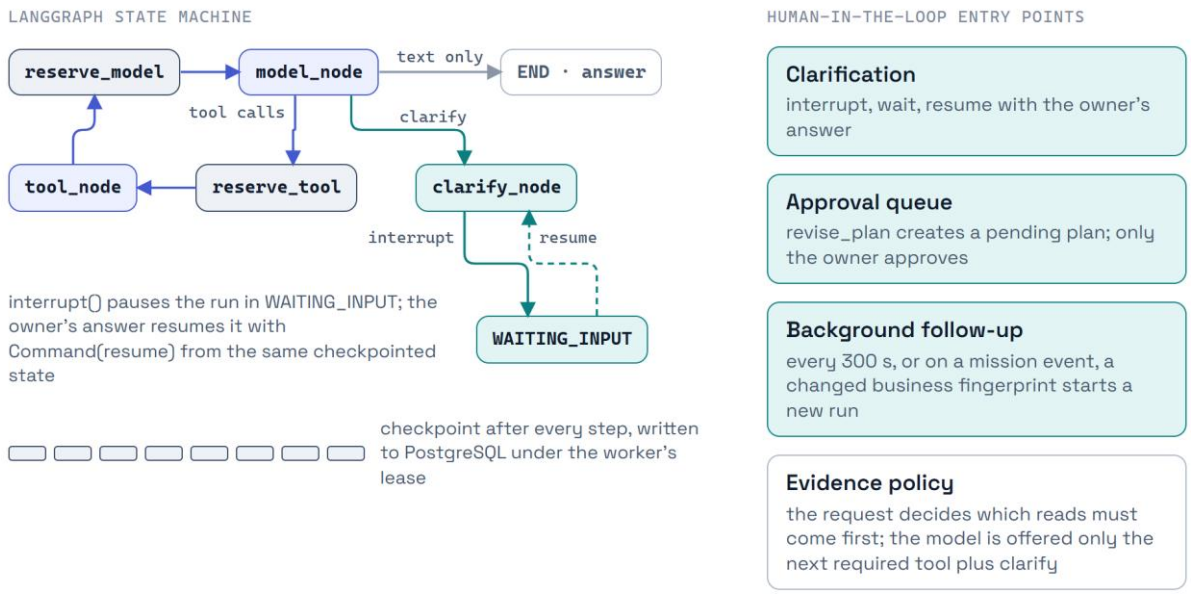


Figure 2-10. The agent runtime. Every step is checkpointed; clarification interrupts the graph and resumes from the same state; three entry points bring the owner into the loop.

State machine. The graph has five nodes (Figure 2-10). `reserve_model` records the model call before it is sent; `model_node` calls the model; `reserve_tool` records each requested tool call; `tool_node` executes it; `clarify_node` asks the owner a question. The loop ends when the model answers without tool calls. Each tool call is keyed by its invocation ID and argument hash, so a resumed run replays a step without repeating its effect.

Checkpoints. After every step the graph state is written to PostgreSQL, under the worker's lease. If a worker stops, another worker resumes the run from its last checkpoint.

Three human-in-the-loop entry points.

- *Clarification*. When the agent needs information, `clarify_node` raises a LangGraph `interrupt`; the run enters WAITING_INPUT. The owner's answer is posted with the interrupt's ID, and `Command(resume=…)` continues the run from the same checkpointed state.
- *Approval queue*. The agent can propose a changed plan with `revise_plan`, which supersedes the pending plan with a new one capped at the owner's quantity. The new plan joins the owner's approval queue like any other plan.
- *Background follow-up*. When follow-up is switched on for a conversation, the agent worker checks every 300 seconds (configurable from 60 seconds to a day) whether the store's business fingerprint has changed. The fingerprint is a digest of the store's state version, the mission's version, the current plan, the current purchase action and the mission status. A change, or a new mission event, starts a background run with a follow-up message, and the result appears in the conversation.

Tools and evidence policy. Table 2-5 lists the agent's tools. The **evidence policy** decides from the wording of each request which reads must come first: a question about stock requires `get_dashboard`, one about the plan requires `get_plan`, one about supplier terms requires `search_documents`, an explicit memory instruction requires `memory_edit`, and so on. The runtime then offers the model only the next required tool, plus `clarify`, and requires a tool call. The answer is therefore built on current data in the order the question needs.

Table 2-5. The conversational agent's tools.

Kind	Tools
Read the business	`get_dashboard`, `get_plan`, `get_forecast`, `get_mission`, `get_action`, `read_timeline`, `read_sales_summary`, `get_recovery_cases`, `get_recovery_case`, `read_experiences`
What-if	`evaluate_plan` (read-only evaluation with a quantity cap)
Change, subject to approval	`revise_plan`, `request_check`, `analyze_recovery_case`
Memory	`memory_edit`
Documents	`search_documents`, `read_document_evidence` (Section 2.10)
Ask the owner	`clarify`

Delegated work items. A task handed over with "hand me a task" is processed by the work worker. It claims the item under a 300-second lease, replays the item's conversation, and records each completed tool call with its result, so a restart continues where it stopped without repeating a write. The item ends as RESULT_READY, WAITING_INPUT (with a question for the owner) or BLOCKED, and a new message from the owner sends it back for another round.

The ordering policy as a restricted action space. For research and post-training, the core ordering decision is formalised as a **restricted policy**: given the owner's request and the current plan, choose exactly one of five actions with exact arguments — `evaluate_plan(plan_id, max_purchase_qty)`, `revise_plan(plan_id, max_purchase_qty, expected_mission_version)`, `clarify(question)`, `handoff(reason)` or `no_action`. Ten task types cover what-if trials, explicit revisions, negated changes, missing quantities, references to earlier turns, withdrawals, invalid quantities, out-of-scope requests, ambiguous units and removing a cap. Because every action can be executed against the real backend, the policy can be evaluated and trained by its effect on business state. Over 200 dialogues, a language model made every decision in the dialogue correctly, action and arguments, in 99.0% of them, against 61.5% for a hand-written rule policy (Section 4.4); Section 2.11.1 describes how a smaller open model is trained for the same contract.

2.9 Context engineering and memory

What a model sees decides what it can do, so every model call assembles its context by **layered context assembly**, and writes a **manifest** of what went in. Long-term knowledge about the owner lives in a **bounded, versioned memory** modelled on Nous Research's Hermes agent.

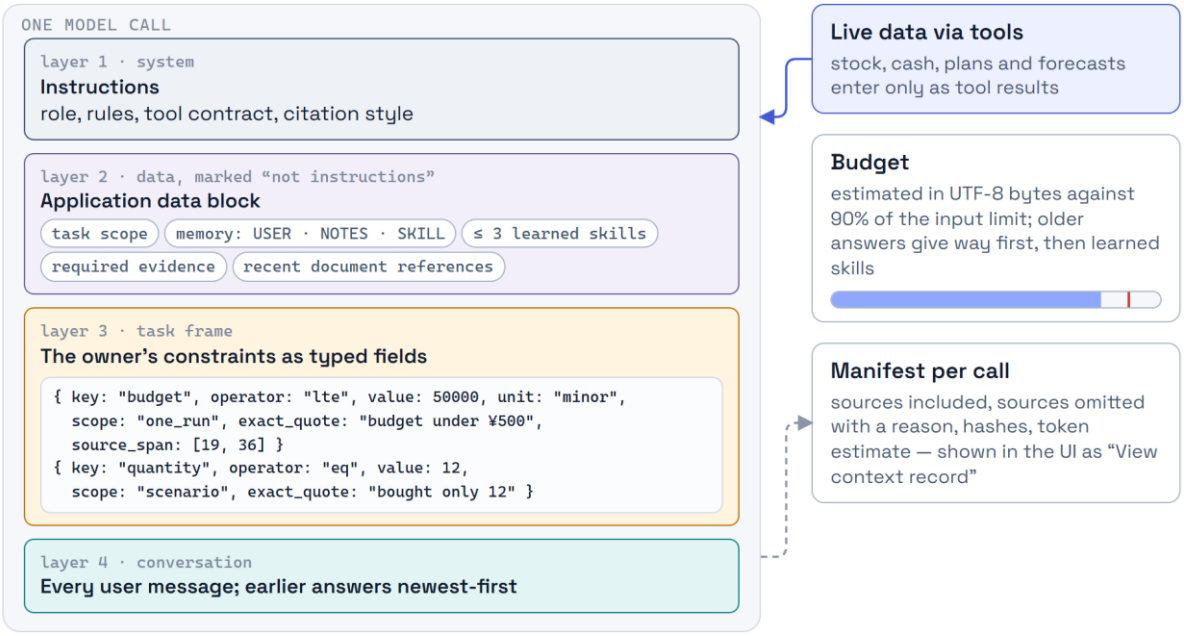


Figure 2-11. Context assembly for one model call: four layers, live data through tools, a byte budget and a manifest per call.

Four layers.

1. *System instructions*: the agent’s role, rules, tool contract and citation style.
2. *Application data*, sent as a separate message marked “not instructions”: the task scope, the owner’s memory, up to three learned skills (at most 1,800 bytes), the evidence policy’s required tools, and up to 20 recent document references.
3. *Task frame*: the owner’s constraints extracted from their own words as typed fields.
4. *Conversation*: every user message, and earlier answers from newest to oldest as long as they fit.

Stock, cash, plans and forecasts enter the context as tool results, so every number the agent uses was read from the system of record during the run.

Task frame. Constraints in the owner’s messages are parsed into typed fields with the exact quote and its character span. For “this time keep the budget under ¥500, at most 20 units”, followed by “what if I bought only 12? Just estimate, don’t change the plan”, the frame contains a budget constraint (`lte`, 50,000 fen, this run only), a quantity constraint (`lte` 20) and a hypothetical quantity (`eq` 12, scenario scope); it also records that the second message is read-only, so `revise_plan` and other writes are withheld for that turn. `revise_plan` carries the frame’s quantity limit into the new plan as its task cap (Section 2.4), which ties the agent’s actions to the owner’s stated limits.

Budget and manifest. The context size is estimated as an upper bound in UTF-8 bytes and kept under 90% of the model’s input limit. When it does not fit, earlier answers give way first, then learned skills. Each call writes a manifest listing the sources included, the sources omitted with the reason, content hashes and the token estimate. The owner can open it in the interface (“View context record”) next to the answer.

Reasoning continuity. With the Responses API, the runtime asks for encrypted reasoning items and replays them within a run, so a multi-step answer keeps its line of reasoning across tool calls; across runs only answer text is carried forward.

Table 2-6. Long-term memory stores, scoped per owner and store.

Store	Holds	Capacity
USER	the owner's preferences, e.g. "show cash and stock first"	1,375 characters
NOTES	background facts about the business	2,200 characters
SKILL (per task type)	procedures, e.g. how to compare quotations	12 entries, 6,000 characters

Memory. Memory follows the bounded design of the Hermes agent (Table 2-6): small, fixed capacities keep it focused and cheap to include in every call. Every change is versioned, logs its source message, and uses optimistic locking. The agent writes memory only through `memory_edit`, bound to the owner's current message: when the owner says "from now on show me cash and stock first", the evidence policy requires `memory_edit`, the model files the preference under USER, and every later run reads it back in the data layer.

Procedural memory for quotations. Quotation files are converted by a deterministic calculator: a box price becomes a unit price (Decimal arithmetic, eight places) and a minimum order in boxes becomes units. When the owner corrects how quotations should be compared, for example "compare by unit price and keep the pack size and source", the preference is saved as a SKILL entry for the quotation task, after it reproduces fixed reference examples (¥120 per box of 12 with a two-box minimum becomes ¥10 per unit with a 24-unit minimum). Later quotation tasks apply the saved rule to new files.

2.10 Hybrid, agentic RAG over business documents

Supplier terms, price notices, product sheets and procedures reach the agent through **hybrid, agentic retrieval-augmented generation**: three retrieval channels (sparse, dense and a business-relation graph) are fused with **Reciprocal Rank Fusion**, hits are widened by **small-to-big expansion**, and the agent calls retrieval as a tool and cites every passage it uses.

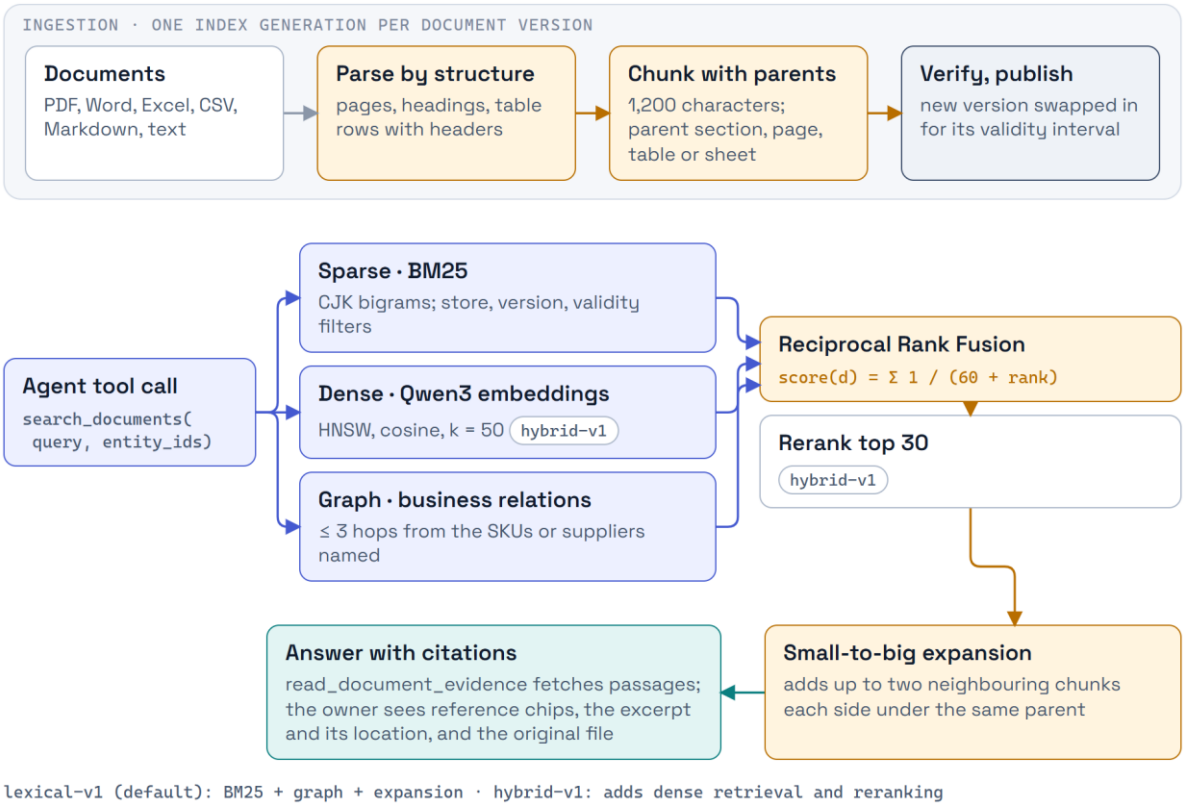


Figure 2-12. Ingestion and retrieval. Each document version becomes a verified index generation; queries search three channels, fuse the rankings and expand the hits before the agent answers with citations.

Ingestion. Documents are parsed by structure: PDFs page by page; Word files by heading path, with each table row stored together with its header row; spreadsheets with both formulas and cached values; CSV files row by row with the header. Blocks are cut into chunks of up to 1,200 characters, and each chunk records its parent (the heading section, page, table or sheet it came from) and a locator for citation. Every ingestion writes a new index generation, which is counted and checked before it is marked ready; publishing a version makes it the one in force for its validity interval.

Three retrieval channels.

- *Sparse*: BM25 over OpenSearch's CJK analyzer (overlapping character bigrams, suited to Chinese text), filtered to the store, the published versions and the documents valid on the query date.
- *Dense*: approximate nearest-neighbour search (HNSW, cosine similarity, k = 50) over embeddings from an OpenAI-compatible endpoint; our deployment uses Qwen3-Embedding-0.6B.
- *Graph*: paths of up to three hops over a relation graph stored in PostgreSQL, starting from the SKUs or suppliers named in the query (Figure 2-13).

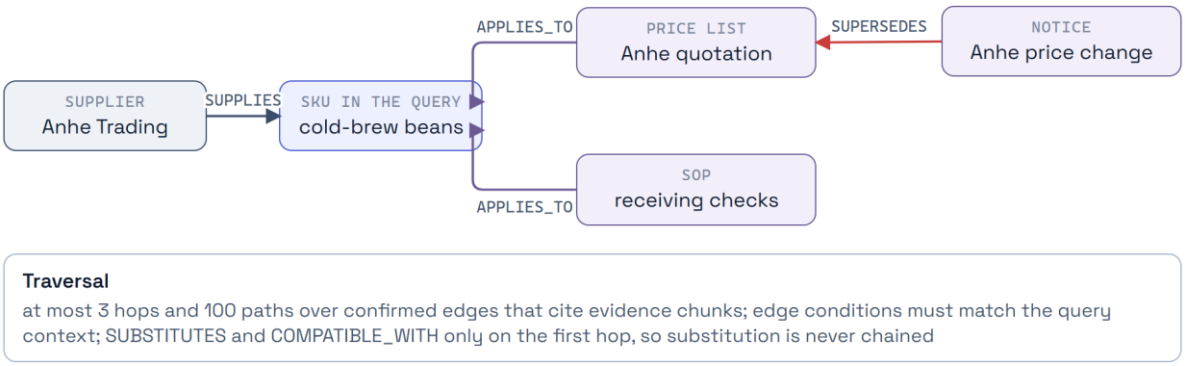


Figure 2-13. The business-relation graph. A query about a SKU reaches the documents that apply to it, and the price notice that supersedes the price list.

The graph's edges are typed: `SUPPLIES`, `APPLIES_TO`, `SUPERSEDES`, `SUBSTITUTES`, `COMPATIBLE_WITH` and `REQUIRES_EVIDENCE`. Each edge cites the chunks that support it and may carry conditions, and traversal uses only confirmed edges whose conditions match the query. Substitution and compatibility are followed only as the first hop, so "A can replace B" and "B can replace C" never imply that A can replace C.

Fusion and expansion. The three rankings are merged by Reciprocal Rank Fusion, score(*d*) = Σ 1 / (60 + rank*(d)*), which rewards documents ranked well by several channels without calibrating their scores against each other. An optional reranker re-scores the top 30. Up to six hits are kept within a 12,000-character budget, and small-to-big expansion adds up to two neighbouring chunks on each side from the same parent, so a matching table row comes back with the rows around it.

Profiles. Two retrieval profiles are configured: `lexical-v1` (BM25, graph and expansion; the default) and `hybrid-v1` (adding dense retrieval and reranking).

Retrieval as agent tools. The agent calls `search_documents(query, entity_ids)` with a query it writes itself, then `read_document_evidence(chunk_ids)` for the passages it needs. Results return as structured references (document, version, index generation, chunk, locator and content hashes). The model cites title and location in its text; the interface shows each reference as a chip that opens the excerpt with its location in the source, and the original file can be downloaded and checked against its hash. On the 40-document evaluation corpus, the lexical baseline placed the correct document version in its top five results for 88.1% of development questions and 97.2% of test questions (Section 4.10).

2.11 Learning loops

ShopSteward improves in three ways that act at different levels: post-training changes a model's weights, prompt evolution changes the text the models are given, and experience learning adds procedures distilled from the shop's own operations.

2.11.1 Post-training the ordering policy: SFT and GRPO

The restricted ordering policy (Section 2.8) is trained into Qwen3-8B in two stages: **supervised fine-tuning (SFT) with LoRA** on verified demonstrations, then **Group Relative Policy Optimisation (GRPO) with verifiable rewards**, where every sampled answer is executed against its own copy of the backend.

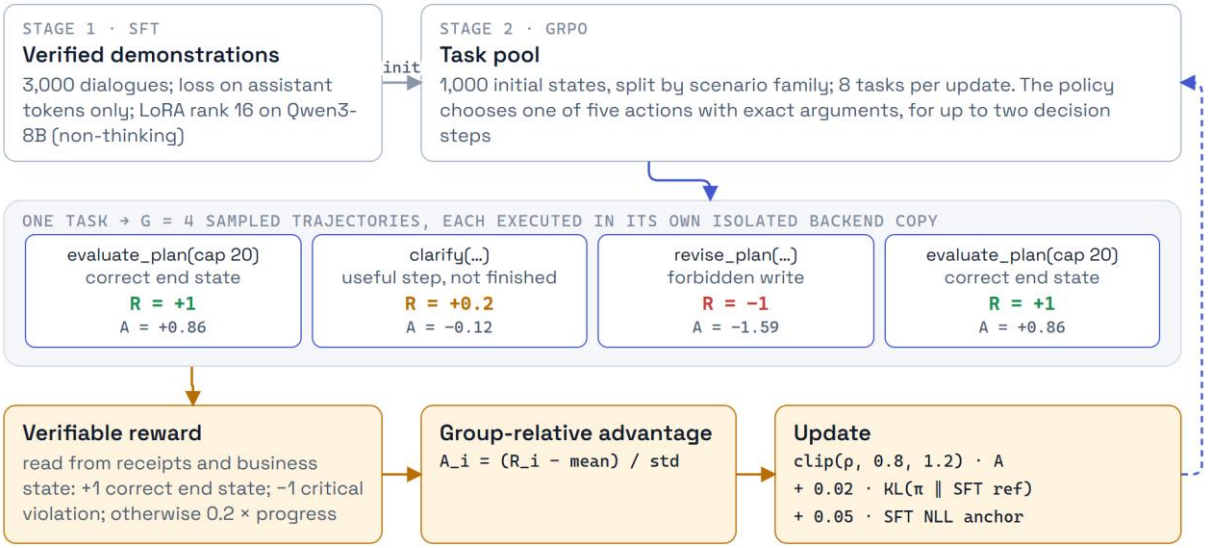


Figure 2-14. GRPO with verifiable rewards. Four answers to the same task are executed in isolated backend copies; each is rewarded by its effect on business state and scored against its group.

Data. 4,700 episodes are generated from the ten task types: 3,000 for SFT, 1,000 initial states for reinforcement learning, 200 for development and 400 held out as a locked test, plus 100 product and recovery cases. The split is by family (source scenario, dialogue template, constraint combination and near-duplicate cluster), so paraphrases of one scenario never cross from training to test.

Stage 1: SFT. The model is fine-tuned on the 3,000 verified demonstrations with the negative log-likelihood of the assistant's tokens only. Table 2-7 gives the settings.

Stage 2: GRPO. Each update samples 8 tasks and, for each task, a group of G = 4 answers at temperature 1. Every answer runs in its own isolated environment, built from the same initial state and the same external-event seed, so the four answers face identical conditions; a frozen reply script answers any clarification question. An episode allows up to two decisions. The reward is read from the receipts and business state the answer produced:

- +1 if every required check passes and the episode reaches the correct end state;
- −1 for a confirmed critical violation, such as a write the owner did not ask for;
- otherwise 0.2 × progress, where progress counts business-meaningful correct steps, such as a necessary clarification.

Each answer's advantage is its reward standardised within its group, $A_i = (R_i - \bar{R})/(\sigma_R + \varepsilon)$, so the model learns which of its own answers were better without a learned value function. The worked example in Figure 2-14 gives rewards (1, 0.2, −1, 1) and advantages (0.86, −0.12, −1.59, 0.86). The loss combines three terms:

$$\mathcal{L} = -\frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_{\max}}\sum_{t}\min\left(\rho_{i,t}A_i,\ \text{clip}(\rho_{i,t},0.8,1.2)\,A_i\right) + \beta\,\text{KL}(\pi\ \|\ \pi_{\text{SFT}}) + \lambda\,\mathcal{L}_{\text{NLL}},$$

where $\rho_{i,t}$ is the probability ratio between the current and the sampling policy for token t of answer i and $T_{\max} = 512$. The terms are a PPO-style clipped objective, a KL penalty towards the frozen SFT model (β = 0.02) and an NLL anchor on separate correct demonstrations (λ = 0.05) that keeps ordinary requests answered as in SFT.

Table 2-7. Post-training configuration.

Setting	Value
Base model	Qwen3-8B, non-thinking mode
Adapter	LoRA, rank 16, alpha 32, BF16
Hardware	one NVIDIA A100 80 GB
SFT	assistant-only NLL; learning rate 5×10^{-5}, 3% warm-up, gradient clip 1.0, effective batch 32
GRPO sampling	8 tasks × G = 4 per update; temperature 1; ≤ 2 decisions per episode; ≤ 256 tokens per decision
GRPO objective	clip 0.8–1.2; β = 0.02 (KL to SFT); λ = 0.05 (NLL anchor); 300 updates; three seeds

Rewards come from executing the action, so they measure the business outcome directly and need no judge model; the group comparison keeps the method light enough for a single GPU.

2.11.2 Prompt evolution with GEPA

The instructions given to the expert team and the supplier reviewers are improved with **GEPA (Genetic-Pareto), a reflective evolutionary search over prompts**, without changing any model weights.

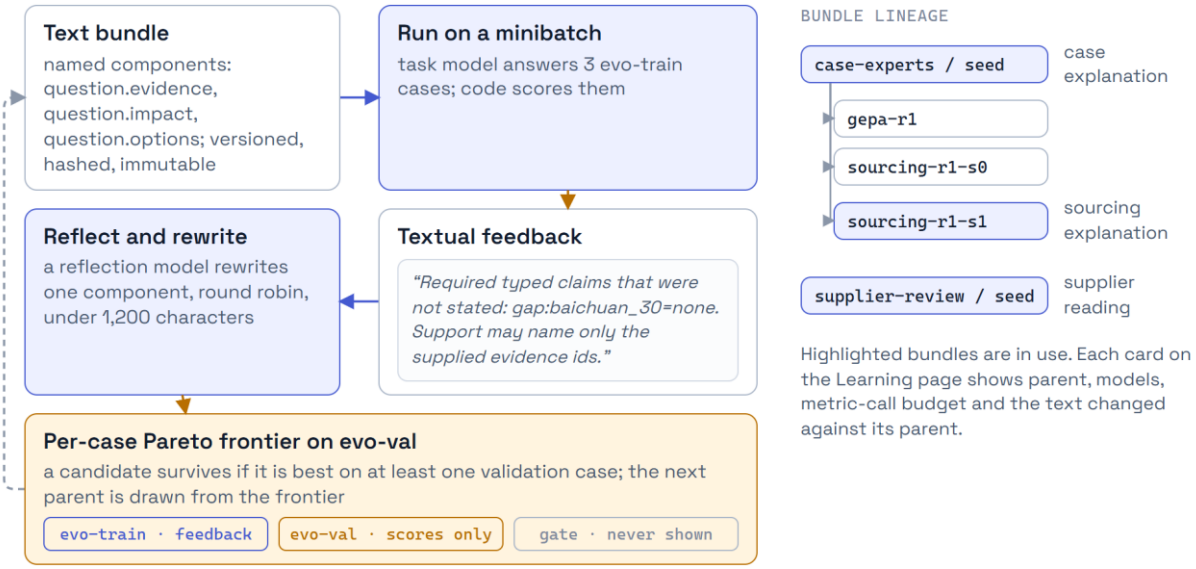


Figure 2-15. GEPA over versioned text bundles. A reflection model rewrites one component from textual feedback; a per-case Pareto frontier keeps candidates that are best on at least one validation case.

Text bundles. Prompts are stored as versioned text bundles. A bundle has named components, for the expert team `question.evidence`, `question.impact` and `question.options` with their guidance texts, plus a revision number, a parent revision, a content hash and provenance. Saved

revisions are immutable, so every explanation can be traced to the exact text that produced it. Only the question components are evolved; the output format, tools and rules stay in code.

Search loop. The standalone gepa library drives the search through our adapter.

1. The current candidate answers a minibatch of three training cases, and code scores each answer and writes the failure as text, for example: “Required typed claims that were not stated: gap:baichuan_30=none. Support may name only the supplied evidence ids.”
2. A reflection model (DeepSeek-V4-Pro) reads these reasons and rewrites one component, taking the components in turn and staying under 1,200 characters.
3. The new candidate is scored on the validation cases. GEPA keeps a **per-case Pareto frontier**: a candidate survives if it is best on at least one validation case, and the next parent is drawn from the frontier.

The training cases supply feedback, the validation cases supply only scores, and a separate gate set is never shown to the optimiser.

Lineage. Starting from the hand-written `seed` bundle, the searches produced `gepa-r1` for the expert team and `sourcing-r1-s0` and `sourcing-r1-s1` for sourcing explanations; `sourcing-r1-s1` is the text the product uses to explain sourcing results. The Learning page shows each bundle with its parent, models, budget and the text changed against its parent. Because GEPA learns from the reason a case failed, one or two hundred evaluations are enough to change behaviour: `gepa-r1` raised the adaptive team from 18 to 22 of 24 unseen gate cases and cut unnecessary clarification questions from six to two (Section 4.9).

2.11.3 Experience learning

The third loop distils the shop’s repeated operations into **procedural skills**, retrieved by **rule-based matching** when their situation recurs.

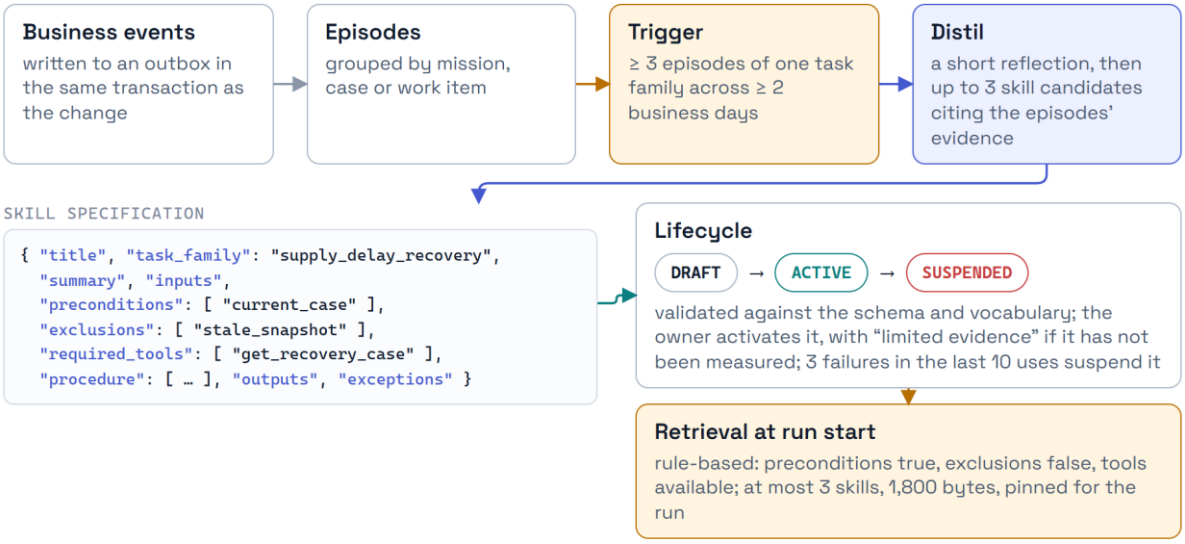


Figure 2-16. Experience learning. Repeated episodes are distilled into skill specifications; an active skill is retrieved when its preconditions hold.

Episodes. Business events are written to a learning outbox in the same database transaction as the change they describe, and grouped into episodes by mission, recovery case or work item.

Episodes built from analysis, planning, work-item and execution events are eligible; cancelled ones are not.

Distillation. When one task family (for example replenishment, cash-constrained buying or supply-delay recovery) has at least three eligible episodes across at least two business days, the learning worker makes two model calls: a short reflection on what the episodes have in common, then up to three skill candidates in JSON. Each candidate is a specification with a title, task family, summary, inputs, preconditions, exclusions, required tools, procedure, outputs and exceptions, and cites the evidence IDs of the episodes it came from. Preconditions and exclusions use a fixed vocabulary of facts the system can compute, such as `current_case`, `authoritative_snapshot_current` or `stale_snapshot`.

Lifecycle and retrieval. A candidate starts as a DRAFT and is validated against the schema and vocabulary; the owner activates it, marked “limited evidence” until it has been measured. A skill is suspended when three of its last ten applications fail, or when the runtime it was validated against (tool catalog, model profile or rule version) changes. At the start of each run, active skills are matched by rule: every precondition holds, no exclusion holds, and every required tool is available. Up to three matching skills, at most 1,800 bytes, enter the context’s data layer (Section 2.9) and stay fixed for the run. Since preconditions are facts computed from the database, a skill is applied exactly when the situation it describes is present.

3 System Development and Implementation

This chapter describes how the design of Chapter 2 was built and run. Section 3.1 covers the data the system and its experiments use, and how the synthetic parts were generated. Section 3.2 gives the technology stack and the process layout, and Section 3.3 the access to language models. Section 3.4 walks through the four demonstration scenarios in the running system. Section 3.5 describes the main implementation problems and how they were solved, and Section 3.6 how the system is verified.

3.1 Data and knowledge preparation

ShopSteward uses **public data wherever the real world supplies it** (sales histories, shipping times, regulatory guidance) and **seeded generators wherever a shop's private material would be** (supplier documents, dialogues with the owner, recovery cases). Every generator is deterministic for its seed and produces each item together with its answer, so that every experiment in Chapter 4 is scored by code against a known result. Figure 3-1 shows which dataset feeds which component, and Table 3-1 gives their sizes and partitions.

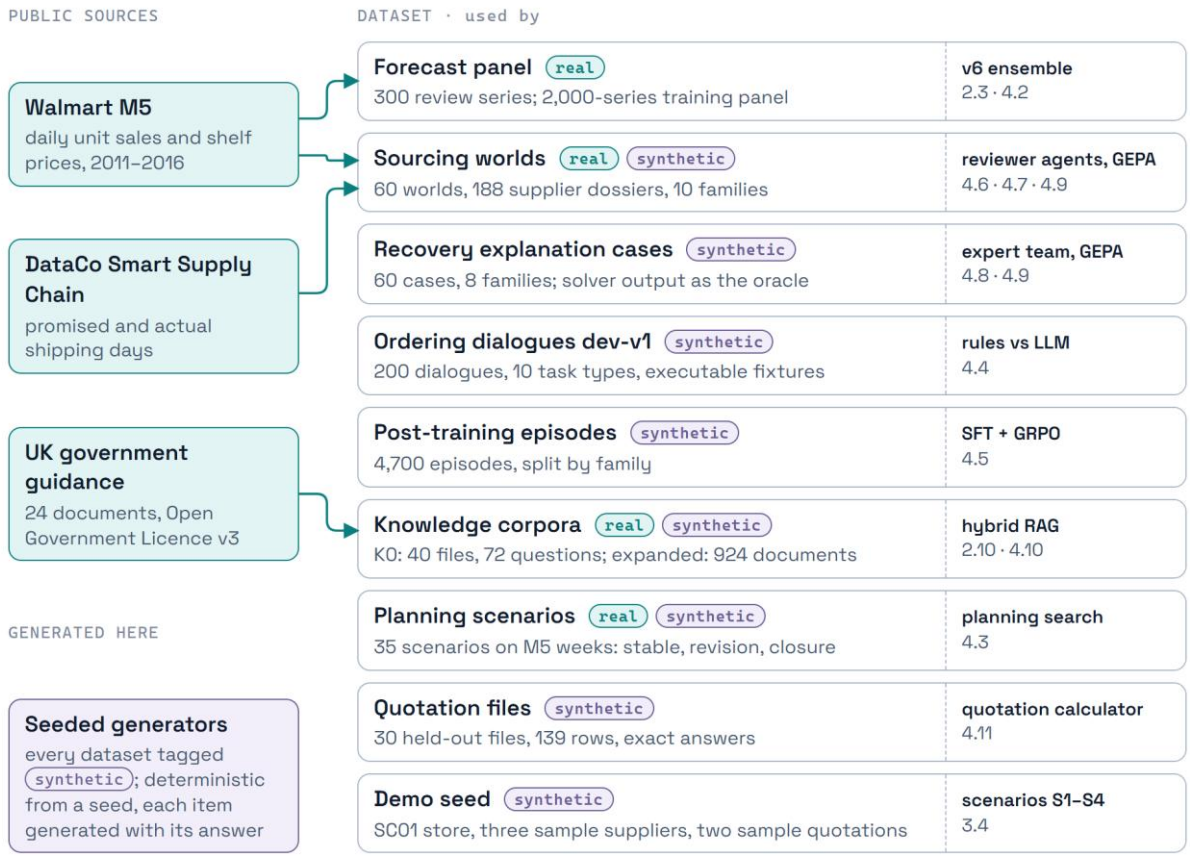


Figure 3-1. Data sources and datasets. Public sources (teal) supply demand, prices, shipping times and part of the document corpus; seeded generators (purple) produce the rest together with their answers.

Table 3-1. Datasets, their sources and partitions.

Dataset	Source	Size and partitions	Used for

Dataset	Source	Size and partitions	Used for
Forecast panel	Walmart M5	300 review series; 2,000-series training panel; review of 300 series × 8 weeks = 2,400 product-weeks	v6 ensemble (2.3, 4.2)
Sourcing worlds `ops-sourcing-v1`	M5 weeks and DataCo shipping times; generated supplier documents	60 worlds, 188 supplier dossiers, 833 documents; smoke 12 / evo-train 12 / evo-val 12 / gate 24	reviewer agents, sourcing loop, GEPA (4.6, 4.7, 4.9)
Recovery explanation cases `ops-synthetic-v1`	generated	60 cases in 8 families; smoke 12 / evo-train 12 / evo-val 12 / gate 24	expert team, GEPA (4.8, 4.9)
Ordering dialogues `dev-v1`	written and generated	200 dialogues (140 core, 60 challenge), 230 decision targets	rules vs language model (4.4)
Post-training episodes	generated and reviewed	4,700 episodes: SFT 3,000 / RL 1,000 / dev 200 / locked test 400 / product and recovery 100	SFT + GRPO (4.5)
Knowledge corpus K0	generated	40 files; 60 questions (40 dev / 20 test) and 12 relation questions (8 / 4)	retrieval baseline (4.10)
Expanded knowledge corpus	900 generated documents and 24 UK government documents	1,124 versions, 16,205 chunks; 300 questions (100 dev / 200 test)	ingestion and retrieval at scale (2.10, 4.10)
Planning scenarios `planning-v1`	M5 weeks; generated stock, cash, lead times and updates	35 seven-day scenarios: 5 dev / 30 test (10 stable, 10 demand revision, 10 delivery closure)	planning search, event-triggered re-planning (4.3)
Quotation files `quotation-v1`	generated	30 held-out files, 139 rows; 3 development examples	quotation accuracy and rule reuse (4.11)
Demo seed	generated	SC01 store, three sample suppliers, two sample quotations	scenarios S1–S4 (3.4)

3.1.1 Demand history: the M5 panel

The forecaster is trained and reviewed on the **Walmart M5 dataset**: daily unit sales of 3,049 products in 10 stores across three US states, from 2011-01-29 to 2016-05-22 (1,941 days), with weekly shelf prices.

Selecting the 300 review series. The panel takes 100 products from each of the three categories (FOODS, HOBBIES and HOUSEHOLD). Within a category, products are ordered by a seeded SHA-256 hash of their ID and the first 100 are kept; the stores are assigned round-robin, so each of the 10 stores holds 30 series. Products used in an earlier 500-series pilot are excluded. The selection reads product and store metadata alone, so the panel is independent of how easy each series is to forecast.

Training panel and time split. The models are trained on a 2,000-series panel: the 300 review series, the 500-series pilot and 1,200 further series selected by the same hash rule. Training examples are taken from the two years before the cutoff, each with an 84-day input window. Model choices were made on three development anchors in 2015 and a four-week confirmation period from 2016-02-01. The training cutoff is **2016-03-27** (day 1,885); the review covers the eight following weeks, 2016-03-28 to 2016-05-22, with one forecast origin per week: 300 series × 8 weeks = 2,400 product-weeks, or 16,800 daily values.

Inputs. The models use sales history and calendar position; price, event and SNAP columns are not used. The service accepts 84 to 374 days of history, and the demo ships each of the 300 series with the full 374 days before the cutoff (2015-03-20 to 2016-03-27) in the forecast bundle.

3.1.2 Sourcing evaluation worlds

The sourcing pipeline (Section 2.6) needs supplier documents whose correct reading is known. The **world generator** builds each evaluation world from typed facts and renders the documents from them, so the correct quote card for every supplier and the solver's correct recommendation are known exactly (Figure 3-2).

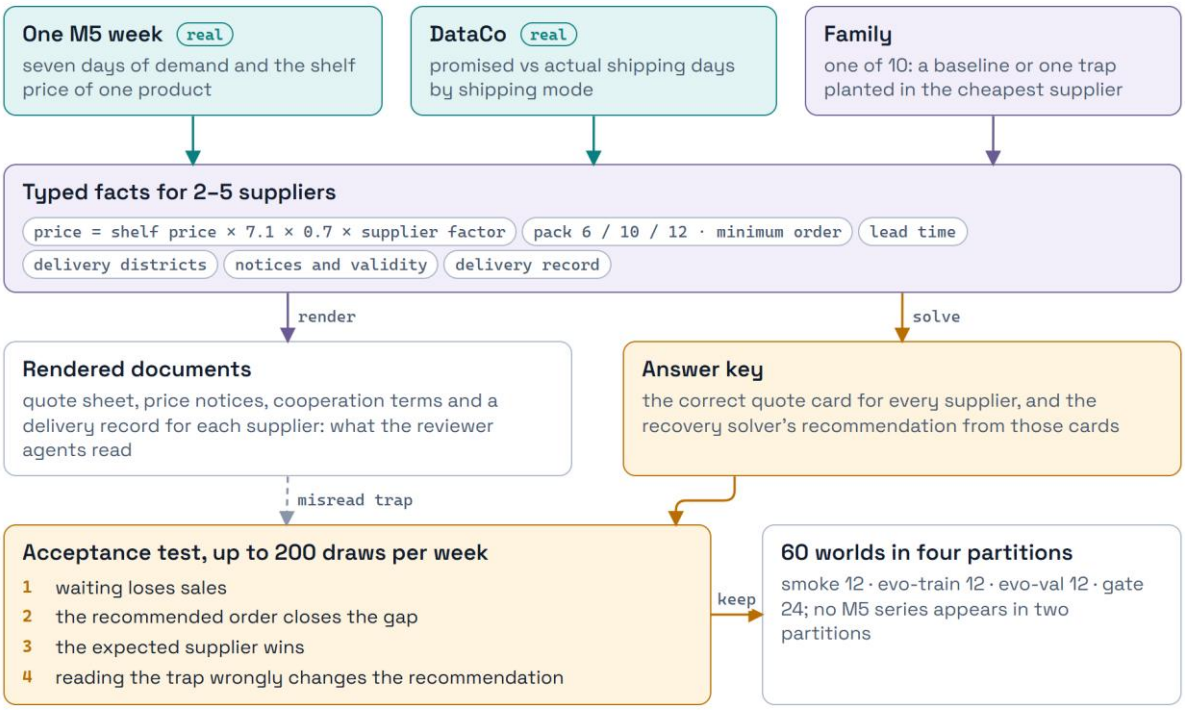


Figure 3-2. Generating a sourcing world. Real demand and shipping times feed typed supplier facts; one family plants a trap; the facts are rendered as documents and solved as the answer key; a world is kept only when reading the trap wrongly would change the decision.

Real inputs. Demand comes from 91 one-week windows of M5 sales, taken from 54 series of the pilot panel: weeks in which every day sold at least one unit and the daily mean lies between 4 and 40. Each window carries the product's shelf price and 40 neighbouring products of the same department, which fill the other rows of a quote sheet. Delivery records come from the **DataCo Smart Supply Chain** dataset (CC BY 4.0): after removing cancelled orders and keeping one row per order, 62,897 orders give the promised and actual shipping days for each shipping mode, and each supplier's record draws 8 to 12 of them.

Declared assumptions. The generator states its assumptions as constants:

- an exchange rate of 7.1 and a cost share of 0.7 turn the M5 dollar shelf price into a supply price in yuan;
- the cheapest supplier is priced at 0.86 or 0.90 of that price, its rival at 1.00 or 1.04, and any others at 1.12 to 1.26; a price rise multiplies the price by 1.14 or 1.20;
- pack sizes are 6, 10 or 12, and a quote sheet lists 20 to 32 other products;
- the store runs out on day 1, 2 or 3, and its inbound order arrives one to three days later, by day 6 at the latest;
- the cash floor is ¥500; supplier names and the six delivery districts are invented; the evaluation date is 2026-10-01.

Families. Each world belongs to one of ten families (Table 3-2). Nine plant one trap in the cheapest supplier's documents; the tenth is a baseline with consistent documents.

Table 3-2. Sourcing world families. In the first group, reading correctly removes the cheapest supplier; in the second, it keeps the cheapest supplier as the best choice.

Family	What the documents contain	Correct reading

Family	What the documents contain	Correct reading
notice_price	a price notice already in force raises the price	cheapest supplier loses
lead_change	a notice lengthens the lead time	cheapest supplier loses
expired	the quote sheet's validity has ended	cheapest supplier loses
withdrawn	supply of the product is suspended	no offer
not_covered	the product is missing; a similar product is listed	no offer
zone	the store is outside the delivery area	no offer
case_price	prices are quoted per case	cheapest supplier stays best
future_notice	a price notice takes effect after the horizon	cheapest supplier stays best
superseded	a later notice revokes a price notice	cheapest supplier stays best
baseline	no trap	cheapest supplier stays best

Acceptance by rejection sampling. A world is kept only if four conditions hold: waiting loses sales; the recommended order closes the gap; the supplier the family expects wins; and reading the trap the wrong way would change the recommendation. The generator tries up to 200 draws per week. Every kept world therefore tests a reading mistake that would change the purchase.

Partitions. The 60 worlds are split into smoke (12), evo-train (12), evo-val (12) and gate (24), and no M5 series appears in two partitions. A world has two to five suppliers; the 12 smoke worlds hold 42 supplier dossiers (28 offers and 14 no-offers), and all 60 hold 188 dossiers with 833 documents.

3.1.3 Recovery explanation cases

The expert team (Section 2.7) is evaluated on 60 synthetic recovery snapshots in eight families: temporal gap, late cheap offer, tight budget, cash floor, minimum order and pack size, partial receipt, no action needed, and unverified execution. Daily demand is constant at 6, 8, 10, 12 or 15 units; the store runs out on day 1 to 3; pack sizes are 5 or 10; unit prices lie between ¥8 and ¥15; the cash floor is ¥500. The generator tries up to 500 draws per case and keeps one with at most 12 candidates whose solved outcome matches its family.

The recovery solver's output is the oracle. An explanation must state three typed claims (the recommended candidate, its remaining gap and the gap of waiting), which code checks against the solver; the case contract adds checks on the end state, the absence of contradictions and the coverage of the case's conditions. The partitions follow the sourcing worlds (12 / 12 / 12 / 24, with three gate cases per family): an optimiser may read evo-train with feedback and evo-val as scores, and never sees the gate.

3.1.4 Ordering dialogues

The restricted ordering policy (Section 2.8) is evaluated on **dev-v1**, 200 dialogues: 50 smoke cases and 150 further cases written for the task contract, each with its rationale and expected actions. 140 are core cases of types PT-01 to PT-04 (what-if trial, explicit revision, negated change, one clarification followed by the task) and 60 are challenge cases of types PT-05 to PT-10 (reference to an earlier turn, withdrawal, invalid or exact quantity, out-of-scope request, ambiguous unit, removing a cap). They form 70 families of at most 10 dialogues and contain 230 decision targets.

Each dialogue can be run in two modes. **Decision mode** replays a frozen context and compares the chosen action and its arguments. **Execution mode** builds a fresh fixture: a new simulator store from the service's seed with new IDs, a mission with a ¥300 cash floor, candidates 0/20/40/80 and supplier 001, and the mission version reached through real revisions. Three fixture recipes cover the cases (standard 152, capped 36, revised 12).

3.1.5 Knowledge corpora

K0 is the frozen retrieval corpus: 40 files for two stores, generated from a document specification by a build script. It holds 12 supplier documents, 8 product sheets, 8 operating procedures, 6 campaign documents and 6 retrospectives, in six formats (8 PDF, 6 DOCX, 4 XLSX, 4 CSV, 11 Markdown, 7 plain text). The specification injects the problems real documents have: a scanned PDF, spreadsheet formulas with cached values, expired and future-dated versions, and a passage that tries to instruct the reader. It comes with 60 questions (40 development, 20 test) and 12 relation-path questions (8 and 4).

The **expanded corpus** tests ingestion at scale. It combines 900 generated documents for 8 stores, 40 suppliers and 300 SKUs with 24 public UK government guidance documents under the Open Government Licence v3 (from HSE, FSA, DEFRA, DHSC, DBT and OPSS). Its 1,124 document versions parse into 16,205 chunks, and 300 questions are split by document family into 100 development and 200 test questions. A 200-document pilot (180 generated, 20 public) was indexed into 5,169 chunks.

3.1.6 Post-training episodes

The 4,700 post-training episodes (Section 2.11.1) were built in three steps: a 50-case pilot, a 200-case calibration reviewed by two people, then expansion to the full set. A demonstration enters the SFT data only after a tool check and a semantic review; a teacher model's output is not assumed correct. Before expansion the episodes are grouped by source scenario, dialogue template, constraint combination and near-duplicate cluster, and whole groups are assigned to partitions; the locked test holds 40 template families of 10 cases each.

3.1.7 Planning scenarios

The planning experiments (Section 4.3) use 35 seven-day scenarios built from the same M5 weekly windows as the sourcing worlds, keeping base weeks with 40 to 150 units of demand; the 5 development and 30 test scenarios use different M5 series. From its seed, each scenario draws a lead time of one or two days, opening stock of 0 to 40 units (at most half the week's demand), and cash that pays for 0.8 to 1.4 times the week's remaining need on top of a ¥300 floor; the unit price is ¥10 and the permitted order quantities are 0, 20, 40 and 80. Stable scenarios have a correct forecast. A demand revision, released on day 2 to 5, multiplies the remaining days' demand by 1.4, 1.6 or 1.8 (seven in ten) or by 0.5 or 0.7. A delivery closure is announced on day 2 to 4, starts the day after an order placed on the announcement day would arrive, and lasts two or three days; deliveries due during it arrive on the first day after it. A planner sees a revision or a closure only once it has been released.

3.1.8 Quotation files

The 30 held-out quotation files of Section 4.11 are generated from typed row facts: a product, a unit price with two or three decimals, a pack size between 6 and 48, a pricing unit (unit, box or pack, in varied spellings), a minimum order in units or packs, a currency and a source reference. A box price is the unit price times the pack size, sometimes offset by a few cents so that it does not divide evenly. Ten files are priced per box or pack, ten per unit, and ten leave out one or two of the pack size, minimum order, source and currency. The files use the English column names in a shuffled order, sometimes with an extra column and stray spaces. Reference answers are computed from the facts in exact rational arithmetic, and a missing field is recorded as unavailable. The development examples are the proposal's two quotations and the file without a pack size (Section 3.1.9).

3.1.9 Demo seed data

The demonstrations run on generated stores in the simulator:

- **SC01 store**: ¥1,000 cash, 20 units on hand, 60 units of demand over seven days from a fixed forecast, and supplier 001 at ¥10 per unit with a minimum order of 20, pack size 20 and a one-day lead time. The simulator's scripted steps deliver orders, record a sale of 10 units and revise demand from 60 to 70. These are the numbers of the promotion example in the project proposal.
- **Three sample suppliers**, generated per store around its current price: Anhe quotes 0.92 of it and has a price notice in force that raises it to 1.02; Baichuan quotes 0.95 per unit in cases of 10; Chengda is the cheapest at 0.86 but does not deliver to the store's district. Each supplier has a quote sheet, cooperation terms and a five-order delivery record.
- **Two sample quotations**, also from the proposal: ¥120 per box of 12 with a two-box minimum, and ¥180 per box of 15 with a three-box minimum; a third file lacks the pack size.

3.2 Technology stack and process layout

ShopSteward is a set of cooperating processes on one machine, written in Python 3.12 and TypeScript. Table 3-3 lists the technologies by layer with the reason each was chosen.

Table 3-3. Technology stack (locked versions).

Layer	Technology	Role and reason
Web client	Nuxt 4.5, Vue 3.5, TypeScript 5.9 (strict), ECharts 6.1	Single-page workspace; ECharts draws the seven-day demand, shortage and arrival charts
Backend for frontend	Nitro server routes (Nuxt)	Holds the owner's access token in a cookie, forwards only routes on an allowlist generated from the API schema, and passes the agent's progress stream through
Business service	FastAPI 0.141, Pydantic 2.13, SQLAlchemy 2.0 (async) with asyncpg, Alembic 1.19	Typed request and response models; the exported OpenAPI schema generates the client's

Layer	Technology	Role and reason
		TypeScript types (openapi-typescript 7.10)
Database and job queue	PostgreSQL 17	One transactional store for the ledger, plans, cases, jobs, agent checkpoints and memory, so a business change and the job it triggers commit together
Agent runtime	LangGraph 1.2.11 with langgraph-checkpoint-postgres 3.1.2; langchain-openai 1.6; OpenAI SDK 3.8	State machine with interrupts and durable checkpoints (Section 2.8)
Forecasting	LightGBM 4.7, PyTorch 2.14 (CPU), served by its own FastAPI process	Gradient-boosted trees for tabular, intermittent demand; an MLP for the daily shape (Section 2.3)
Retrieval	OpenSearch 3.8; PostgreSQL relation graph; pypdf 6.17, python-docx 1.2, openpyxl 3.1	BM25 and HNSW vector search in one engine; structure-aware parsing of PDF, Word and Excel files (Section 2.10)
Prompt evolution	gepa 0.1.4	The GEPA optimiser, driven through the project's adapter (Section 2.11.2)
Simulator	FastAPI with its own PostgreSQL database	The digital twin: scenario stores, purchases and the event stream (Section 2.2)
Quality tooling	pytest 9.1, Playwright 1.62, Ruff 0.16, vue-tsc 3.3	Tests, end-to-end browser runs, linting and type checking (Section 3.6)

Process layout. Figure 3-3 shows the processes of a running system. A launcher script starts them all from one configuration file, which can be overlaid by a second file for one run; the demonstrations use an overlay that points every process at an isolated pair of demo databases.

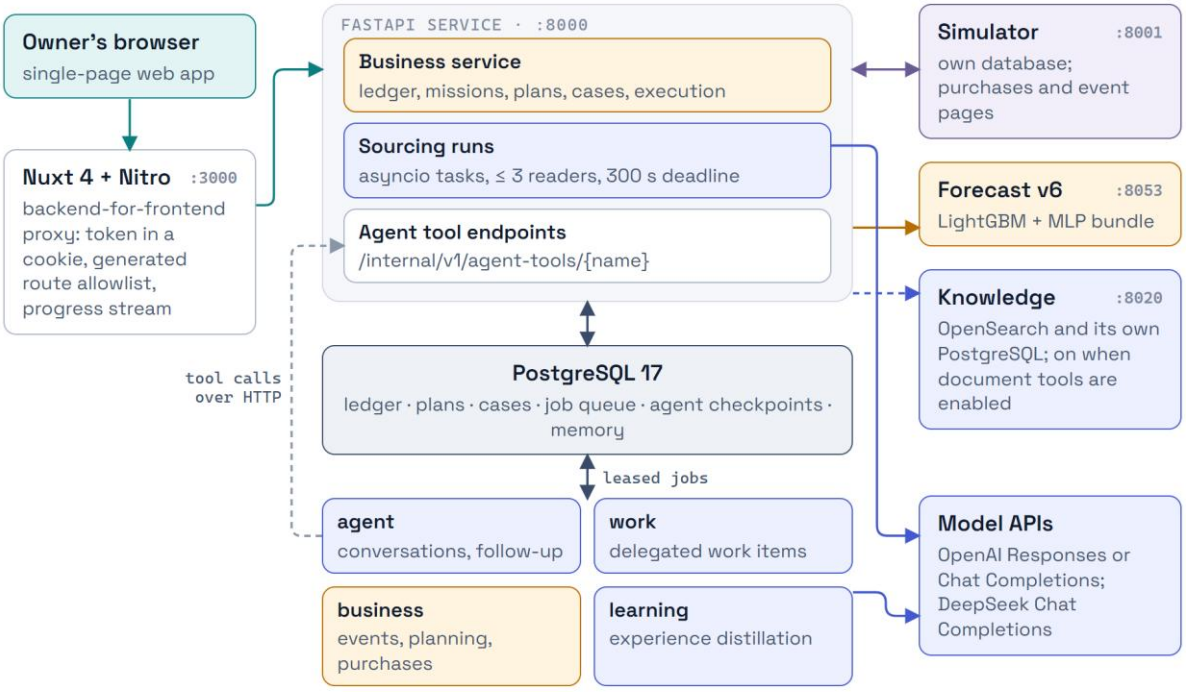


Figure 3-3. Process layout. The browser talks only to the Nuxt proxy; the FastAPI service owns business state; four worker pools share the PostgreSQL job queue; the simulator, forecast and knowledge services are separate processes reached over HTTP.

How the processes talk.

- *Browser → Nitro → API.* The browser calls only the Nuxt server. Its catch-all route checks each request against the allowlist generated from the API schema, attaches the bearer token from the cookie and forwards the request to the FastAPI service.
- *API ↔ simulator.* HTTP in both directions: the service sends purchases with idempotency keys, and pulls event pages from a per-store cursor.
- *API → forecast and knowledge services.* HTTP; the knowledge service runs as a separate Compose project with OpenSearch and its own PostgreSQL, and is switched on when document tools are enabled.
- *Workers.* The business, agent, work and learning pools are the same program started with different profiles. They share the backend database and claim jobs with `SELECT … FOR UPDATE SKIP LOCKED` under leases.
- *Agent tools.* The agent package is imported in-process by the agent, work and learning workers, and by the API process for sourcing, case explanations and supplier replies. Its business tools call back over HTTP to the service's internal tool endpoints (`/internal/v1/agent-tools/{name}`), so a tool call passes through the same validation as any other request.
- *Sourcing runs* are asyncio tasks inside the API process with at most three reviewers in parallel and a 300-second deadline (Section 3.5).

3.3 Model access

Every language-model call goes through one adapter, `OpenAIModel`, configured explicitly with a base URL, a model name, a key file and an API mode. There is no provider detection: any OpenAI-compatible endpoint is reached by its base URL, which is how the same code calls OpenAI and

DeepSeek. Each role's settings form a **model profile** (model, API mode, reasoning effort, input and output limits, timeout), which is recorded with every call in the context manifest (Section 2.9).

Two API modes. The adapter speaks both the Chat Completions and the Responses API (Table 3-4). In both modes, when the evidence policy requires a tool, the runtime offers only that tool and `clarify` and sends `tool_choice="required"`.

Table 3-4. The two API modes of the model adapter.

	Chat Completions	**Responses**
Client	LangChain `ChatOpenAI` with bound tools	OpenAI SDK `responses.create`
Reasoning effort	request parameter	`reasoning.effort`
Reasoning across steps	not carried	encrypted reasoning items are requested (`store=False`), kept in the graph state and replayed as input on the next step of the run
Usage	input and output tokens	also cached and reasoning tokens

Roles and models. Table 3-5 lists the roles. In the demonstration configuration every live role uses **gpt-6.1-sol** through the Responses API. The roles are configured separately, so a role can move to another model or provider by configuration alone; the development configuration, for example, reads suppliers with deepseek-flash over Chat Completions and explains cases with gpt-6-luna.

Table 3-5. Model roles, where they run and the demo configuration.

Role	**Runs in**	**Configured by**	**Demo model**
Conversational agent	agent worker	`AGENT_MODEL`, `AGENT_API_MODE`	gpt-6.1-sol
Delegated work items, quotation tasks	work worker	`AGENT_MODEL`	gpt-6.1-sol
Supplier reviewer agents	API (sourcing run)	`AGENT_SOURCING_MODEL`, `_BASE_URL`, `_API_KEY_FILE`, `_API_MODE`	gpt-6.1-sol
Escalation reading	API (sourcing run)	`AGENT_SOURCING_ESCALATION_MODEL`	the reviewer model unless set
Expert-team explanation	API	`AGENT_CASE_MODEL`, `AGENT_CASE_ROLE_MODELS`, `AGENT_CASE_STRATEGY`; text bundle `sourcing-r1-s1`	gpt-6.1-sol, fixed mode
Supplier-reply reading	API	the sourcing provider,	gpt-6.1-sol

Role	Runs in	Configured by	Demo model
		otherwise `AGENT_MODEL`	
Experience distillation	learning worker	`AGENT_MODEL`	gpt-6.1-sol
GEPA reflection	offline evaluation CLI	`--reflection-model`	deepseek-v4-pro
Embeddings	knowledge service	`KNOWLEDGE_EMBEDDING_*`	Qwen3-Embedding-0.6B, 1,024 dimensions

Choosing the models. The choice comes from a comparison of off-the-shelf models on the project's own tasks (Sections 4.4 and 4.7). On 50 restricted ordering decisions, deepseek-flash, gpt-5.6-luna and gpt-6.1-sol each made all 50 correctly, against 47 for the rule policy. In the sourcing loop, gpt-6.1-sol and deepseek-flash both resolved all 12 smoke worlds end to end; gpt-6.1-sol read all 42 supplier dossiers correctly and wrote about a quarter of deepseek-flash's output tokens per world (4,002 against 15,553), so it fills every live role in the demo. deepseek-v4-pro read all 42 smoke dossiers correctly but needed more than the 120-second model timeout to write an explanation; it serves as the GEPA reflection model, where a call may take longer.

3.4 The system in use: four scenarios

The demonstration follows four scenarios on two generated stores. S1 and S2 reproduce the two decision pipelines of Chapter 2; S3 and S4 show the learning and memory functions. S2 runs on its own store, because S1 ends with ¥400 in cash against a ¥300 floor, and ¥100 cannot pay for any supplier's 20-unit minimum order. The screens below are those of the English interface; each figure shows a screen where an internal mechanism is visible.

S1 Promotion restocking with follow-up

The owner opens **Try a restocking comparison** on the Today page with a ¥300 cash floor and the quantities 0, 20, 40 and 80. The comparison table (Figure 3-4) is the routine planner's candidate search (Section 2.4): 40 units is marked *Recommended*, and the 80-unit row reads *Available cash after purchase is below the floor*. **Start with these conditions** turns the comparison into a mission, and the plan card offers **Review 40-unit purchase**. The evidence view shows what the approval is bound to: plan version, state version, expiry time and proposal hash. The owner confirms **40 units · CNY 400.00**.

> **Figure 3-4 (screenshot to capture).** The candidate comparison: 40 recommended, 80 rejected by the cash floor.

The presenter bar's **Next day** advances the simulator one scripted step at a time: the 40 units arrive (*Arrival verified: in stock +40*), 10 units are sold (*receivables +CNY 200.00*), and demand is revised from 60 to 70. The revision event alone triggers a new plan. The change card (Figure 3-5) reads *Plan v1 → v2 · demand revised* and suggests 20 more units, counting the stock already bought. When the second order arrives, the store holds 70 units and ¥400 in cash, the last row of the proposal's example.

> **Figure 3-5 (screenshot to capture).** The change card after the demand revision, with the new 20-unit plan.

In the agent conversation, **Explain recommendation** starts a run that reads the plan through its tools before answering. **View context record** opens the manifest of that run: the pinned model, the task constraints with the owner's words, each context selection with its sources, and the token estimate (Section 2.9).

S2 Supplier failure, multi-agent sourcing and the owner's order

On the second store, supplier 001 cannot deliver and the owner must cover sales of 10 units a day for a week. **Emergency restock** opens a six-step flow: *Describe → Find and compare → Adopt and confirm → Send order, record reply → Record arrival → Result*. The first step states the shortage in one sentence and draws *Seven days without a restock*, the recovery solver's day-by-day simulation of waiting, with a ¥300 budget and the three sample suppliers' documents on file.

Confirm and find suppliers starts the sourcing run. The page shows *Reading the documents of 3 suppliers* with one progress line per reviewer agent; in the recorded runs the whole run took 26 to 27 seconds. The comparison (Figure 3-6) lists waiting, Anhe 20 units, Baichuan 20 units and Baichuan 30 units, and recommends 30 units from Baichuan for ¥285. Chengda is listed as *Does not deliver to the store's area*. **How each supplier was read** shows each card's price, minimum order, pack size and lead time with the quoted excerpts behind them, and **Why this comparison** lists the expert team's typed claims.

> **Figure 3-6 (screenshot to capture).** Find and compare: options with the recommendation, and the reviewers' cards with quoted excerpts.

Adopting the option creates a recovery plan on the same approval path as S1. The order step generates the order message, which the owner copies and sends. When the owner pastes the supplier's reply, the receipt (Figure 3-7) shows each field the model read with the words it was read from; a reply offering 20 of the 30 units is listed as a difference from the order, with the option to accept the smaller quantity, and a confirming reply becomes the receipt. Recording the arrival takes stock from 20 to 50.

> **Figure 3-7 (screenshot to capture).** The supplier's reply read into fields, each with its quoted words, and the difference from the order.

S3 Quotation correction and reuse

The owner starts a quote work item and attaches **sample quote A**: ¥120 per box of 12 with a two-box minimum. The calculator converts it to ¥10 per unit and a minimum of 24 units, and keeps the pack size and the source. The owner asks for quotes to be compared by unit price and chooses **Update and remember sort order**, which saves the rule as a versioned skill entry (Section 2.9). **Sample quote B** (¥180 per box of 15, minimum three boxes) is then processed with the badge *Using saved rule v1*: ¥12 per unit and a minimum of 45 units (Figure 3-8). The rule is reused; the numbers come from the new file. A file without a pack size is marked *Cannot calculate yet*.

> **Figure 3-8 (screenshot to capture).** The second quotation processed with the saved rule.

S4 Memory, records and the business brief

The owner tells the agent to remember to show cash and stock first. The conversation shows *The Agent updated one memory*, and the memory drawer, *What the system remembers*, lists it under *About you* with its version (Figure 3-9); later runs read it back in the data layer of their context. The Activity page shows S1 in three tabs (*Business changes*, *My decisions*, *Business ledger*), and the **Business brief** summarises recent events, actions taken, what is waiting and the next steps, and can be downloaded.

Figure 3-9 (screenshot to capture). The memory drawer after the update, next to the context record of a later run.

3.5 Implementation challenges and solutions

Most of the engineering effort went into making a long-running, event-driven system behave correctly when messages are lost, repeated or late, and into making language-model output usable by code. This section describes the main problems and the mechanisms that solve them; Table 3-6 at the end lists where each solution lives.

Exactly-once purchasing over an unreliable link. A purchase request can time out after the supplier has accepted it. Retrying blindly could buy twice, and giving up could leave cash reserved for an order that exists. Each purchase therefore moves through explicit states (Figure 3-10). Its action ID is the idempotency key: the simulator replays a repeated ID with the same body and rejects one with a different body. A lost reply moves the action to UNKNOWN with its cash still reserved, and reconciliation runs after 5, 10, 30 and 60 seconds, always reading the receipt first and resending the same ID only when no receipt exists. The ledger entry for an order is keyed by a digest, so it is booked once, and a store gate admits one unresolved purchase per store. Evidence that contradicts itself stops automatic bookkeeping and flags the action for manual review.

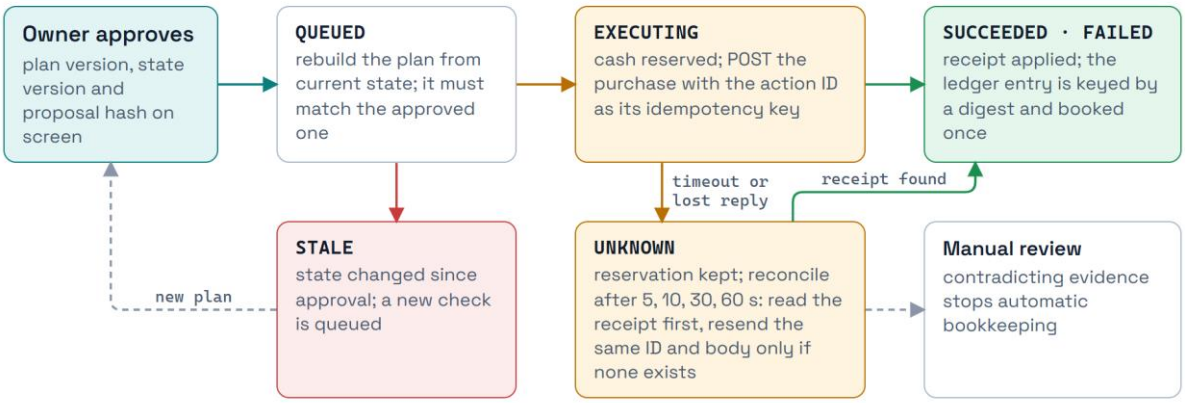


Figure 3-10. Purchase execution states. After a lost reply the action waits in UNKNOWN with its cash reserved until reconciliation finds the receipt or confirms that none exists, so each approved purchase is placed once.

Resuming an agent run without repeating a write. When a worker dies, its lease passes to another worker, which resumes the run from the last checkpoint. Two things could go wrong: the old worker could still write a checkpoint, and the replayed graph could call a write tool a second time. Every checkpoint write re-checks the job's lease on the same database connection, before and after the write, so only the current lease holder can save state. Tool invocation IDs are deterministic (run, model call, position), and each result is stored under the invocation ID with a hash of its arguments; a replayed call returns the stored result, and a call with the same ID but different arguments is refused. An answer to a clarification is bound to the ID of the question it answers.

Consuming the simulator's event stream. Events arrive in pages and may be repeated or out of order after a restart. A page is accepted only if it continues the store's cursor without a gap, each event carrying exactly the next sequence number. An event repeated with the same content hash has no effect; the same ID with different content is a conflict that rolls back the batch, and the

conflicting evidence is saved for inspection. Purchase receipts referenced by the batch are fetched before the database transaction, so the batch is then applied atomically.

Bursts of events and duplicate re-plans. A day of business produces many events, and each could trigger a re-plan. Partial unique indexes allow one active check job per mission and one active sync job per store. A new trigger merges into the active job and raises its target versions; if that job is already running, a single follow-up check is queued. The plan's input hash ignores synchronisation timestamps, so an unchanged plan is reused, and a purchase the owner has already rejected is not proposed again.

Lock contention in the job dispatcher. Two contention bugs were reproduced in tests. The dispatcher selected the first 100 due jobs and then skipped those whose store was locked, so a busy first page starved every later store; it now skips locked stores before the limit (`FOR UPDATE OF store SKIP LOCKED`). Recovering orphaned purchases waited on a busy store's lock and blocked the claiming of unrelated jobs; it now takes each store lock without waiting and moves on.

Sourcing behind short request timeouts. A sourcing run reads three suppliers and takes tens of seconds, longer than the client's 25-second request timeout. The request therefore starts the run as a background asyncio task, held by a strong reference so it is not garbage-collected, and returns at once. Each reviewer's card is written as soon as it is verified, and the page polls the run's progress. Updates apply only while the run is still the case's current run, and a run past its 300-second deadline is shown as interrupted.

Checking model citations without rejecting correct work. Verbatim citation checks (Section 2.6) first rejected correct answers in three ways, each found by examining failed evaluation runs. Reviewers quoted a table row together with the "unit: box" line it depends on, which are not adjacent in the document; a quote of several lines now counts when every line occurs in the document. Models copied 64-character evidence IDs imperfectly, cutting them short or repeating a stretch; an ID now resolves when its first 24 characters match exactly one supplied ID, and the full ID is written back. Finally, the runtime's rule that only tool results may be cited conflicted with the explanation step, whose evidence is supplied in its context; gpt-6.1-sol followed the rule literally and cited nothing until one sentence stated that supplied evidence may be cited.

Differences between model APIs. The Responses API rejected output items replayed as input when they carried the SDK's attribute name `async_` and null fields; the adapter now serialises them by alias without null values before replaying them. In the evaluation harness, tool choice and temperature are configurable per model, because DeepSeek's thinking mode refuses a forced tool choice and gpt-6.1-sol refuses a temperature setting.

A context budget for long business state. The context size is estimated in UTF-8 bytes (Section 2.9). A recovery plan alone takes about 12,000 bytes, so a conversation about a recovery case exceeded the default 24,000-token input budget. The budget is a per-deployment setting, and the demo configuration sets it to 64,000. When the context still does not fit, earlier answers give way first and learned skills second; if the required content alone exceeds the budget, the run ends with an explicit context error, and the owner's messages are never cut.

Settlement dates on a simulated clock. The recovery solver needs the date of each of the seven settlement days. These are derived from the store's simulated clock, and a saved scenario is re-dated to the clock when it is analysed, so a case created on one simulated day and analysed on another still covers the right seven days. Supplier replies such as a delivery date are read against the simulated date as well.

A bilingual interface. The interface has an English and a Chinese version. Translations are keyed by the source strings and looked up at render time, and a checker parses every Vue template and script to flag interface text without a translation, empty translations and mismatched placeholders.

Table 3-6. Implementation challenges and where the solutions live.

Challenge	Mechanism	Code
Exactly-once purchasing	idempotent action ID, UNKNOWN state, receipt-first reconciliation, digest-keyed ledger, store gate	`backend/app/execution/jobs.py`, `accounting.py`; `simulation/simulator/repository.py`
Resuming agent runs	lease-fenced checkpoints, deterministic invocation IDs with argument hashes	`agent/src/shopsteward_agent/checkpoint.py`, `runtime.py`; `backend/app/agent_bridge/tools.py`
Event stream	contiguous pages, content hashes, atomic batches	`backend/app/operations/jobs.py`, `repository.py`
Duplicate re-plans	partial unique indexes, job merging, input hash	`backend/app/scheduling/models.py`, `repository.py`; `backend/app/planning/jobs.py`
Dispatcher contention	skip locked stores before the limit; non-waiting store locks	`backend/app/scheduling/dispatcher.py`; `backend/app/execution/jobs.py`
Long sourcing runs	background task, incremental cards, polling, deadline	`backend/app/sourcing/router.py`, `run.py`
Citation checks	multi-line quotes, evidence-ID prefixes	`agent/src/shopsteward_agent/offer_cards.py`, `cases/runner.py`
API differences	alias serialisation, configurable tool choice and temperature	`agent/src/shopsteward_agent/model.py`; `posttraining/src/shopsteward_pt/eval/policies.py`
Context budget	byte estimate, ordered omission, configurable limit	`agent/src/shopsteward_agent/context/builder.py`
Simulated clock	settlement dates from the store clock	`backend/app/operations_cases/processor.py`; `frontend/app/utils/recovery.ts`
Bilingual interface	source-keyed catalogue, template checker	`frontend/app/i18n/`; `frontend/scripts/check-i18n.mjs`

3.6 Testing and verification

The system is verified at four levels: unit tests of the decision logic, integration tests against real databases and services, end-to-end runs of the interface in a browser, and static checks that keep the client and the service in agreement. The evaluation harnesses of Chapter 4 add a fifth level that judges model behaviour by its effect on business state. Table 3-7 summarises what each level checks.

Table 3-7. Verification levels.

Level	What is checked	How
Unit	planner rules, recovery solver, quotation calculator, task constraints, evidence policy; the forecast service's causal input (future sales cannot reach the features) and its parity with reference predictions on all 300 series	pytest; pure functions, no database
Agent runtime	graph transitions, interrupts and resumes, tool replay, both API modes on the wire	pytest with scripted model doubles; the OpenAI protocol through an HTTP mock transport
Integration	job queue, purchase execution and reconciliation, event sync, recovery cases, migrations, knowledge ingestion and search	pytest against a dedicated PostgreSQL test database migrated to the latest revision; each knowledge test creates and drops its own schema; OpenSearch for ingestion tests
End-to-end	the owner's journeys: the same task across pages and refreshes, recovery flows with partial confirmations, documents, forecasts, overview charts, onboarding, language switching	Playwright drives the real Vue interface; most specs serve recorded API responses to reach exact protocol states, and selected runs use the real backend (the SC01 restocking flow) and real models
Static	strict TypeScript, Ruff lint and format, migration drift (`alembic check`), API contract drift, untranslated interface text	the client's API types are regenerated from the exported OpenAPI schema and compared byte for byte with the committed files
Behaviour (Chapter 4)	ordering decisions, planning, sourcing worlds, explanations, retrieval, quotations	exact enumeration for planning; execution-mode predicates on business state before and after a dialogue (for example "current plan unchanged", "no purchase created", "purchase within

Level	What is checked	How
		cap”); answer keys of the generated worlds; the solver as oracle; Recall@5, MRR@5 and nDCG@5

Pull requests to `main` run a Windows workflow that installs the locked dependencies, checks the API contract, type-checks and builds the client, and runs the platform-independent backend tests.

4 Experiments

Each section of this chapter takes one design choice from Chapter 2, compares it with its alternatives on data that the choice was not tuned on, and reports results scored by code. Section 4.1 describes the common method. Sections 4.2 to 4.5 cover the quantitative core (forecasting, planning search, the ordering policy and its post-training), Sections 4.6 to 4.9 the language-model pipelines (sourcing and explanation, and the prompt evolution that tunes them), and Sections 4.10 to 4.12 retrieval, quotation processing and delegated work over time. Section 4.13 maps every design decision to its evidence.

4.1 Method

Partitions. Every experiment separates the data used to make a choice from the data used to report it (Table 4-1). Forecasting models were selected on development periods and are reported on the eight weeks after the training cutoff; the planner's beam width was fixed on five development scenarios before the 30 test scenarios were run; prompt evolution reads feedback from one partition, selects on a second and is reported on a gate partition it never sees; the post-trained policy was selected on a development set and reported once on a locked test.

Scoring by code. No experiment uses a judge model. Each result is checked against a reference that code can compute: the recovery solver's recommendation, the answer key of a generated sourcing world, predicates over the business state before and after a dialogue (for example "the current plan is unchanged" or "no purchase was created"), exact decimal values for quotations, and the known relevant document versions for retrieval.

Comparisons. Conditions are compared on the same cases. Where a difference is summarised, the 95% interval is a percentile bootstrap over independent cases (10,000 resamples): scenarios for planning, task families for post-training.

Table 4-1. Experiments, their data and how they are scored.

Section	Question	Reported on	Scored by
4.2	Which forecasting model feeds the planner?	300 M5 series, 8 review weeks (2,400 product-weeks)	weekly WAPE against actual sales
4.3	Does beam search find the best order sequence, at what cost; does checking on every update help?	30 seven-day scenarios (beam width fixed on 5 others)	exact enumeration; unmet demand and stock carried in simulation
4.4	Rules or a language model for ordering decisions?	200 dialogues; 140 executed against the real backend	gold actions and arguments; business-state predicates
4.5	Does SFT + GRPO train a small open model for the same contract?	locked test of 400 tasks	execution in isolated backend copies
4.6	One reader per supplier or one for all?	12 worlds, 42 suppliers	answer keys of the generated worlds
4.7	Does the sourcing	12 worlds (24 in one	five stage checks

Section	Question	Reported on	Scored by
	loop reach the right purchase end to end?	round)	against the answer key
4.8	Which orchestration mode explains best?	24 gate cases	solver output as oracle
4.9	Does GEPA improve the prompts?	24 gate cases; 36 validation runs	as 4.7 and 4.8
4.10	Does retrieval return the right document versions?	60 questions (40 dev / 20 test)	known relevant versions
4.11	Are quotation fields exact, and does a saved rule carry over?	30 held-out quotation files	reference answers in exact arithmetic
4.12	Does delegated work survive restarts, timers and new conversations?	process acceptance runs with a real model	durable state and message counts

4.2 Forecast model selection

Question. Which model should give the planner its weekly demand? The forecast fixes the quantity every plan is built on, so the choice was made on accuracy over whole weeks.

Data and protocol. The 300 M5 series of Section 3.1.1 were used throughout. Models were compared on three development anchors in 2015 (four weekly origins each, 3,600 product-weeks), each with early stopping on the 28 days before the anchor; the final number of boosting rounds or epochs is the median across anchors. The selected recipe was confirmed on four weeks from 2016-02-01 and then trained to the cutoff of 2016-03-27. The review covers the eight weeks after the cutoff (2016-03-28 to 2016-05-22), one forecast origin per week, without retraining: 2,400 product-weeks and 16,800 daily values.

Metrics. For each product and origin, the predicted seven-day total P is compared with the actual total A. The headline metric is **weekly WAPE** = $\Sigma|P - A| / \Sigma A$; bias = $\Sigma(P - A) / \Sigma A$; daily MAE is reported alongside. A candidate had to beat the 28-day moving average by at least 5% with |bias| ≤ 10%.

Path to the ensemble. The first model, a daily LightGBM with Poisson loss, improved on the 28-day average by 4.4% on a 500-series pilot, short of the 5% bar. Two changes carried the next versions: early stopping on weekly error instead of daily error (6.0% better on the same model), and features aligned to the target day. Applying the resulting models unchanged to 300 new products lost to the moving average, so from v5 the models were trained on the review panel's own history. v5 blended a weekly-total LightGBM (W2, L1 loss) with a multilayer perceptron (N3) and passed the bar with a 5.6% gain. v6 added multiscale features (88 in total) and new objectives, and selected by development WAPE: an L1 model per product category (F6), a Tweedie model (F3) and the v5 blend, weighted 0.5 / 0.25 / 0.25 (Table 2-3).

Table 4-2. Eight-week review, 2,400 product-weeks.

Model	Weekly WAPE	Bias	Daily MAE

Model	Weekly WAPE	Bias	Daily MAE
Seasonal naive (same weekday last week)	43.91%	—	—
28-day moving average	37.26%	−2.47%	—
N3 MLP alone	36.24%	−5.66%	0.849
F6 LightGBM, L1 per category, alone	35.51%	−7.87%	0.873
F3 LightGBM, Tweedie, alone	35.41%	−1.50%	0.894
W2 LightGBM, weekly L1, alone	35.31%	−7.32%	0.876
v5 blend (W2 + N3)	35.18%	−6.49%	0.849
v6 ensemble	**34.98%**	−5.93%	0.869

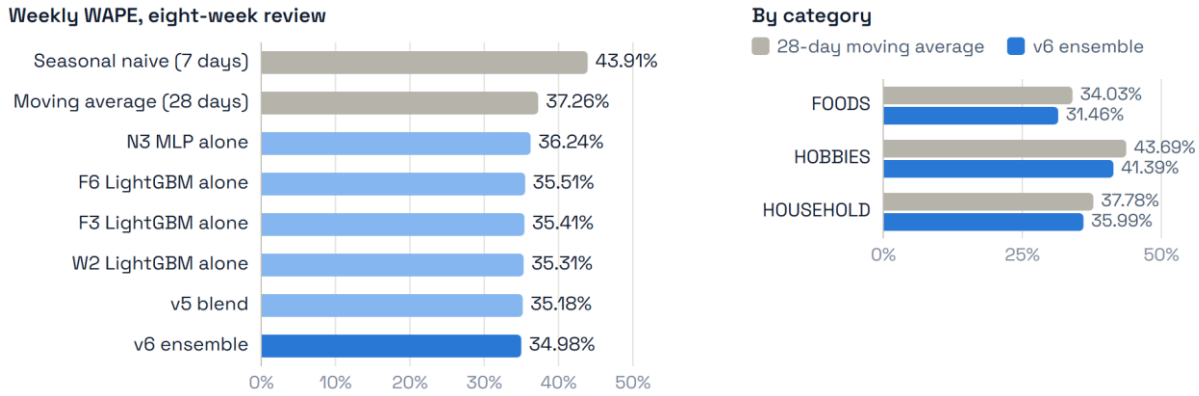


Figure 4-1. Forecast accuracy on the eight-week review. The ensemble is more accurate than each of its components and than the moving average in every product category.

Results. The v6 ensemble reached a weekly WAPE of 34.98%, 6.1% better than the 28-day moving average (37.26%) and better than each of its components alone (Table 4-2, Figure 4-1). The components err differently: the Tweedie model is nearly unbiased (−1.5%) where the L1 models under-forecast by about 7–8%, and the MLP predicts the daily shape directly; blending them lowers the error below the best single model. The gain holds in every category (FOODS 34.03% → 31.46%, HOBBIES 43.69% → 41.39%, HOUSEHOLD 37.78% → 35.99%) and in every one of the eight review weeks. Grouping the series by their share of zero-sales days shows where the difficulty lies: 26.4% WAPE for series with fewer than 30% zero days against 27.7% for the moving average, and 60.7% against 63.9% for series with more than 70% zero days.

4.3 Planning search: daily rule, beam search and enumeration

The project proposal set out to compare three ways of planning orders over a week: a daily replenishment rule, beam search and exhaustive enumeration, and to measure the effect of re-planning as soon as an update arrives. Both experiments were run on the multi-period planner of Section 2.4.

4.3.1 Search quality and cost

Scenarios. Thirty-five seven-day scenarios were generated from real M5 sales weeks: five for development and thirty for testing (ten stable, ten with a demand revision, ten with a delivery closure), with no M5 series shared between the two sets. In the test set weekly demand ranges

from 31 to 135 units, opening stock from 0 to 35 units and cash from ¥500 to ¥1,600 against a ¥300 floor; the unit price is ¥10, the lead time is one or two days, and the permitted order quantities are 0, 20, 40 and 80. A demand revision multiplies the remaining days' demand by 1.4–1.8 (seven in ten) or 0.5–0.7 from a day between 2 and 5. A delivery closure is announced on day 2, 3 or 4 and starts the day after an order placed on the announcement day would arrive; deliveries due during the closure arrive on the first day after it.

Daily loop. Each simulated day follows the proposal's order: receipts, the scheduled planning run, the release of any update, and sales. Only the first order of the chosen plan is placed; the planner runs again the next day. All three methods rank complete order sequences by the same lexicographic score over the visible forecast: unmet demand, then stock carried (unit-days), then total quantity.

Methods. The *daily rule* chooses the feasible quantity with the least forecast shortage before the next daily order could arrive, counting stock in transit, and the smaller quantity on a tie. *Beam search* extends each partial plan by every permitted quantity, drops infeasible extensions, completes each survivor with the daily rule to score it, and keeps the best B partial plans; it returns the best complete plan it has scored. *Enumeration* scores every sequence (16,384 on day 1). The beam width was swept from 1 to 16 on the development scenarios, and the smallest width with no gap to enumeration, $B = 1$, was fixed before the test run.

Table 4-3. Planning methods on the 30 test scenarios (daily replanning, 210 decisions).

Method	Decisions at the optimum	Excess projected stock (unit-days)	Unmet demand (units)	Stock carried (unit-days)	Plans evaluated per decision	Time per decision
Daily rule	195 (92.9%)	190	360	1,506	4	0.04 ms
Beam search, B = 1	**210 (100%)**	**0**	384	**1,389**	7.1	0.22 ms (max 1.0)
Enumeration	210 (100%)	0	384	1,389	3,121	2.11 ms (max 17.0)

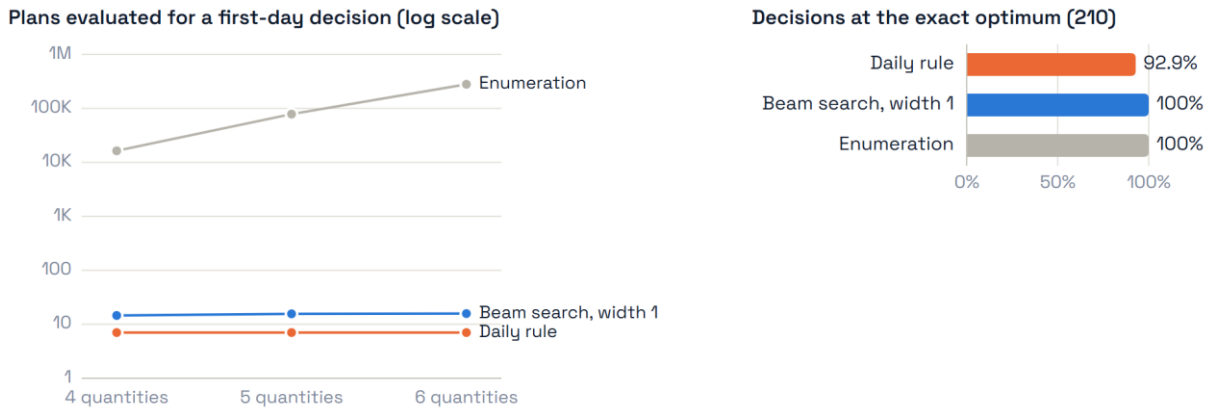


Figure 4-2. Left: plans evaluated for a first-day decision as the number of permitted quantities grows; enumeration grows exponentially while beam search stays near 15. Right: share of the 210 decisions that reached the exact optimum.

Results. Beam search with a width of one chose the optimal plan in all 210 decisions, with 7.1 plans evaluated per decision against 3,121 for enumeration (Table 4-3). Completing each partial plan with the daily rule gives the beam an accurate estimate of every extension, so a single retained plan suffices. To see how the costs scale, the first-day decision of every test scenario was solved again with five and six permitted quantities (Figure 4-2): enumeration grew from 16,384 to 78,125 and 279,936 sequences (17.5, 80 and 270 ms), while beam search stayed at 14.6 to 15.8 evaluations (about 1.2 ms) and remained optimal in all 30 cases at every menu size.

The daily rule never left more projected demand unmet than the optimum, but in 15 of 210 decisions it ordered earlier or more than needed, carrying 190 more projected unit-days. Over the episodes the optimal plans carried 7.8% less stock (1,389 against 1,506 unit-days; paired difference 3.9 unit-days per scenario, 95% interval [1.3, 6.9]). The rule's extra stock also cushioned changes that no planner could foresee, so under daily replanning it left slightly less demand unmet (paired difference 0.8 units per scenario, interval [−2.3, 0.0]). Responding to those changes is the subject of the second experiment.

4.3.2 Response timing: re-planning on each update

Design. The planner was held fixed at beam search, and two conditions were compared on the 20 test scenarios that contain an update: *daily replanning* sees an update at the next morning's planning run; *event-triggered* replanning also runs a check as soon as the update is released, on the same day. Both conditions apply the same cash and delivery rules.

Table 4-4. Daily vs event-triggered replanning (beam search; 10 scenarios per type).

Scenario type	Condition	Unmet demand	Stock carried	Purchases
Demand revision	daily	151	501	21
	daily + event check	**91**	522	25
Delivery closure	daily	172	421	18
	daily + event check	**88**	757	23
All 20	daily	323	922	39
	daily + event check	**179**	1,279	48

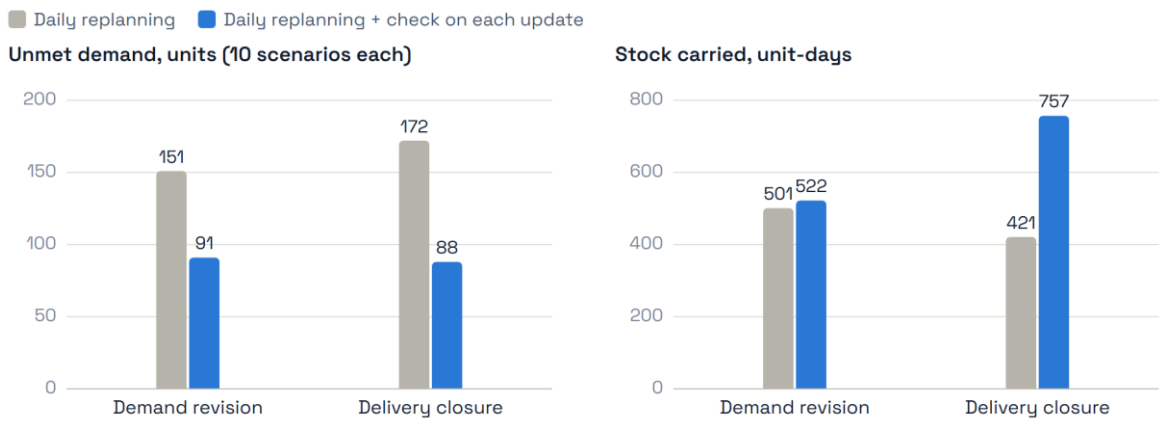


Figure 4-3. Unmet demand and stock carried with and without the event-triggered check. Checking on each update cuts unmet demand by 45%; under a closure it buys ahead, so more stock is carried.

Results. Checking on each update cut unmet demand from 323 to 179 units, by 45% (Table 4-4, Figure 4-3); per scenario the reduction is 7.2 units (95% interval [4.6, 9.9]), 6.0 units for demand revisions ([2.4, 9.9]) and 8.4 for closures ([4.9, 11.9]). A revision is acted on a day earlier; a closure notice is acted on while an order can still arrive before the closure begins, which is why the closure scenarios carry more stock in the event condition (757 against 421 unit-days): the goods are bought ahead of the gap. With event checks the daily rule and beam search reach the same unmet demand (179 units), and beam search carries 2% less stock (1,279 against 1,306 unit-days). These results support the event-driven design of Section 2.2: the planner keeps lean plans, and every business event triggers a new check.

4.4 Rules or a language model for ordering decisions

Question. The owner's requests about a plan ("what if I bought only 12?", "make it 20", "cancel that") must be mapped to exactly one of five actions with exact arguments (Section 2.8). Can a hand-written rule policy do this, or does it need a language model?

Data and conditions. The 200 dialogues of dev-v1 (Section 3.1.4) cover the ten task types. The **rule policy** applies keyword rules to the last user message (handoff keywords, box quantities and exact quantities lead to clarification, cancellation words remove the cap, conditional wording leads to evaluation, otherwise revision). The **language model** is gpt-5.6-luna through the Responses API at temperature 0, with the five actions as required tools and a single request per decision. Decision mode runs all 200 dialogues on frozen contexts; execution mode runs the 140 core dialogues against the real backend, each in a fresh fixture.

Metrics. A dialogue succeeds in decision mode when every expected step is attempted with the right action, tool arguments strictly equal to the reference, any clarification passes three checks (relevant, no guessing, no request for an ID the system already has) and no extra decision is made. In execution mode it must also leave the business state as required: the receipt is valid, the plan is unchanged or a new plan is pending as appropriate, cash, stock and goods in transit are unchanged, and no purchase was created. An *unnecessary clarification* is a first action of `clarify` where the reference does not ask.

Table 4-5. Rule policy vs language model on dev-v1.

Metric	Rule policy	Language model
Decision success (200 dialogues)	123 (61.5%)	**198 (99.0%)**
Execution success (140 core dialogues)	89 (63.6%)	**139 (99.3%)**
Correct action, attempted steps	181/227 (79.7%)	228/230 (99.1%)
Exact arguments, tool steps	91/151 (60.3%)	151/151 (100%)
Clarification when needed	53/59 (89.8%)	57/59 (96.6%)
Unnecessary clarification	28/141 (19.9%)	**0/141 (0%)**
Correct handoff / correct no-action	3/10 · 3/10	10/10 · 10/10
Writes attempted when no write was wanted	12	0

Metric	Rule policy	Language model
Median time per decision	< 0.1 ms	1.9 s

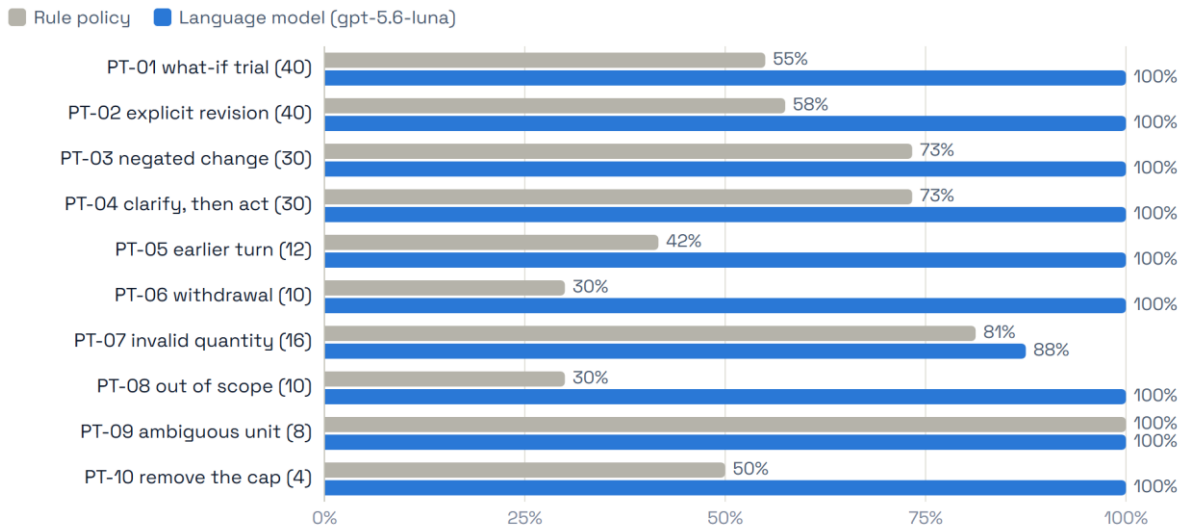


Figure 4-4. Decision success by task type. The rule policy handles the unit-ambiguity cases but fails on negation, references to earlier turns, withdrawals and out-of-scope requests.

Results. The language model succeeded in 99.0% of dialogues against 61.5% for the rules, and in 99.3% of executed dialogues against 63.6% (Table 4-5). The rules fail where meaning depends on more than keywords (Figure 4-4): a negated change ("don't change the plan, just tell me what 12 would cost"), a reference to an earlier turn, a withdrawal or an out-of-scope request. In execution mode the rule policy made seven writes the owner had not asked for; four changed a plan and three were rejected by the backend. The language model made none, and never asked an unnecessary question.

Prompt repair. The language model's three failures (a negative quantity turned positive, a minimum quantity handed off instead of clarified, a re-asked question after the quantity was given) led to an eight-line addition to the instructions: keep the sign of a quantity, separate the current target from negated or quoted numbers, clarify that only caps are supported when a minimum is requested, treat "units" as pieces, and act without asking again once the information is complete. With it, decision success rose to 199/200 and execution success to 140/140, for about 30% more tokens per successful dialogue.

Choice of model. Fifty of the dialogues were then run with four hosted models under the repaired instructions (Table 4-6). All three of deepseek-flash, gpt-5.6-luna and gpt-6.1-sol decided every task correctly, so the choice among them rests on cost and on the other roles they play (Section 3.3).

Table 4-6. Hosted models on 50 ordering tasks (58 decisions, 38 with arguments).

Policy	Tasks	Actions	Arguments	Tokens in / out	Median time
Rule policy	47/50	56/58	35/38	0 / 0	< 1 ms
gpt-5.6-luna	50/50	58/58	38/38	55,651 / 4,006	2.1 s
deepseek-flash	50/50	58/58	38/38	79,837 / 8,379	1.0 s

Policy	Tasks	Actions	Arguments	Tokens in / out	Median time
deepseek-v4-pro	49/50	57/58	38/38	83,417 / 9,838	2.2 s
gpt-6.1-sol	50/50	58/58	38/38	55,590 / 2,623	3.0 s

4.5 Post-training: SFT and GRPO

Question. Can a small open model be trained to make the same ordering decisions, with rewards computed by executing its actions (Section 2.11.1)?

Data and protocol. Qwen3-8B was fine-tuned with LoRA on 3,000 verified demonstrations and then trained with GRPO from 1,000 initial states (Section 3.1.6). Every design choice (SFT length, reward design, advantage normalisation, the NLL anchor, sampling) was made on a 200-task development set; the 400-task locked test (40 template families of 10 tasks) was run once at the end. A task succeeds when every required check passes, the end state truly holds and no critical violation occurs.

Table 4-7. Locked test, 400 tasks.

System	Success	Episodes with a critical-violation attempt
Rule policy	304 (76.0%)	24
Qwen3-8B without training	292 (73.0%)	31
SFT, one epoch (S10)	348 (87.0%)	12
Best SFT-only model (S12)	356 (89.0%)	10
SFT + GRPO (R25)	**376 (94.0%)**	**2**
Hosted frontier model (reference)	384 (96.0%)	1

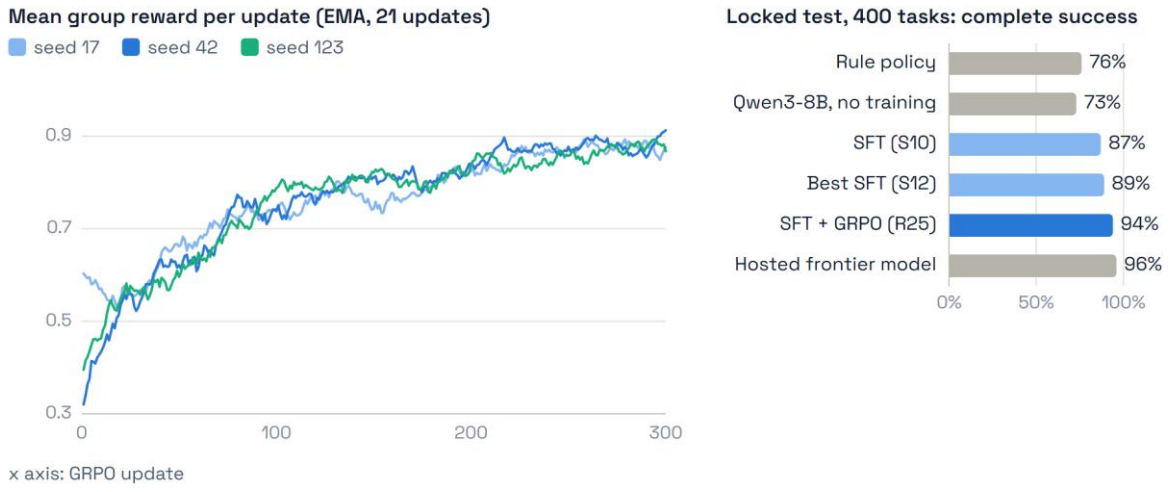


Figure 4-5. Left: mean group reward during GRPO for three seeds. Right: success on the locked test.

Results. SFT followed by GRPO raised success from 73.0% for the untrained model and 89.0% for the best SFT-only model to 94.0%, within two points of the hosted frontier model used as a reference (Table 4-7). Against the best SFT-only model the failures fell from 44 to 24 (45% fewer):

28 tasks were solved only by the GRPO model and 8 only by the SFT model. The family-level paired bootstrap gives a gain of 5.0 percentage points (95% interval [2.25, 8.00]). Episodes with an attempted critical violation fell from 10 to 2, and the backend rejected both attempts, so no violation took effect. Training raised the mean group reward from 0.55 over the first 30 updates to 0.88 over the last 30 (seed 42), and the three seeds ended within two development tasks of each other (189, 190 and 188 of 200).

Table 4-8. Locked test by task family: best SFT-only model (S12) vs SFT + GRPO (R25).

Family	Tasks	S12	R25
Read-only trial or negated change	100	94	**98**
Explicit revision	100	88	**95**
Clarify, then continue	80	65	**72**
Reference to an earlier turn	40	33	**35**
Quantity and wording boundaries	40	38	38
Withdrawal or out of scope	40	38	38

What the ablations showed (development set, 200 tasks). Rewarding the end state alone (184) worked better than rewarding valid format (176) or action and arguments (181); standardising advantages within each group and anchoring to the SFT demonstrations together gave the best branch (R25, 190), against 186 with standardisation alone and 185 with centring alone. Doubling the group size to eight added one task (191), not enough to justify twice the rollouts; sampling only hard tasks (181) made the model ask too many questions.

4.6 One reviewer per supplier or one reader for all

Question. Should each supplier's documents be read in a separate call, isolated from the others, or should one call read every supplier together (Section 2.6)?

Design. The 12 smoke worlds (42 suppliers) were read in both arrangements with the same instructions and output format. This comparison isolates the reading arrangement: documents are placed directly in the prompt, without tools or citation checks. A card is *decision-safe* when its status, unit price, minimum order, pack size, lead time and validity dates are all correct, and *exact* when every graded field is correct; a world's *decision* is right when the solver, given the model's cards, recommends what it recommends from the true cards.

Table 4-9. Reading arrangement on 12 worlds (42 supplier cards).

Model	Arrangement	Right decisions	Decision-safe cards	Exact cards	Invalid cards	Median time per world
gpt-5-nano	one call per supplier	**7/12**	**33**	**29**	**4**	**51.5 s**
	one call for all	4/12	21	14	15	101.5 s
gpt-6-luna	one call per supplier	12/12	**42**	**42**	0	**6.2 s**

Model	Arrangement	Right decisions	Decision-safe cards	Exact cards	Invalid cards	Median time per world
	one call for all	12/12	41	40	0	13.3 s

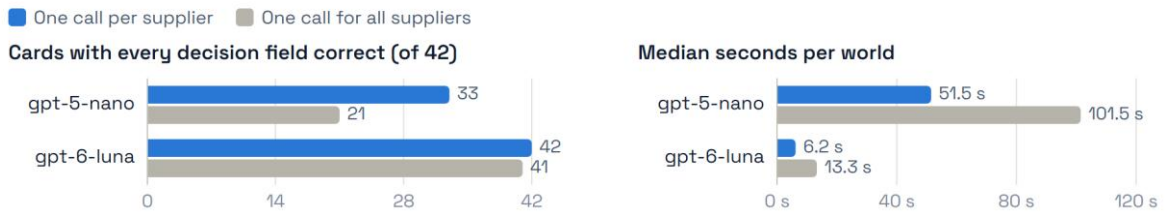


Figure 4-6. Reading each supplier separately produces more correct cards with a small model and halves the time per world, because the readers run in parallel.

Results. With the small model, isolation raised the decision-safe cards from 21 to 33 of 42, the exact cards from 14 to 29, and the right decisions from 4 to 7 of 12, and cut invalid cards from 15 to 4 (Table 4-9). With the stronger model both arrangements read nearly every card correctly, and isolation still halved the time per world (6.2 against 13.3 seconds) because the readers run in parallel. Isolation costs about 15% more input tokens per world (10,150 against 8,812), since the brief is repeated for each supplier. One reviewer per supplier is therefore the design of the sourcing pipeline.

4.7 The sourcing loop end to end

Question. Does the complete loop (reviewer agents reading through their document tool, citation verification with one re-read, the recovery solver and the explanation) reach the right purchase with a faithful explanation?

Metric. A world passes only if all five stages pass: the brief matches the world; every supplier received a card that passed verification; all cards are decision-safe; the recommendation equals the answer key's; and the explanation states every required claim (the recommended candidate, its remaining gap and the gap of waiting) without contradicting the solver.

Table 4-10. End-to-end sourcing runs (one model in every role unless noted).

Round	Model	Worlds	Passed	Decision-safe cards	Tokens in / out per world	Median time per world
10-08	gpt-6-luna	smoke 12	8	42/42	20,894 / 5,080	32.5 s
10-09	deepseek-flash	smoke 12	11	41/42	24,913 / 17,253	40.1 s
10-11	gpt-6-luna	gate 24	15	77/78	20,472 / 4,970	36.5 s
10-13, before the instruction	deepseek-flash	smoke 12	9	38/42	26,890 / 15,888	53.8 s

Round	Model	Worlds	Passed	Decision-safe cards	Tokens in / out per world	Median time per world
fix						
	gpt-6.1-sol	smoke 3	0	11/11	22,865 / 2,506	28.7 s
10-13, after the fix	**deepseek-flash**	smoke 12	**12**	42/42	23,720 / 15,553	44.3 s
	gpt-6.1-sol	smoke 12	**12**	42/42	21,124 / 4,002	59.4 s

Results. Reading was accurate from the first round: the reviewer agents produced 42 of 42 decision-safe cards on the smoke worlds and 77 of 78 on the gate worlds (Table 4-10). The failures of the early rounds were in the explanation, which omitted the gap statement for the recommended order; Section 4.9 shows how GEPA evolved the explanation text until it stated the gap in every validation run. In the final round one more defect surfaced: the runtime's rule that only tool results establish references conflicted with the explanation step, whose evidence is supplied in its context, and gpt-6.1-sol, following the rule literally, cited nothing. One added sentence ("evidence IDs listed below are supplied references; cite them as you would a successful tool result") resolved it. After the fix, both deepseek-flash and gpt-6.1-sol passed all 12 worlds end to end, gpt-6.1-sol with about a quarter of the output tokens. Read on its own, deepseek-v4-pro was also correct on all 42 suppliers, at a median of 22.5 seconds per supplier.

4.8 Four orchestration modes for the explanation

Question. Should the solver's result be explained by one call over the frozen evidence, by one agent with tools, or by a team of role experts (Section 2.7)?

Design. The four modes ran on the 24 gate cases of the recovery explanation set (eight families × 3) with deepseek-flash and the seed text, one run per case, under the shared budget of 14 model calls, 28 tool calls and 150 seconds. An explanation succeeds when the merged result is usable, every required claim is stated, no claim contradicts the solver and there is no critical failure, such as an unnecessary question to the owner.

Table 4-11. Orchestration modes on 24 gate cases.

Mode	Correct	Model calls per case	Tool calls per case	Tokens in / out	Median time	Unnecessary questions
fixed	**23**	**1**	0	4,546 / 6,376	22.2 s	**0**
single agent	21	2	4.0	13,754 / 4,817	17.7 s	1
static team	21	6.3	6.5	36,772 / 12,669	27.3 s	3
adaptive team	18	7.8	6.8	44,372 / 14,271	34.1 s	6

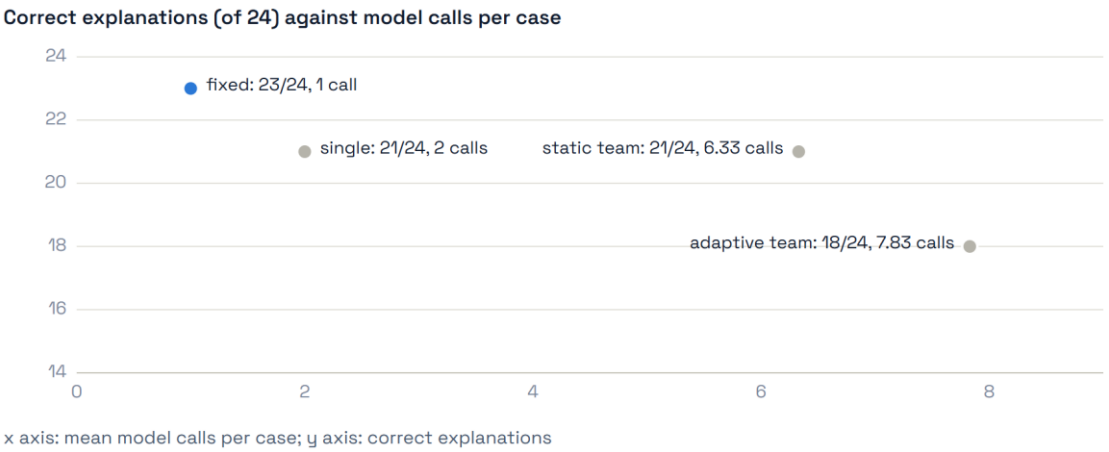


Figure 4-7. Correct explanations against model calls per case. One call over the frozen evidence explained the most cases at the lowest cost.

Results. The fixed pipeline explained 23 of 24 cases correctly with one model call per case, the best of the four modes at about a tenth of the team modes' input tokens (Table 4-11, Figure 4-7). The evidence of a recovery case is frozen and fits in one context, so a single call sees all of it at once; the team modes split the same evidence across experts and add the chance that an expert asks the owner a question the evidence already answers (six such questions in the adaptive team). No mode contradicted the solver. The fixed pipeline is therefore the default, and the team modes remain available as configuration.

4.9 Prompt evolution with GEPA

Question. Can GEPA's reflective search improve the instructions without changing model weights (Section 2.11.2)?

Expert team (`gepa-r1`). GEPA evolved the three expert questions with deepseek-flash as the task model and deepseek-v4-pro as the reflection model, drawing feedback from 12 evo-train cases and selecting on 12 evo-val cases, with a budget of 100 evaluations (105 used). The validation score rose from 8 of 12 for the seed text to 11 of 12 for the fourth accepted candidate, which changed the evidence and impact questions (Figure 4-8). On the 24 gate cases, which the search never saw, the evolved text improved all four modes (Table 4-12). In the adaptive team, unnecessary questions to the owner fell from six to two and follow-up calls from 24 to 17.

Table 4-12. Gate cases correct (of 24), seed text vs `gepa-r1`, paired by case.

Mode	Seed	gepa-r1	Failed → passed	Passed → failed
fixed	23	**24**	1	0
single agent	21	**24**	3	0
static team	21	**22**	3	2
adaptive team	18	**22**	6	2

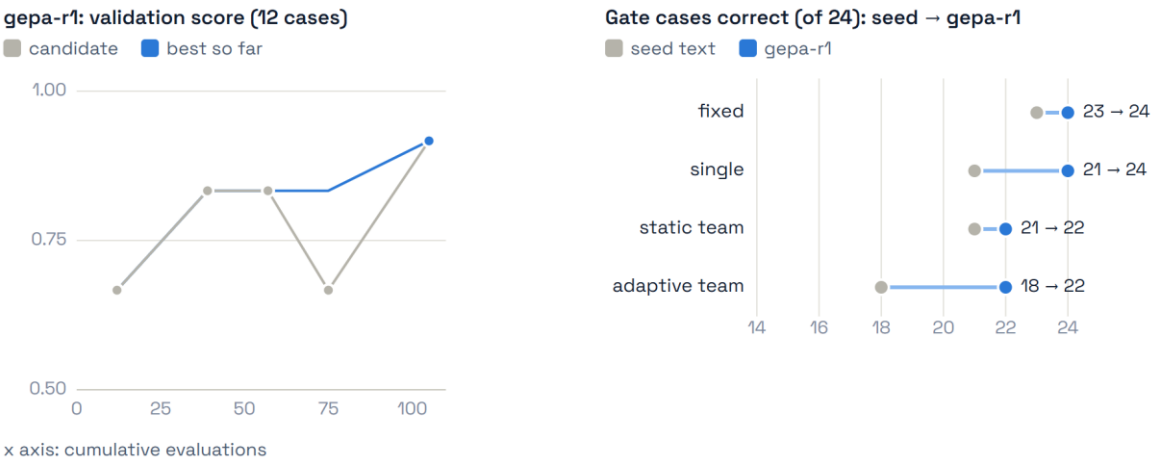


Figure 4-8. Left: validation score of each candidate during the gepa-r1 search. Right: gate results of the seed text and the evolved text in each mode.

Sourcing explanation (`sourcing-r1-s0`, `sourcing-r1-s1`). Two searches evolved the explanation step of the sourcing loop, with gpt-6-luna as the task model and deepseek-v4-pro reflecting, a budget of 200 evaluations each and validation on 12 evo-val worlds run three times (36 runs). The first search raised validation passes from 28 to 35 of 36, the second from 22 to 36 of 36. Both searches independently added the same rule to the impact question: state the gap for every candidate, including when it is none, which is exactly the claim the early end-to-end rounds had omitted (Section 4.7). Reflection for both searches cost about ¥5. `sourcing-r1-s1` is the explanation text used by the product and by the final end-to-end round.

4.10 Retrieval

Question. Does lexical retrieval return the right document versions from a mixed-format corpus, under the store, publication and validity filters?

Design. The K0 corpus (Section 3.1.5) was parsed into 165 chunks and searched with BM25 over the CJK analyzer, with every result filtered to the store, the published versions and the documents valid on the query date. The full question is sent as the query, the top 50 chunks are deduplicated to the first five document versions, and each question is scored against its known relevant versions. Recall@5 is the share of relevant versions in the top five; MRR@5 and nDCG@5 reward placing them first.

Table 4-13. Lexical retrieval on K0.

	Development	Test
Questions (with a relevant document)	40 (35)	20 (18)
Recall@5	88.1%	97.2%
MRR@5	0.814	0.944
nDCG@5	0.817	0.938
Results outside the allowed scope	0	0
Latency p50 / p95	10.0 / 13.4 ms	9.7 / 13.2 ms

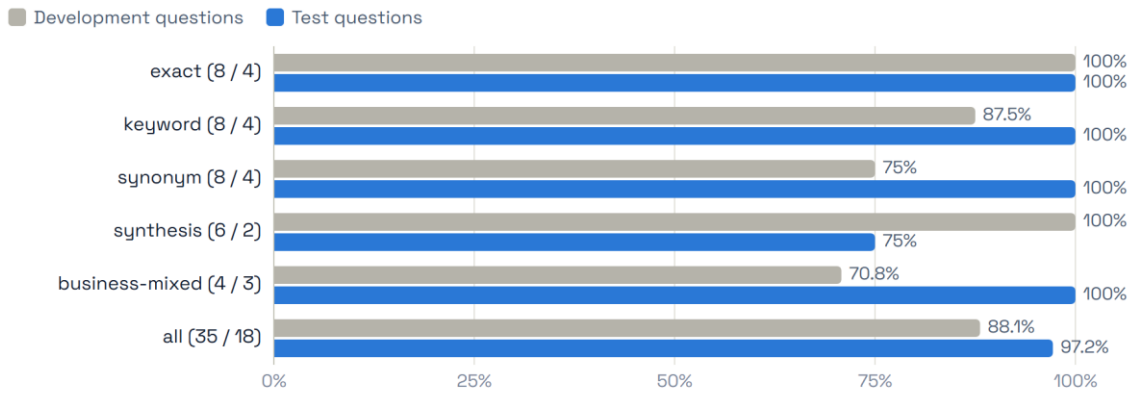


Figure 4-9. Recall@5 by question type (development / test question counts in brackets).

Results. Lexical retrieval placed the relevant version in the top five for 88.1% of development and 97.2% of test questions, with no result outside the allowed scope and a median latency of 10 ms (Table 4-13). Exact and keyword questions are answered almost perfectly; questions phrased with synonyms are the hardest for keywords (75% on development; Figure 4-9), which is the case the dense channel of the hybrid profile is designed for.

Relation paths and scale. The 12 relation questions (8 development, 4 test) returned the reference paths with both fixed one-to-three-hop joins and recursive traversal, so the simpler fixed joins were kept. On the expanded corpus all 1,124 document versions parsed completely into 16,205 chunks in 18.7 seconds, and a 200-document pilot ingested over HTTP produced 200 ready index generations with the same 5,169 chunks in PostgreSQL and OpenSearch.

Dense channel and the agent. A first test of the dense channel embedded 324 chunks of 32 documents with Qwen3-Embedding-0.6B; dense ranking alone placed every target document in the top six for 8 of 10 development queries (recall 86.7%), with a median of 94 ms to embed a query. An end-to-end run of the hybrid profile through the conversational agent completed all six scenarios, with all 50 citations valid and a median retrieval time of 310 ms.

4.11 Quotation accuracy and rule reuse

The proposal promised two tests of quotation processing: field accuracy on 30 held-out files, and the reuse of an owner's correction on new files.

Held-out files. Thirty CSV files were generated (Section 3.1.8); the development examples, the proposal's two quotations and the file without a pack size, are excluded. There are ten priced per box or pack, ten priced per unit, ten with missing fields, three to six products each, 139 rows in total. Each file uses the English column names in a shuffled order, sometimes with an extra column, stray spaces and any of three accepted spellings of the currency; box prices do not always divide evenly by the pack size, so some unit prices need eight decimal places; minimum orders are given in units or in packs. The reference answers are derived from the facts the files were generated from, in exact rational arithmetic, independently of the calculator.

Field accuracy. Four required fields were scored per row: the unit price (compared exactly), the minimum order in units, the pack size and the source. A field that is genuinely missing counts as correct when the result marks it unavailable.

Table 4-14. Quotation field accuracy on 30 held-out files.

File type	Fields correct
Priced per box or pack	180/180
Priced per unit	176/176
With missing fields	200/200
All	**556/556 (100%)**, including 82 of 82 missing fields marked unavailable

Every field was correct, including the 82 fields that had to be marked unavailable: a box price without a pack size has no unit price, while a unit price stays available when only the pack size is missing.

Rule reuse. The same 30 files were processed twice, each as an independent task: once with the owner's saved rule ("compare by unit price", version 1, which passed its check against the corrected example and a missing-field case before activation) and once without it. A result follows the owner's procedure when comparable unit prices appear in ascending order with unavailable ones last. With the rule, all 30 results followed it with no further correction; without it, one file came out in that order by chance, and the owner would have had to repeat the correction 29 times. The rule is applied by the quotation service, so the outcome does not depend on the model that handles the conversation; the prices in each result come from the new file.

4.12 Delegated work across time: completion and recovery

The proposal's central test is whether the owner can leave a task and come back to work done correctly. Two kinds of evidence address it. The scripted-versus-model comparison of Section 4.4 runs both policies over the same business tools, storage and backend: the model-driven workflow completed 99.3% of executed dialogues against 63.6% for the scripted rule policy, with no unwanted write. The process acceptance runs below exercise the runtime itself with a real model (gpt-4.1-mini) on an isolated database, using genuine worker processes and the normal user interfaces.

Table 4-15. Process acceptance checks of the agent runtime.

Check	Result
Memory across new conversations	3 stores, 24 runs, 51 checks: preferences and skills saved, read back, corrected and deleted through versions 1 → 2 → 3; new conversations read the current plan
Background follow-up	an independent background run started without a new user message; no repeated run during an unchanged 65-second window; a stock event started another background run
Worker killed mid-run	the worker was killed while the run was RUNNING and restarted 14 seconds later; attempt 2 succeeded and published exactly one assistant message
Two workers, one conversation	each of two messages was served by a different

Check	Result
	worker, each run publishing exactly one message
Cancellation	a running run was cancelled: CANCELLED persisted, no assistant message
Clarification	the model asked for clarification; the run waited in WAITING_INPUT and resumed in the same run with one message
Plan checks	three real-model explanations of the 40-unit recommendation and the rejected 80-unit option; a hypothetical cap of 20 left the mission unchanged, an explicit cap of 20 produced a new pending plan

Together with the planning experiments of Section 4.3, these runs show the three behaviours the proposal set out to demonstrate: a task survives the end of a conversation and a process restart, it reacts to a change without being asked, and the owner keeps control over every purchase.

4.13 Summary: design decisions and their evidence

Table 4-16. Each design decision of Chapter 2 and the experiment that supports it.

Design decision (Chapter 2)	Evidence	Section
Weighted LightGBM + MLP ensemble for demand	weekly WAPE 34.98% vs 37.26% for the moving average; better than every component, in every category	4.2
Beam search with rule completion for order sequences	optimal in 210/210 decisions with 7 evaluations vs 3,121; stays near 15 evaluations as enumeration grows to 279,936	4.3.1
Event-driven re-planning	unmet demand −45% (7.2 units per scenario, [4.6, 9.9])	4.3.2
A language model for the five-action ordering decision	99.0% vs 61.5% decision success; 99.3% vs 63.6% executed; no unwanted write	4.4
SFT + GRPO with executed rewards	94.0% vs 89.0% (best SFT) on the locked test, +5.0 pp [2.25, 8.00]; critical attempts 10 → 2	4.5
One reviewer agent per supplier	decision-safe cards 33 vs 21 (small model); half the time per world	4.6
Verified quote cards, solver, explanation in one loop	12/12 worlds end to end for two models	4.7
Fixed explanation pipeline as	23/24 with one call per case;	4.8

Design decision (Chapter 2)	Evidence	Section
default	team modes 18–21 with 2–8 calls	
GEPA on versioned text bundles	gate +1 to +4 cases in every mode; sourcing validation 22 → 36 of 36	4.9
Scoped lexical retrieval as the default profile	Recall@5 88.1% / 97.2%, no scope leak, 10 ms	4.10
Deterministic quotation calculator with a reusable rule	556/556 fields; rule followed on 30/30 new files	4.11
Durable runtime with checkpoints, leases and follow-up	restart, follow-up, cancellation and clarification checks passed with a real model	4.12

5 Findings and Discussion

Chapter 4 tested each design choice on its own; this chapter reads the results together. Section 5.1 draws out the findings that hold across the experiments, Section 5.2 compares the delivered system with the project proposal (Appendix A), Section 5.3 discusses the limitations, Section 5.4 sets out future work and Section 5.5 concludes.

5.1 Key findings

Figure 5-1 collects the headline result of each component, grouped as in Figure 2-1. Read together, the experiments support five findings about how an agent that carries purchasing work over several days should be built.

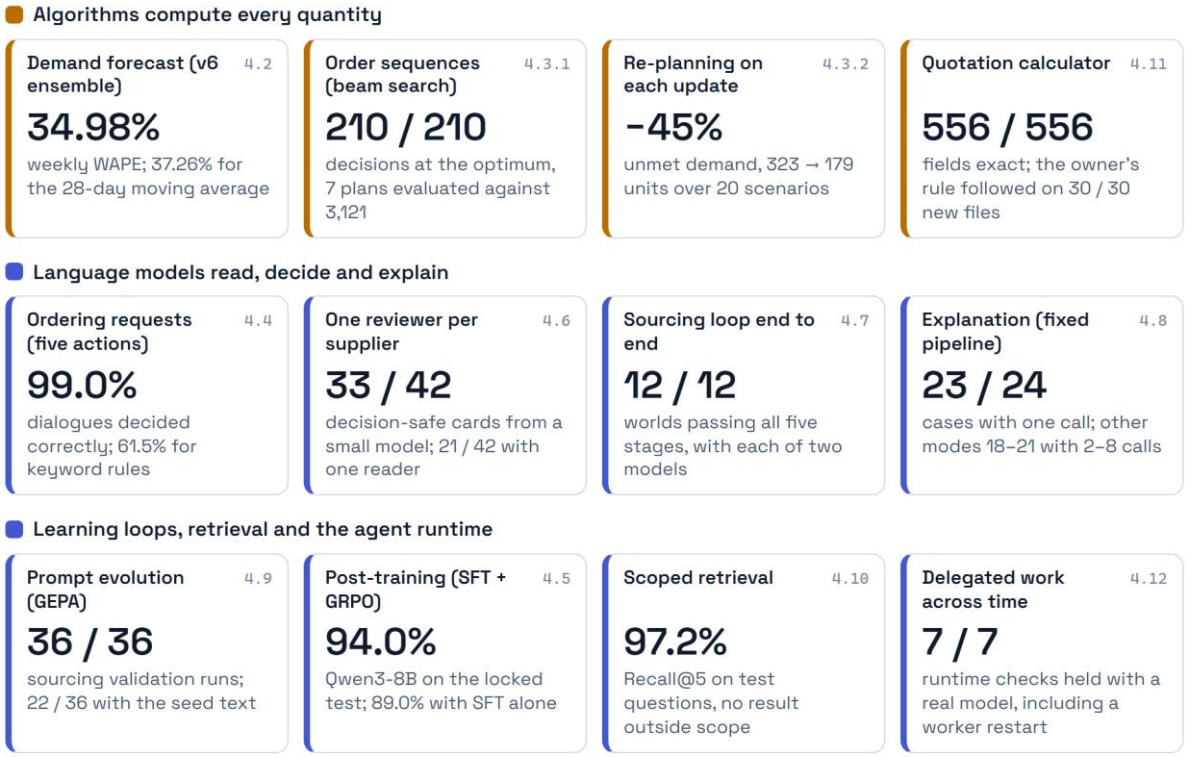


Figure 5-1. Headline result of each component. Amber tiles are components that compute quantities and money, blue tiles are components that use language models, as in Figure 2-1. The number at the top right of each tile is the section that reports the experiment.

Finding 1: algorithms where the answer can be computed, language models where it depends on meaning. Every component with an exact reference reached it. The quotation calculator returned all 556 fields correctly, including the 82 that had to be marked unavailable, and beam search chose the optimal order sequence in all 210 decisions. The owner's requests show the other side: keyword rules decided 61.5% of the dialogues correctly and the language model 99.0%. The rules failed on negated changes, references to earlier turns, withdrawals and out-of-scope requests (Figure 4-4), where the intent depends on more than the keywords present. The two sides meet at typed tools: the model calls `evaluate_plan` or `revise_plan` with an exact quantity cap, the planner builds the plan and the owner approves it. Each side is therefore measured against its own reference, exact arithmetic for the algorithms and business-state predicates for the model, which is why every experiment in Chapter 4 could be scored by code without a judge model.

Finding 2: bounded search keeps plans lean, and event-driven timing reduces unmet demand. Beam search with rule completion matched exhaustive enumeration in every decision while evaluating 7 plans per decision against 3,121 for enumeration, and carried 7.8% less stock than the daily rule. Completing each partial plan with the daily rule gives every extension an accurate score, so a beam of width one already retains the best one. As the menu of permitted quantities grew, the beam stayed near 15 evaluations while enumeration grew to 279,936 sequences (Figure 4-2). Unmet demand responds to timing: checking the plan as soon as an update arrives cut it by 45% (7.2 units per scenario, 95% interval [4.6, 9.9]), and after a closure notice the planner bought ahead of the gap. The search decides what each plan contains; the event-driven core of Section 2.2 decides when a plan is recomputed.

Finding 3: the structure of the evidence decides the structure of the agents. Supplier documents are independent of one another, so giving each supplier its own reviewer helped. With a small model, decision-safe cards rose from 21 to 33 of 42 and invalid cards fell from 15 to 4; with a strong model, the parallel reviewers halved the time per world. The evidence of a recovery case is the opposite: it is frozen and fits in one context. One call that sees all of it explained 23 of 24 cases, while the team modes split the same evidence across experts, used six to eight calls per case and asked the owner questions the evidence already answered. Work is therefore split across agents where the evidence separates and kept in a single call where it must be read together. The system applies this rule in both pipelines: orchestrator–worker for sourcing, and the fixed pipeline as the default explanation mode with the team modes available as configuration.

Finding 4: instructions and weights both improve under code-scored feedback. The predicates that score the experiments also drive the two learning loops. GEPA rewrote instruction text from failures reported in words and improved the gate result of every orchestration mode, by one to four of 24 cases. The two sourcing searches, run independently, added the same rule to the impact question: state the gap for every candidate. This is the claim the early end-to-end rounds had omitted (Section 4.7), so reflection on scored failures located the actual defect. GRPO with executed rewards raised Qwen3-8B from 89.0% after supervised fine-tuning to 94.0% on the locked test, within two points of a hosted frontier model, and cut episodes with an attempted critical violation from 10 to 2. Instructions also need maintenance as models change: gpt-6.1-sol follows instructions literally and cited nothing in the sourcing explanation until one sentence stated that supplied evidence IDs count as references. Keeping instructions as versioned text bundles (Section 2.11.2) lets such a change be made as a new version and compared with the previous one on the same cases.

Finding 5: delegation rests on durable state, timed checks and bound approvals. The proposal's central question was whether the owner can leave a task and come back to work done correctly. The runtime checks of Section 4.12 show the three behaviours it set out to demonstrate. A task survived a worker killed mid-run and was picked up by new conversations; background runs started from the follow-up timer and from a stock event without a new message; and a quantity limit stated by the owner became a new pending plan awaiting approval, while a hypothetical one left the mission unchanged. Corrections carry over as well: the owner's saved quotation rule was followed on all 30 new files, against 1 of 30 without it. These behaviours come from state held in PostgreSQL (checkpoints, missions, versioned memory and skills) and from the event loop, so they persist across conversations and process restarts.

5.2 Deviations from the proposal

The proposal (Appendix A) set out a delegated purchasing task for one shop, one item and one supplier: business rules that keep the cash reserve, beam search compared with a daily rule and exhaustive enumeration, re-planning when an update arrives, and an owner's correction reused as a quotation procedure. All of these were built, and the planning and quotation comparisons ran on

the planned 30 seven-day scenarios and 30 held-out files (Sections 4.3 and 4.11). The delivered system extends the proposal in four directions (Table 5-1).

Table 5-1. Extensions beyond the proposal.

Proposal	Delivered	Reason	Impact
One supplier; a late delivery is followed until the goods arrive	Supplier-failure pipeline: recovery cases, simulation-based optimisation over every feasible order, and one reviewer agent per alternative supplier with verified quote cards (Sections 2.5–2.6)	With one supplier, a failed delivery leaves the owner only the choice of waiting	The owner can recover by buying elsewhere; 12 of 12 worlds resolved end to end (Section 4.7)
Purchases only in simulation	Assisted ordering: the system writes the order message for the owner to send and reads the supplier's reply into verified fields (Section 2.6)	Suppliers found through documents have no connection to the system	A recommendation becomes an order a real supplier can receive, and the owner keeps control of sending it
Demand as a scenario input, with no trained forecasting model	v6 ensemble of three LightGBM models and an MLP, trained on 300 M5 series and serving the planner (Section 2.3)	A shop has its own sales history, and every order quantity rests on the forecast made from it	Weekly WAPE 34.98% against 37.26% for the moving average (Section 4.2); adds the data-mining technique group, so the system covers all four groups against three in the proposal (Table 2-1)
A language model interprets goals, calls tools and explains results	A scored benchmark of the ordering decision, SFT + GRPO post-training of an open model, and GEPA prompt evolution (Section 2.11)	The proposal asked whether model behaviour improves outcomes enough to justify its cost; a scored comparison answers this and also provides a training signal	99.0% for the model against 61.5% for rules (Section 4.4); Qwen3-8B trained to 94.0% (Section 4.5); evolved instructions better in every mode (Section 4.9)

Each extension keeps the proposal's division of labour: the new algorithmic parts compute quantities, the new language-model parts read, decide or explain, and every purchase still passes through the owner's approval.

5.3 Limitations

The system's limitations follow from the scope set in the proposal and from building it as a simulated, single-shop system. Section 5.4 returns to most of them.

1. **Synthetic evaluation data.** Apart from the M5 and DataCo histories, most evaluation sets are generated by seeded programs: the supplier dossiers, the explanation cases, the ordering dialogues, the post-training episodes and the quotation files (Section 3.1). Results on them describe the system under the stated assumptions of each generator, and real documents and requests will vary more.
2. **Forecasts for slow-moving items.** Items that sell on only a few days are the hardest to forecast: weekly WAPE is 26.4% for series with fewer than 30% zero-sales days and 60.7% for series with more than 70% (Section 4.2). Plans for such items rest on a less certain demand estimate.
3. **One product per mission.** Each mission plans one product with its usual supplier. Orders for different products are planned separately, each against the store's cash, and perishable goods, pricing and promotions remain outside the scope set by the proposal.
4. **Validated in simulation.** Sales, deliveries and supplier acceptances come from the digital twin, and the system has not yet been used by a merchant in daily operation. Its effect on an owner's time and stock outcomes in practice is still to be measured.
5. **Dependence on hosted models.** The language-model roles run on hosted models, so cost per call, response time and availability depend on the provider, and a new model version can require changes to the instructions (Section 4.7).

5.4 Future work

Each direction in Table 5-2 builds on a mechanism that is already in place.

Table 5-2. Directions for future work.

Direction	Builds on	Research or market value
Joint planning across products under one cash reserve	Beam search with rule completion, which stayed near 15 evaluations per decision as enumeration grew to 279,936 sequences (Section 4.3.1)	A shop buys many items against the same cash; a joint order menu is a larger search space of the kind the beam handled at low cost
Connection to live store systems	The digital twin's four event types and the per-store event cursor (Section 2.2)	A connector that emits sales, demand updates, supplier acceptances and receipts, and accepts purchases, can take the simulator's place, so the agent runs on a real shop's records
Serving the post-trained model	The model gateway's OpenAI-compatible APIs and the executed-reward harness (Sections 3.3 and 4.5)	An 8B open model at 94.0% on the locked test can be self-hosted for the ordering decision, lowering the cost per decision and keeping the shop's data in house
Hybrid retrieval as the default profile	The dense channel, RRF fusion and the optional reranking step of the knowledge service (Section 2.10)	Questions phrased with synonyms are the hardest case for keywords (75% Recall@5 on development); the dense channel is designed for them, and the K0 set can measure the gain
Field evaluation with merchants	The versioned plans and approvals of every mission and the context manifest written for	Whether owners delegate purchasing in practice needs real use: how often plans are approved, how many

Direction	Builds on	Research or market value
	every model call (Sections 2.2 and 2.9)	corrections become skills, how much time each decision takes

5.5 Conclusion

ShopSteward set out to let a small merchant hand over a continuing purchasing task and return to work done correctly. The delivered system is an event-driven, neuro-symbolic agent. Algorithms compute every quantity: a LightGBM and MLP ensemble forecasts demand, candidate search and beam search plan routine orders, and simulation-based optimisation compares recovery orders when a supplier fails. Language models handle language: reviewer agents read supplier documents into verified quote cards, the conversational agent turns the owner's requests into exact tool calls, and an explanation step states what the solver found. A LangGraph runtime with PostgreSQL checkpoints keeps each task alive across conversations and restarts, and the owner approves every purchase.

The project's academic contribution is a set of controlled comparisons inside one working system, all scored by code. Beam search with rule completion reached the exact optimum at a small fraction of enumeration's cost; re-planning on each update cut unmet demand by 45%; splitting work across agents raised accuracy where the evidence separates, and a single call was best where it does not; and the same executable rewards that evaluate the system also improved it, through GEPA on instruction text in every orchestration mode and through GRPO on model weights, which brought an 8B open model to 94.0%, within two points of a hosted frontier model.

For a small merchant, the result is an assistant that keeps a goal active between conversations, grounds each recommendation in current records, recovers from a supplier failure by finding and pricing alternatives, and gives an account of what happened while the owner was away. The project brings the course's four technique groups (business rules, informed search, data mining and cognitive systems) together with recent research methods, namely orchestrator–worker agents, GRPO with verifiable rewards and reflective prompt evolution, and applies them to a retail purchasing problem built on public data, M5 and DataCo, in a system that runs end to end.

References

[1] IHL Group. Retail inventory crisis persists despite $172 billion in improvements. 10 September 2025. https://www.ihlservices.com/news/analyst-corner/2025/09/retail-inventory-crisis-persists-despite-172-billion-in-improvements/

[2] inFlow Inventory. State of Inventory Management 2026: a survey of 400 operators. 2026. https://www.inflowinventory.com/blog/state-of-inventory-management-2026/

[3] Forthcast. State of Shopify Inventory Operations 2026. Published 18 July 2026; interviews conducted February–March 2026. https://www.forthcast.io/research/state-of-shopify-inventory-ops-2026

[4] Square. Square announces new inventory management features for retailers. Press release, April 2021. https://www.businesswire.com/news/home/20210422005230/en

[5] Mavadia, A. Managing inventory in Shopify just got easier. Shopify, 17 June 2026. https://www.shopify.com/blog/enhanced-inventory-management

[6] Shopify. Proactive business recommendations from Sidekick. Shopify Changelog, 10 December 2025. https://changelog.shopify.com/posts/proactive-business-recommendations-from-sidekick

[7] Sujith, A. Planning with the Demand to Supply to Buy flow. Prediko Help Center, 25 June 2026. https://help.prediko.io/en/articles/12973461-planning-with-the-demand-supply-buy-flow

[8] Baek, J., Fu, Y., Ma, W. and Peng, T. AI agents for inventory control: human–LLM–OR complementarity. arXiv:2602.12631v2, 4 May 2026. https://arxiv.org/abs/2602.12631

[9] Quan, Y. and Liu, Z. InvAgent: a large language model based multi-agent system for inventory management in supply chains. arXiv:2407.11384, 2024. https://arxiv.org/abs/2407.11384

[10] Duan, Y., Hu, Y. and Jiang, J. Ask, clarify, optimize: human–LLM agent collaboration for smarter inventory control. arXiv:2601.00121, 31 December 2025. https://arxiv.org/abs/2601.00121

[11] Dietvorst, B. J., Simmons, J. P. and Massey, C. Overcoming algorithm aversion: people will use imperfect algorithms if they can (even slightly) modify them. Management Science, 64(3), 2018, pp. 1155–1170.

[12] Zhang, Z. Comprehension without competence: architectural limits of LLMs in symbolic computation and reasoning. arXiv:2507.10624, 2025. https://arxiv.org/abs/2507.10624

[13] Makridakis, S., Spiliotis, E. and Assimakopoulos, V. M5 accuracy competition: results, findings, and conclusions. International Journal of Forecasting, 38(4), 2022, pp. 1346–1364.

[14] Ke, G. et al. LightGBM: a highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems 30, 2017.

[15] Ziel, F. M5 competition uncertainty: overdispersion, distributional forecasting, GAMLSS and beyond. International Journal of Forecasting, 38(4), 2022, pp. 1546–1554.

[16] Chao, X., Chen, J. and Wang, S. Dynamic inventory management with cash flow constraints. Naval Research Logistics, 55(8), 2008, pp. 758–768.

[17] NUS ISS. Machine Reasoning Day 1, JR v1, 2026. Informed search strategies and evaluation functions, PDF pp. 46–47.

[18] Anthropic. How we built our multi-agent research system. Engineering blog, 13 June 2025. https://www.anthropic.com/engineering/built-multi-agent-research-system

[19] Shao, Z. et al. DeepSeekMath: pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024. https://arxiv.org/abs/2402.03300

[20] Agrawal, L. A. et al. GEPA: reflective prompt evolution can outperform reinforcement learning. arXiv:2507.19457, 2025. https://arxiv.org/abs/2507.19457

[21] LangChain. LangGraph documentation: persistence, checkpoints and human-in-the-loop. https://langchain-ai.github.io/langgraph/

[22] Nous Research. Hermes Agent: persistent memory and skills. https://hermes-agent.nousresearch.com/

Appendix A Project Proposal

The project proposal is reproduced on the following pages as it was submitted, under the group's earlier number (50); the group has since been renumbered 40.

ShopSteward

Project proposal

A personal operations agent for small merchants

Project proposal | 8 September 2026

Project overview

A promotion is approaching. The owner needs enough stock to meet demand, but every purchase leaves less cash for the rest of the business. Once an order is placed, the work continues: stock must arrive, sales change what is needed, and a late delivery can undo a sensible plan.

ShopSteward is a proposed operations agent that follows this work through. The owner can ask, "Keep this promotion stocked while preserving my cash reserve." The assistant compares purchase options and asks for confirmation. It then follows the order, checks changing conditions after the conversation ends and returns with a new decision when the owner needs to act.

The project brings language-model tool use together with business rules and informed search. Explicit corrections can also become reusable procedures for later tasks. A controlled business simulation will make the consequences visible: what was bought, whether the reserve held, and whether the assistant followed through. On returning, the owner should see what happened and what still needs a decision.

Group information

Canvas group number _____50_____

Member	Full name	NUS student ID
1	Ying Hao	A0326313N
2	Liu Kairui	A0353368W
3	Feng Tangyu	A0356670X
4	Zhang Junming	A0355479L

1 The merchant and the opportunity

A small merchant often serves customers and manages purchasing in the same working day. A stock decision competes for attention with the very sales it is meant to support. The owner may know the business well, yet still have to assemble the latest cash balance, shelf count and supplier information before deciding what to buy.

The decision also has a memory. Forty units ordered yesterday may cover the shortage visible today. Sales recorded today may not become spendable cash until later. A demand revision can make yesterday's purchase insufficient. Each new recommendation therefore depends on what has already happened and what is still outstanding.

Evidence and existing products

Forthcast's 2026 interviews with 13 Shopify operators recorded manual spreadsheet planning in 11 interviews and inventory cash or cash-visibility concerns in six. This vendor-run qualitative study provides a signal from a small recruited panel; its counts cannot be projected to all merchants [4].

Existing approach	Relevant capability	Implication for ShopSteward
Shopify Sidekick [5]	Sales-based replenishment advice, purchase orders and receiving workflows.	Conversational purchasing is an established capability.
Prediko [6]	Demand and supply plans connected to incoming stock, budgets and purchase orders.	A useful proposal must address more than the next order quantity.
Business record systems [3]	Point-of-sale and integrated business records.	Reliable assistance needs current, traceable inputs.

ShopSteward will examine the value of keeping a delegated goal active across conversations and business events, while retaining an explicit correction for later work. The comparison is a test of these behaviours in a defined setting; it does not assume that existing products lack them.

A focused first user

The target is an owner who controls purchasing and sells non-perishable goods with identifiable stock, costs and quantities. This can include an online seller or a neighbourhood retailer. Enterprise Singapore's support for point-of-sale and business systems makes offline retail a relevant product context [3], although demand for this particular assistant remains untested.

The first demonstration centres on one shop preparing for a promotion. Its intended benefit is less repeated explanation and manual checking: the owner sees what changed, why a decision is needed and what will happen after confirmation. Real-user adoption will require subsequent field evaluation.

2 A delegated task in action

Consider a constructed shop scenario. The owner has 20 units in stock, expects demand for 60 and holds 1,000 currency units (CU). A unit costs 10 CU to buy and sells for 20 CU. The owner asks ShopSteward to prepare for the promotion while keeping at least 300 CU available.

The assistant compares quantities of 0, 20, 40 and 80. Buying 40 covers the shortage and leaves 600 CU; buying 80 would leave only 200 CU and is excluded. The owner sees the quantity, total cost and cash effect, then confirms that specific purchase in the interface.

What happens after confirmation

The owner can close the conversation. ShopSteward retains the task and checks the order while it is in transit. Those 40 incoming units count towards the need, so another check should recommend waiting. After delivery and a sale of 10 units, a demand update raises the remaining requirement from 50 to 70. The assistant then explains the new gap and proposes a further 20 units for fresh confirmation.

Event	Cash	Stock	In transit	Unsettled sales
Initial state	1,000	20	0	0
Confirm and order 40	600	20	40	0
Receive 40	600	60	0	0
Sell 10	600	50	0	200
Raise remaining demand to 70	600	50	0	200
Confirm and order another 20	400	50	20	200
Receive the additional 20	400	70	0	200

Table 1 Expected states for a synthetic acceptance case. Cash and unsettled sales are in CU; stock and in-transit quantities are units.

Purchasing reduces cash when the simulated order is accepted; receiving moves units into stock. Sales proceeds remain unsettled throughout this case. The final row is a stocking checkpoint, with subsequent sales and settlement still outstanding. A brief will make these distinctions visible alongside the task history and next check.

Delivery scope

The purchasing demonstration uses one merchant, one item and one supplier. The assistant also supports business queries, background follow-up, task recovery and a quotation procedure reused on a new file. Purchasing and payments occur only in simulation. Multiple merchants, perishables, pricing, marketing and live store connections are outside the proposed delivery.

3 System design and reasoning

The language model interprets the owner's goal, calls bounded business tools and explains their results. Business services calculate options and enforce the rules. The interface sends the owner's confirmation directly to those services. The assistant can request a revised proposal; it has no tool that independently approves or places a purchase.

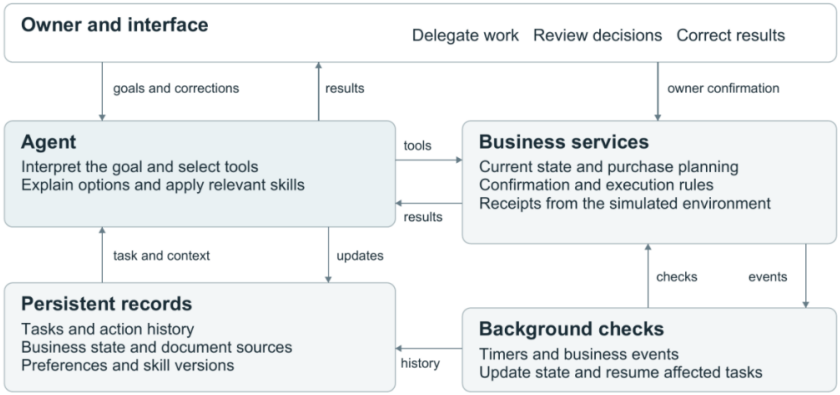


Figure 1 Logical responsibilities. The confirmation path reaches business services directly. Persistent records support both background checks and the assistant's next interaction.

Course technique group	Role in the proposed system
Decision automation	Business rules enforce the cash reserve, current confirmation and one purchase per approved action.
Cognitive techniques	Language-model tool use connects natural-language goals with task context, explanations and saved procedures.
Business resource optimisation	Informed search compares purchase sequences. Beam search is the proposed method, evaluated against a daily rule and enumeration.

These three groups connect through the same task and business state, as required by the course brief [1]. The planner returns numerical options; the model helps the owner interpret them; execution rules remain in force regardless of which option is proposed.

A confirmation is bound to the item, quantity, amount, proposal and relevant state. A material change requires reassessment and a new confirmation. Retries reuse the original action identity; an uncertain response triggers a receipt check. Timers and business events resume the persistent task after records are updated. Closing a conversation preserves the task; stopping follow-up preserves its history and still resolves an action already in progress.

4 Personalisation and information

An owner should be able to improve how the assistant works through an ordinary correction. "Show cash and stock first" is a presentation preference. "Compare quotations by unit price, and keep the pack size and source" changes the procedure used to process a file. ShopSteward will retain these changes with their intended use.

A correction that survives the next task

In a constructed quotation example, the first file lists 120 CU for a box of 12 units and a minimum order of two boxes. The owner asks for comparison by unit price while retaining pack size, minimum order and source. The assistant should show 10 CU per unit and propose a saved procedure for making that conversion.

A later conversation supplies a different item: 180 CU per box of 15, with a minimum order of three boxes. Applying the procedure must produce 12 CU per unit from the new file. The reusable element is the processing method; the new price and quantities come from the new input.

A skill is a saved procedure defining when it applies, which tools to use and what the output must contain. An explicit correction creates a version linked to its source task. Before activation, the system checks the revision against the corrected example and a missing-field case. Subsequent runs record the selected version and their tool results. Such revisions cannot change purchasing permissions or the cash reserve.

Information with distinct roles

The business ledger supplies current cash, stock and outstanding orders. Reference documents retain evidence such as a supplier's pack size or terms, with the source and version available for inspection. Preferences describe the owner's requested presentation; skills describe processing steps. A document or remembered statement cannot silently replace a ledger value.

The core quotation workflow uses comma-separated values (CSV) files containing currency, price basis, pack size, minimum-order units and source row. Missing pack size blocks conversion of a box price; an explicit unit price can still be reported. Processing a quotation produces a traceable result without updating stock or submitting an order. Broader document retrieval can be evaluated separately from this workflow.

Data preparation

A controlled simulator supplies versioned initial states and dated purchase, delivery, sales and demand-update events. Demand is a scenario input; this proposal does not depend on training a forecasting model. Quotation files and reference answers will be constructed separately, checked for units and missing fields, and split into development and held-out sets. Future events become visible only when released. Reference answers remain separate from the assistant's input and are used only by the evaluator.

5 Purchasing across several periods

Buying the right total quantity still leaves a timing decision. An early order protects availability but holds stock and commits cash sooner. A later order can preserve flexibility, provided it arrives before the demand. The proposed planner will compare these consequences over several decision periods.

A simple timing example

Consider a separate three-day illustration with 20 units in stock, 1,000 CU cash, a 300 CU reserve and a unit purchase cost of 10 CU. Demand is 10, 10 and 40 units. Orders arrive before sales the following day; sales proceeds remain unsettled. Both plans below serve all 60 units.

Purchase plan	Cash after day 1	Closing stock on days 1 2 3	Stock carried
Order 40 on day 1	600 CU	10, 40, 0	50 unit-days
Order 40 on day 2	1,000 CU	10, 0, 0	10 unit-days

Table 2 Calculated illustration, not algorithm output. Stock carried is the sum of closing daily inventory; ten units held for one day are ten unit-days.

The planner first minimises projected unmet demand, then stock carried, with smaller total purchase quantity breaking remaining ties. Every candidate must preserve the cash reserve and respect delivery timing. Existing orders remain committed; only the next purchase is recommended for confirmation. Later choices are reconsidered as new information arrives.

Why compare search with simpler methods

Method	Purpose in the comparison	Limitation
Daily replenishment rule	A cheap, explainable next-order baseline.	Does not retain alternative purchase sequences.
Exhaustive enumeration	An exact reference under shared planning assumptions.	Sequence count grows with the horizon and permitted quantities.
Beam search	A bounded search over alternative partial plans.	May discard the best plan; ranking and beam width matter.

Beam search is an informed search strategy covered in Machine Reasoning [8]. At each period it expands permitted quantities, rejects constraint violations and completes each partial plan with the daily rule to estimate the consequences. It retains a limited number of the best-ranked partial plans. Beam width is selected on development cases.

Four quantities over seven days produce 16,384 sequences before pruning, small enough to support an exact reference comparison. This experiment will test the quality and runtime tradeoff of bounding the search, rather than presume that search beats a simple rule. Research on inventory control motivates examining language models and operations research together [7]; their effects will be measured separately here.

6 Evaluation questions and measures

The central test is whether the owner can leave a delegated task and return to work that has been carried out correctly. A test episode follows one task to a specified endpoint, including any required waiting, confirmation and resolution of pending actions. The fixed states in Table 1 and the quotation examples establish correctness before comparative evaluation.

Measure	Definition and interpretation
Task success rate	Successful episodes / attempted episodes. All expected checkpoints, a retrievable result and no execution violation are required.
Unmet demand	Sum of daily max(0, realised demand minus stock available after receipts and before sales). Unserved units are lost sales.
Stock carried	Sum of closing daily inventory in unit-days, assessed alongside unmet demand.
Execution violations	Unauthorised, repeated or reserve-breaching purchases. Required count in the acceptance suite is zero.
Quotation accuracy	Correct required fields / evaluated fields. Decimal unit prices are checked exactly; a genuinely missing field marked unavailable is correct.
Planning gap and cost	Excess projected unmet demand versus enumeration; on a tie, excess projected unit-days. Record runtime, model calls, tokens and cost per episode.

Comparisons that isolate the contribution

For planning, a scripted controller compares the daily rule, Beam search and enumeration using identical visible demand assumptions and daily replanning times. The daily rule chooses the feasible quantity with the least forecast shortage before the next daily order could arrive, counting stock already in transit; ties use the smaller quantity.

For response timing, the planner is held fixed at Beam search. Daily replanning is compared with daily replanning plus a check when a demand or delivery update arrives. A scripted controller isolates this timing effect from language-model behaviour. Both conditions enforce the same confirmation and execution rules.

For the assistant, completion and recovery checks compare the model-driven workflow with a scripted workflow using the same business tools, storage and scheduler. The checks include a real timed wake-up without a new user message, a business event, a new conversation and a process restart. Prescribed waiting and the absence of a duplicate purchase are part of success.

For procedure reuse, the same model and tools process new quotations with and without the revised skill, while display preferences remain fixed. Each file starts an independent task with identical base context. The control excludes the correction conversation and any memory containing that revision. The skill is frozen before held-out testing, with no learning between test files.

7 Study protocol and delivery

The proposed evaluation budget is 30 independent seven-day business scenarios: ten under stable conditions, ten with a demand revision and ten with a delivery delay. Initial stock, usable cash, demand and event times will vary. A further 30 held-out quotation files will cover box prices, unit prices and missing fields. Five development scenarios and separate quotation examples are excluded from final results.

These are planning targets. A development pilot will establish feasible runtime and model cost; the final sample sizes, beam width and configurations will then be frozen before held-out evaluation. The evaluation record will preserve that specification alongside software, model, prompt, data and skill versions.

A shared environment

Each simulated day begins with receipts and scheduled planning. Demand or delivery updates are then released, followed by the extra check in the event-driven condition and then sales. Future information is withheld until release. A delivery disruption applies through a shared supplier time window to affected outstanding and newly placed orders. Its notification precedes each affected receipt, so different policies face the same supply conditions.

All policies share the initial conditions, external shocks, permitted quantities and cash rules. The batch evaluator approves only current proposals that pass cash, version and uniqueness checks; the interactive demonstration requires the owner's explicit confirmation. Model-based conditions are planned for three repeats, averaged within each case. Paired differences will use 95% bootstrap intervals over independent cases, not individual repeats. Timeouts count as failures, and costs include retries.

What the evidence can establish

A successful demonstration would show a task surviving the end of a conversation, reacting appropriately to a change and preserving the owner's control over purchases. A successful procedure test would show a correction changing a later result on genuinely different input. Comparative experiments will determine whether these behaviours improve outcomes enough to justify their cost; an inconclusive or unfavourable result remains informative.

The simulation makes purchases and failures observable without a live merchant account. It simplifies operations to non-perishable stock with known unit prices and scenario-supplied demand. Sales proceeds remain unsettled over the evaluation horizon. Closing stock, pending deliveries and unsettled proceeds stay visible at the endpoint; closing cash is not treated as profit. Real merchant income, adoption and willingness to delegate require later field evidence.

Proposed delivery

Development will first establish the purchasing and follow-up path, then verify correction reuse on new quotation files, before running the controlled comparisons. The final package will contain a runnable system, a report with methods and findings, reproducible configurations and a user guide, together with the course's two five-minute videos covering business use and system design [1].

The project will demonstrate what it takes for a merchant to entrust an assistant with continuing work: a goal that remains active, decisions grounded in current records, and a clear account of what happened when the owner was elsewhere.

References

[1] NUS ISS. IRS practice module and exam briefing, v2.17, 2026. Technique groups and final delivery requirements, PDF pp. 6 and 9-14. Course briefing

[2] NUS ISS. IRS practice module project proposal and final presentation guidelines, v016, 2026. Proposal content and optional sections, PDF pp. 2-14. Proposal guidelines

[3] Enterprise Singapore. Business Productivity Solutions. Heartland enterprise support and examples of business systems. Accessed 8 September 2026. Enterprise Singapore

[4] Forthcast. State of Shopify Inventory Operations 2026. Published 18 July 2026; interviews conducted February-March 2026. Study and methodology

[5] Mavadia, A. Managing inventory in Shopify just got easier. Shopify, 17 June 2026. Shopify inventory workflow

[6] Sujith, A. Planning with the Demand to Supply to Buy flow. Prediko Help Center, 25 June 2026. Prediko planning workflow

[7] Baek, J., Fu, Y., Ma, W. and Peng, T. AI Agents for Inventory Control: Human-LLM-OR Complementarity. arXiv:2602.12631v2, 4 May 2026. Research paper

[8] NUS ISS. Machine Reasoning Day 1, JR v1, 2026. Informed search strategies and evaluation functions, PDF pp. 46-47 (slide labels 51-52). Course handout.

AI assistance

AI tools assisted source review, drafting, language editing and production of the diagram and document layout. The submitting team is responsible for checking citations, validating the system and explaining the work. Presentation speech will comply with the course restriction on AI-generated speech [1].

Appendix B Mapped System Functionalities against MR, RS and CGS

This appendix maps each functionality of ShopSteward to the knowledge, techniques and skills of the three courses of the certificate: Machine Reasoning (MR), Reasoning Systems (RS) and Cognitive Systems (CGS). Table 2-1 gives the summary by technique group; the tables below go down to the course topic, named as in the course slides with the day on which it was taught, and point to the section of this report that describes the implementation. Section B.4 lists the techniques the team learned beyond the courses.

B.1 Machine Reasoning (MR)

Table B-1. ShopSteward functionalities mapped to Machine Reasoning.

Course topic	ShopSteward functionality	How the system applies it	Section
Day 1 · Reasoning as search: problem formulation (states, actions, goal, cost)	Routine planner; recovery solver	The planner searches candidate order quantities from the current inventory position; the recovery solver searches supplier and quantity pairs. Both rank complete outcomes by an explicit lexicographic cost	2.4, 2.5
Day 1 · Informed search: evaluation functions, greedy search, beam search	Multi-day planner	Beam search over sequences of daily orders. Each partial plan is scored by completing it with the greedy daily rule, which serves as the evaluation function; the beam is compared with the greedy rule itself and with exhaustive enumeration	2.4, 4.3
Day 1 · Constraint satisfaction	Feasibility of candidate orders	Every candidate must satisfy the cash floor, the supplier's minimum order, the pack-size multiple, the arrival window and the owner's quantity cap; only consistent candidates are ranked	2.4
Day 2 · Knowledge-based reasoning with rules; forward chaining	Event-driven re-planning; evidence policy	Each new business fact (a sale, a delivery, a demand revision) fires the planning rules on the updated state. The evidence policy derives, from the wording of a request, the tools that must be called before the agent answers	2.2, 2.8
Day 2 · Knowledge representation: frames and slots	Task frame; quote cards	The task frame holds the owner's constraints as typed slots (budget, quantity limits, hypothetical quantity, cancellation), each with the quoted words. A quote card is a frame per supplier offer with slots for status, unit price, minimum order, pack size, lead time, validity and version	2.6, 2.9

Course topic	ShopSteward functionality	How the system applies it	Section
Day 2 · Knowledge acquisition; rule induction	Experience learning	Skills are induced from operation episodes (at least three, across at least two business days) as rules with preconditions, exclusions and a procedure, and are enabled by the owner	2.11.3
Day 3 · Reasoning over knowledge graphs: triples, multi-hop path queries	Business relation graph	Directed relations between documents, products and suppliers (applies to, supersedes, supplies) stored as edges; one- to three-hop path queries return the related documents	2.10, 4.10
Day 4 · Inductive learning: regression, ensembles, boosting, cross-validation, overfitting	v6 demand forecast	Three gradient-boosted LightGBM regressors with different losses, blended with an MLP; models selected on three development anchors with early stopping and judged on eight later weeks	2.3, 4.2
Day 4 · Neural network learning: multi-layer perceptron	v6 forecast component	An MLP predicts the seven daily values directly, beside the weekly-total tree models	2.3

B.2 Reasoning Systems (RS)

Table B-2. ShopSteward functionalities mapped to Reasoning Systems.

Course topic	ShopSteward functionality	How the system applies it	Section
Day 1 · Hybrid reasoning systems: rules + ML integration	Restocking pipeline	The learned forecast supplies demand; business rules decide which orders are feasible; search ranks the feasible ones	2.1, 2.4
Day 1 · Search + optimisation systems	Recovery solver	Simulation-based optimisation: each candidate order is simulated day by day over seven days, lost sales are counted, and candidates are ranked by lost demand, then spend, then closing stock	2.5
Day 1 · RAG + LLM architectures; reasoning systems retrieve before answering	Evidence policy; agentic RAG	The agent must read the dashboard, forecast, plan or documents through its tools before it answers; document search is a tool that returns cited passages	2.8, 2.10
Day 1 · Agentic orchestration patterns	Multi-agent sourcing; expert team	Orchestrator–worker sourcing with one ReAct reviewer agent per supplier; an explanation layer with four orchestration modes, from one fixed call to an adaptive team of role experts	2.6, 2.7, 4.6, 4.8

Course topic	ShopSteward functionality	How the system applies it	Section
Day 1 · Explainability and accountability	Candidate tables, quoted evidence, context manifests	Every plan keeps its candidates and the rule that removed each rejected one; quote-card fields carry the excerpts they were read from; each model call records what entered its context	2.4, 2.6, 2.9
Day 1 · Human oversight (human-in-the-loop)	Approval queue; clarification; assisted ordering	The owner approves every purchase, bound to the exact plan shown; a clarification pauses the agent run until the owner answers; the owner sends supplier orders	2.2, 2.6, 2.8
Day 1 · Failure points of reasoning components	Citation verification with escalation	Each quoted excerpt is matched against the supplier document; a card that fails is read again, optionally by a stronger model	2.6
Day 2 · Data-driven reasoning: knowledge discovered from data	Forecasting; experience learning	Demand knowledge is learned from sales history and procedural knowledge from operation episodes, in place of hand-written expert rules	2.3, 2.11.3
Day 4 · Optimisation: decision variables, objective, constraints	Recovery solver formulation	Decision variables: supplier and quantity. Objective: lost demand, then spend, then closing stock. Constraints: budget, minimum order, pack multiples, at most three quotes	2.5
Day 4 · Evolutionary computation and genetic algorithms: population, fitness, selection, mutation	GEPA prompt evolution	A pool of candidate instruction texts; fitness is the code-scored result on each validation case; parents are selected from the per-case Pareto front; a reflection model mutates one component at a time	2.11.2, 4.9
Day 5 · Paradigms of AI: symbolic, machine learning, multi-agent	Overall architecture	Symbolic rules and search, learned forecasting and language models, and multi-agent sourcing work on one shared business state	2.1
Day 5 · Graph-enabled RAG	Hybrid retrieval with a relation graph	Keyword, dense and graph channels are fused by reciprocal rank fusion, and matching passages are expanded with their neighbours	2.10

B.3 Cognitive Systems (CGS)

Table B-3. ShopSteward functionalities mapped to Cognitive Systems.

Course topic	ShopSteward functionality	How the system applies it	Section
Day 1 AM · Cognitive	The agent as a	It perceives business events and	2.1

Course topic	ShopSteward functionality	How the system applies it	Section
systems: perceive, reason, learn, decide	whole	documents, reasons with the planner, the solver and language-model agents, learns through prompt evolution, post-training and experience, and decides with the owner's approval	
Day 1 AM · Natural language understanding: intents and slots	Ordering decision; task frame	An owner's request is mapped to one of five actions (the intent) with exact arguments (the slots); the task frame extracts constraint slots from the owner's words	2.8, 2.9, 4.4
Day 1 AM · Dialogue management	Clarification	When a required value is missing or ambiguous, such as a quantity given in boxes, the agent asks one question and resumes the same run with the answer	2.8
Day 1 AM · Text vectorisation: TF-IDF and embeddings	Retrieval channels	BM25 term weighting for the keyword channel and Qwen3 embeddings for the dense channel	2.10
Day 1 AM · Language generation: templates and language models	Order messages; explanations	Supplier order messages are generated from templates; explanations are generated by a language model from the solver's result as typed claims that cite their evidence	2.6, 2.7
Day 1 AM, Day 3 · RAG and tool-using agents	Agentic RAG; conversational agent	Document search and evidence reading are tools in a LangGraph tool loop; answers cite the retrieved passages	2.8, 2.10
Day 2 · Reasoning in large language models and its failure types	Neuro-symbolic division of labour	Language models read, interpret and explain; every quantity and amount of money is computed by algorithms; reasoning effort is configured per model role	2.1, 3.3
Day 3 · Cognitive architecture: memory, reasoning, learning, deliberation, reaction, metacognition	Runtime design	Reaction: event-triggered checks. Deliberation: planner search and the recovery solver. Memory: long-term stores for preferences, notes and skills, and a layered working context. Learning: GEPA, GRPO and experience. Metacognition: reflection on the system's own scored failures	2.2, 2.9, 2.11
Day 3 · Knowledge representation and reasoning: frames, semantic networks, knowledge graphs	Representations in use	Frames for the task frame and quote cards, a knowledge graph of business relations, and versioned memory entries	2.6, 2.9, 2.10

B.4 Techniques learned beyond the courses

Table B-4. Techniques the team learned through its own research.

Technique	Where it is used	Section
LangGraph state machine with durable PostgreSQL checkpoints, interrupts and leased workers	Agent runtime that survives restarts and waits for the owner	2.8
Context engineering: layered context assembly with a byte budget and a manifest per call	Every language-model call	2.9
Orchestrator–worker agents with verified citations and a model cascade	Multi-agent sourcing	2.6
Supervised fine-tuning with LoRA, then GRPO with rewards from executed actions	Post-training Qwen3-8B on the ordering decision	2.11.1, 4.5
GEPA reflective prompt evolution over versioned text bundles	Instructions of the explanation and sourcing steps	2.11.2, 4.9
Hybrid retrieval with reciprocal rank fusion, small-to-big expansion and versioned index generations	Knowledge service	2.10
Tweedie and L1 objectives with intermittent-demand features for slow-moving items	v6 forecast	2.3

Appendix C Installation and User Guide

This guide installs ShopSteward from its repository on a Windows machine, starts it with the demonstration configuration, and walks through the four scenarios of Section 3.4. The code is in the SystemCode folder of the repository; commands are run in PowerShell from that folder unless stated otherwise. The same dependencies install on macOS and Linux; Section C.5 gives the per-process start commands for those systems.

C.1 Requirements

Table C-1. Software and accounts.

Item	Version or note
Git	any recent version
Python	3.12, managed with **uv**
Node.js	22, with **Corepack** (the repository pins pnpm 10.17.1)
Docker Desktop	running; it hosts the business PostgreSQL
OpenAI API key	for the language-model features; the demonstration configuration uses gpt-6.1-sol in every role
Free ports	3000 (web), 8000 (business API), 8001 (simulator), 55432 (PostgreSQL); 8053 if the forecast service is enabled

Model calls consume the API usage of the configured account. The routine restocking flow (S1 without the agent conversation) runs without any model.

C.2 Installation

```
git clone --branch submission https://github.com/yhnapoleon/ShopSteward.git
Set-Location ShopSteward/SystemCode
uv sync --frozen --all-packages --all-groups --python 3.12
corepack pnpm install --frozen-lockfile
```

`uv.lock` and `pnpm-lock.yaml` pin every Python and frontend dependency. To use the v6 forecast service, install the optional machine-learning extras as well (Section C.4).

C.3 Configuration and databases

Create the local settings files from their templates:

```
foreach ($m in 'infra', 'backend', 'simulation', 'frontend') {
    if (-not (Test-Path "$m/.env")) { Copy-Item "$m/.env.example" "$m/.env" }
}
```

Fill in the fields of Table C-2. Each credential is an independent random value; `[guid]::NewGuid().ToString('N')` produces one.

Table C-2. Required settings.

File	Fields
`infra/.env`	`POSTGRES_PASSWORD`; the port defaults to `POSTGRES_PORT=55432`

File	Fields
backend/.env	DATABASE_URL with the same password and port; AUTH_TOKENS with a local user (below); SIMULATION_TOKEN equal to the simulator's service token
simulation/.env	SIM_DATABASE_URL pointing to a separate shopsteward_sim database; SIM_SERVICE_TOKEN of at least 24 characters
frontend/.env	NUXT_BACKEND_URL=http://127.0.0.1:8000 (the Windows launcher sets the rest)

The business database URL has the form postgresql+asyncpg://shopsteward:<password>@127.0.0.1:55432/shopsteward. A local administrator is one line in backend/.env:

```
AUTH_TOKENS=[{"token":"<random user token, at least 24 characters>","principal_id":"local-admin","kind":"user","roles":["admin"],"store_ids":[]}]
```

Start PostgreSQL and create the simulator's database. Compose creates the business user and database; the simulator's role and database are created once by hand, with the password used in simulation/.env:

```
docker compose --env-file infra/.env -f infra/compose.yaml up -d --wait
docker compose --env-file infra/.env -f infra/compose.yaml exec postgres `
    psql -U shopsteward -d postgres
```

```
CREATE ROLE shopsteward_sim LOGIN;
\password shopsteward_sim
CREATE DATABASE shopsteward_sim OWNER shopsteward_sim;
\q
```

Apply the migrations from each module's directory, so that each reads its own .env:

```
Set-Location backend
..\.venv\Scripts\python.exe -m alembic upgrade head
Set-Location ../simulation
..\.venv\Scripts\python.exe -m alembic upgrade head
Set-Location ..
```

The business database should report 0020_business_time and the simulator sim_0003_controls (python -m alembic current).

C.4 Models and the forecast service

Demonstration configuration. infra/demo-mature.env.example holds the model settings used for the demonstration: gpt-6.1-sol through the Responses API for the conversational agent, the explanation step and the supplier reviewers, with the work-item and learning workers switched on. Copy it, for example to var/demo.env, and replace the two <path to the OpenAI key file> placeholders with the absolute path of a text file that contains only the API key. The file is laid over backend/.env at start-up (Section C.5), so backend/.env itself stays unchanged.

v6 forecast service (optional). The trained model bundle ships with the repository. Install the extras, check the bundle, and enable the service in backend/.env:

```
uv sync --frozen --all-packages --all-groups --all-extras --python 3.12
.venv/Scripts/python.exe ml/tools/reproduce_v6.py
```

```
FORECAST_V6_ENABLED=true
FORECAST_V6_URL=http://127.0.0.1:8053
FORECAST_V6_TOKEN=<random service token>
```

The check prints `verification: PASS`. Scenarios S1–S4 use the demand stated in each scenario and do not need the forecast service.

Knowledge service (optional). Document retrieval runs as a separate service with its own PostgreSQL and OpenSearch, deployed with `infra/compose.knowledge.yaml` and the knowledge runbook in `docs/runbooks/`. Scenarios S1–S4 do not use it.

C.5 Starting and stopping

Windows. One command starts the simulator, the business API, the business worker, the web frontend and, according to the settings, the agent, work-item and learning workers and the forecast service, and waits until each is ready:

```
.venv/Scripts/python.exe infra/start_windows.py --overlay var/demo.env
```

Open **http://127.0.0.1:3000**. The launcher writes its logs and the IDs of the processes it started to `var/windows-stack-<time>/processes.json`. To stop them:

```
$run = Get-ChildItem var -Directory -Filter 'windows-stack-*' |
    Sort-Object Name | Select-Object -Last 1
$record = Get-Content "$($run.FullName)/processes.json" -Raw | ConvertFrom-J
son
$record.processes | ForEach-Object { taskkill /PID $_.pid /T /F }
```

macOS and Linux, or one terminal per process. Activate the environment in each Python terminal (`source .venv/bin/activate`) and run:

Table C-3. Processes and their start commands.

Process	Directory	Command
Simulator	`simulation/`	`python -m simulator`
Business API	`backend/`	`python -m app`
Business worker	`backend/`	`python -m app.worker --profile business`
Agent worker	`backend/`	`python -m app.worker --profile agent`
Work-item worker	`backend/`	`python -m app.worker --profile work`
Learning worker	`backend/`	`python -m app.worker --profile learning`
Web frontend	SystemCode	`corepack pnpm --filter @shopsteward/frontend dev --host 127.0.0.1 --port 3000`

In this mode the model settings go into `backend/.env` directly, and `frontend/.env` also needs `NUXT_BACKEND_TOKEN` (the administrator token), `NUXT_DEV_TOOLS=true` and `NUXT_AGENT_ENABLED=true`.

C.6 Using the system

First steps.

1. Open http://127.0.0.1:3000. If the page asks for an identity, choose **Connect backend identity**.
2. Open **Settings → Language and region → Display language** and choose **English**.
3. In **Settings**, turn on **Demo mode**. It hides debugging labels and shows the presenter bar, whose **Next day** advances the simulated business by one scripted step and whose **New sample store** creates a fresh store.
4. Choose **Create sample store**. The sample store is a synthetic shop with 20 units of one product in stock, ¥1,000 in cash and an expected demand of 60 units.

The workspace. Today shows what needs a decision; **Follow-up** lists the delegated restocking tasks; **Activity** records business changes, the owner's decisions and the ledger; **Overview** charts cash and sales; **Document library** holds business documents. **Give me a task** hands the system a piece of work, such as processing a quotation, and the **Agent conversation** of a task answers questions and proposes changes. Every purchase waits for the owner's approval.

S1 Promotion restocking with follow-up.

1. On **Today**, open **Try a restocking comparison**, keep the cash floor at ¥300 and the quantities 0, 20, 40 and 80. The table recommends 40 units and marks 80 as leaving cash below the floor (Figure 3-4).
2. Choose **Start with these conditions** to turn the comparison into a follow-up task, open **Review 40-unit purchase**, check **View evidence**, and confirm the purchase of 40 units.
3. Press **Next day** on the presenter bar until the 40 units arrive, 10 units are sold and demand is revised from 60 to 70. A change card proposes 20 more units (Figure 3-5); approve it.
4. In the **Agent conversation**, ask the agent to explain the recommendation, then open **View context record** to see what entered the model's context.

S2 Supplier failure, multi-agent sourcing and the owner's order.

1. Create a second store with **New sample store** and open **Emergency restock**. Describe the shortage (about 10 units a day for a week) with a budget of ¥300.
2. Confirm and let the system find suppliers. One reviewer agent reads each sample supplier's documents; the comparison then recommends an option and shows each supplier's card with the quoted excerpts (Figure 3-6).
3. Adopt the recommended option and approve it.
4. Copy the generated order message. Paste a reply as the supplier would send it, stating the quantity, unit price, total and delivery date of that order; the system reads it into fields and shows the words each field came from (Figure 3-7).
5. Choose **Record arrival** to add the goods to stock.

S3 Quotation correction and reuse.

1. Open **Give me a task**, start a quotation task and attach sample quote A (¥120 per box of 12, minimum two boxes). The result shows ¥10 per unit and a minimum of 24 units.
2. Ask for quotations to be compared by unit price and choose to update and remember the sort order. The rule is saved as version 1.
3. Attach sample quote B (¥180 per box of 15, minimum three boxes). It is processed with the saved rule: ¥12 per unit and a minimum of 45 units (Figure 3-8).

S4 Memory, records and the business brief.

1. In the **Agent conversation**, write “Remember: show me cash and stock first.” The conversation confirms the update, and **What the system remembers** lists the preference with its version (Figure 3-9).
2. Open **Activity** to see the business changes, decisions and ledger of S1, and open the **Business brief** to read or download a summary of recent events, actions and next steps.

C.7 Troubleshooting

Table C-4. Common problems.

Symptom	Check
The API reports live but not ready	database connection; the migration head (`python -m alembic current` in `backend/`)
Setting up a store or a task stays queued	the business worker and the simulator are running, and the two service tokens match
The page shows no identity, or 401 errors	`AUTH_TOKENS` in `backend/.env`; reconnect the backend identity
The agent does not answer	`AGENT_ENABLED`, the key file path, the API mode and the agent worker
A quotation task is saved but never processed	`WORK_PROCESSOR_ENABLED` and the work-item worker
The forecast shows as unavailable	`FORECAST_V6_ENABLED`, the service on port 8053 and its token
The launcher reports a port in use	stop the earlier run (Section C.5) before starting again